

**National University of Ho Chi Minh City**  
**HCM University of Technology**  
**Computer Science and Engineering Department**



**REPORT**  
**CAPSTONE PROJECT HK253-DATN-011**

---

**RESEARCH AND DEVELOPMENT A SMART  
QUERY SOLUTION USING KNOWLEDGE  
GRAPH AND LANGUAGE MODELS**

---

Major: Computer Science

**Thesis Committee:** Council 4CC Computer Science

**Instructor:** Assoc. Prof. Thoại Nam

**Secretary:** Dr. Nguyễn Lê Duy Lai

—o0o—

**Student:** Vũ Hoàng Tùng (2252886)

HỒ CHÍ MINH City, September 2026

# Acknowledgement

This section is dedicated to express our sincere gratitude towards all those who have played a crucial role in the completion of the Specialized Project, also the first phase of the thesis. Your supports have been a great help for me to perform various steps from the fundamental researches and requirement specification to system design and implementation development, laying foundations for the future Capstone Project

I would like to show my foremost appreciation to my supervisor, Mr. Thoai Nam for his heart-warming support, enthusiastic guidance and constructive feedback. His invaluable expertise and advice has really strengthened my vision and understanding, thus been instrumental in shaping the direction of my work.

I am also greatly thankful to Computer Science Faculty of Department for the learning environment, the facilities, the knowledge and the opportunity to orchestrate this work that I'm proud of. The contributions of my peers and colleagues in engaging discussions and shared insights have been enriching.

To Bảo, whose knowledge and engaging discussion on system design has been a great help, I show my appreciation for his passionate and knowledgeable sharing and unwavering support.

To all my family and friends who offered encouragement, shared their knowledge and gave inspiration, I feel blessed that this Project and me have been supported by your love and care.

Last but not least, I acknowledge that this my work would not achieve the goal without the foundation laid by pioneering researchers and engineers, both in the field of artificial intelligence and education. The theories, methodologies, and technologies they have developed have been the bedrock upon which this project is built. The knowledge and inspiration they give me is invaluable, and I am more than honored to stand on the shoulders of giants.

This work stands firm thanks to the contribution of all those mentioned above. Your accumulated impact has been indispensable, and I cherish the roles each played in this academic endeavor.

# DECLARATION OF AUTHENTICITY

I declare that this specialized project entitled "*Research and Development a smart query solution using Knowledge Graphs and Language Models.*" was entirely composed and implemented by myself under the guidance and supervision of Assoc. Prof. Thoại Nam at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

Throughout the project, I ensure that all references and external sources utilized are appropriately cited and thoroughly documented. My source code implementation also have detailed comments on part that take reference from open sources. My source code: <https://github.com/ttkb-hcmut/SmartEdu>

Ho Chi Minh City, September 2026

**The author,**

*Vũ Hoàng Tùng*

# Abstract

Current world is changing at an incredible pace with constant information and knowledge update, while many educational systems remain stagnant. The rapid evolution of Artificial Intelligence has open doors for adaptive, self-directed learning online systems, replacing Learning Management Systems that offer static contents. However, common AI techniques such as Retrieval-Augmented Generation (RAG) struggle with static content, semantic fragmentation, and a lack of profound reasoning, while LLMs have limited context window and hallucination, inconsistent output.

To address these limitations, this project introduces **SmartEdu**, a proactive, mastery-oriented educational ecosystem powered by a graph-grounded multi-agent architecture. By operating LLM on deterministic ingesting pipeline for academic materials, the system constructs an Automated Educational Knowledge Graph (EKG) that accurately maps conceptual hierarchies and prerequisite dependencies. Unstructured multimodal materials are automatically ingested via an asynchronous Producer-Consumer pipeline and processed through a novel Dual-Phase LLM extraction technique. This is reinforced by deterministic NLP filtering—including DBpedia validation and implicit clustering—to ensure strict ontological hierarchy and prerequisite acyclic constraints.

For real-time tutoring, a LangGraph-driven Educational QA Agent operates within an Adaptive Harness, strictly separating reasoning contexts and action schemas. It utilizes Knowledge Graph as a wellstructured ground truth and solid reasoning frame, cooperates open-weights Small Language Models (SLMs) into workflow for localized and privacy-preserving execution, and orchestrates specialized multi-agent workflows—Retrieve, Roadmap, and Teach—to perform dynamic student mastery tracking, automated gap detection using topological algorithms, and personalized learning pathfinding, all while safeguarding student privacy and important resources through a robust Identity-Session dynamic allocation and management.

Initial evaluations of the system demonstrate successful and consistent AI operations, effectively reduce hallucinations, produce consistent results and reasoning, and maintaining structural integrity across complex academic domains. Despite current latency bottlenecks and challenges in macro-level course pathing, SmartEdu establishes a robust, highly scalable foundation for shifting the educational paradigm from linear content delivery to interactive, human-like mentorship.

The project is being actively maintained on Github: <https://github.com/ttb-hcmut/SmartEdu>

# Contents

|          |                                                                  |           |
|----------|------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                              | <b>1</b>  |
| 1.1      | Motivation . . . . .                                             | 1         |
| 1.2      | Goals . . . . .                                                  | 2         |
| 1.3      | Scope . . . . .                                                  | 3         |
| 1.4      | Thesis Structure . . . . .                                       | 3         |
| <b>2</b> | <b>Fundamental Knowledge &amp; Related Work</b>                  | <b>4</b>  |
| 2.1      | Fundamental Knowledge . . . . .                                  | 4         |
| 2.1.1    | Natural Language Processing . . . . .                            | 4         |
| 2.1.2    | Large Language Models (LLMs) . . . . .                           | 6         |
| 2.1.2.1  | LLM Deployment: Proprietary vs Open-Weights . . . . .            | 7         |
| 2.1.2.2  | LLM Hyper-parameters and Inference Control . . . . .             | 7         |
| 2.1.3    | Agentic Systems . . . . .                                        | 8         |
| 2.1.3.1  | AI Agents . . . . .                                              | 8         |
| 2.1.3.2  | Agent Frameworks . . . . .                                       | 10        |
| 2.1.3.3  | Langchain and LangGraph . . . . .                                | 10        |
| 2.1.4    | System Design . . . . .                                          | 11        |
| 2.1.5    | Technology Stack and Frameworks . . . . .                        | 13        |
| 2.1.6    | Database System Design & Management . . . . .                    | 16        |
| 2.1.7    | Deployment and Containerization . . . . .                        | 20        |
| 2.1.8    | Security: Token-Based Authentication and Authorization . . . . . | 21        |
| 2.2      | Related Work . . . . .                                           | 21        |
| 2.2.1    | Document-Knowledge Modeling . . . . .                            | 21        |
| 2.2.2    | Generative AI in Interactive Teaching . . . . .                  | 24        |
| 2.2.3    | Educational Intelligent System . . . . .                         | 25        |
| 2.2.4    | Conclusion and Research Gaps . . . . .                           | 26        |
| <b>3</b> | <b>Requirements Specifications</b>                               | <b>28</b> |
| 3.1      | Functional Requirements . . . . .                                | 28        |
| 3.1.1    | Knowledge Modelling and Management . . . . .                     | 28        |
| 3.1.2    | Intelligent Teaching Assistance . . . . .                        | 29        |
| 3.1.3    | Student Interaction and System Navigation . . . . .              | 29        |
| 3.2      | Non-Functional Requirements . . . . .                            | 30        |
| 3.3      | Use Cases . . . . .                                              | 31        |
| 3.3.1    | Use case 1: Register Account . . . . .                           | 32        |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 3.3.2    | Use case 2: Login/Logout                         | 33        |
| 3.3.3    | Use case 3: Manage Learning Session              | 34        |
| 3.3.4    | Use case 4: Ingest Course Material               | 35        |
| 3.3.5    | Use case 5: Chat with EQA                        | 36        |
| <b>4</b> | <b>Proposed Methodology</b>                      | <b>37</b> |
| 4.1      | Overview                                         | 37        |
| 4.2      | Knowledge Modeling                               | 37        |
| 4.2.1    | Graph Definition                                 | 38        |
| 4.3      | Content Processing and Ingestion                 | 39        |
| 4.3.1    | BookRAG and Docling Integration                  | 40        |
| 4.3.2    | Input Ingestion                                  | 40        |
| 4.3.3    | Dual Phase LLM Extraction                        | 40        |
| 4.3.4    | NLP-Based Concept Filtering and Verification     | 42        |
| 4.4      | Agentic System                                   | 43        |
| 4.4.1    | Agent Harnessing                                 | 43        |
| 4.4.1.1  | Context Engineering and Management               | 44        |
| 4.4.1.2  | Tools                                            | 45        |
| 4.4.1.3  | Scope-based Harnessing                           | 46        |
| 4.4.2    | Agent Orchestration                              | 46        |
| 4.4.2.1  | Workflow: Retrieve (GraphRAG)                    | 46        |
| 4.4.2.2  | Workflow: Teach (Agentic Long Memory)            | 48        |
| 4.4.2.3  | Workflow: Roadmap (Heuristic Graph)              | 48        |
| 4.5      | Self-Evaluation Methodology                      | 49        |
| 4.5.1    | Benchmark Selection                              | 49        |
| 4.5.2    | Metrics                                          | 49        |
| 4.5.3    | Results                                          | 50        |
| <b>5</b> | <b>System design and implementation</b>          | <b>52</b> |
| 5.1      | System Architecture                              | 53        |
| 5.1.1    | Overall Architecture and Module Responsibilities | 53        |
| 5.1.2    | Technical Setup: Docker Infrastructure           | 55        |
| 5.1.3    | Design Paradigm                                  | 55        |
| 5.2      | Core Module                                      | 57        |
| 5.2.1    | LLM Engine                                       | 57        |
| 5.2.2    | Configuration Management                         | 59        |
| 5.3      | Database Design                                  | 59        |
| 5.3.1    | Polyglot Persistence Design                      | 60        |
| 5.3.2    | Cross-Database Reference and Consistency         | 63        |
| 5.4      | Student Module                                   | 64        |
| 5.5      | Knowledge Module                                 | 65        |
| 5.5.1    | Ingestion Workflow                               | 66        |
| 5.6      | Educational QA Agent Module                      | 68        |
| 5.6.1    | Technical Setup                                  | 68        |

|          |                              |           |
|----------|------------------------------|-----------|
| 5.6.2    | LangGraph Workflow Design    | 69        |
| 5.6.3    | Request Lifecycle            | 70        |
| 5.6.4    | Tracing and Observability    | 70        |
| 5.7      | Frontend Module              | 72        |
| <b>6</b> | <b>Experiment</b>            | <b>73</b> |
| 6.1      | Application Deployment       | 73        |
| 6.1.1    | Database Setup               | 73        |
| 6.1.2    | FastAPI App Deployment       | 74        |
| 6.1.3    | Ingestion Orchestrator       | 74        |
| 6.2      | Login                        | 74        |
| 6.2.1    | Service Endpoints            | 74        |
| 6.2.2    | Frontend                     | 76        |
| 6.2.3    | System Runtime               | 76        |
| 6.3      | Course Ingestion             | 77        |
| 6.3.1    | Service Endpoints            | 77        |
| 6.3.2    | Frontend                     | 78        |
| 6.3.3    | System Runtime               | 78        |
| 6.3.4    | Measurement                  | 80        |
| 6.4      | Chat                         | 81        |
| 6.4.1    | Service Endpoints            | 81        |
| 6.4.2    | Frontend and Live Validation | 83        |
| 6.4.3    | System Runtime               | 87        |
| <b>7</b> | <b>Conclusion</b>            | <b>89</b> |
| 7.1      | Achieved Results             | 89        |
| 7.2      | System Limitations           | 89        |
| 7.3      | Future Plan                  | 90        |

# List of Figures

|      |                                                                                                                                                                                                                                        |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1  | Transformers convert raw text into embeddings, which are numeric representations of semantic meaning . . . . .                                                                                                                         | 5  |
| 2.2  | Transformer architecture and its attention mechanism . . . . .                                                                                                                                                                         | 6  |
| 2.3  | Comparison of LLM Deployment Strategies: Traditional approaches (a) Direct cloud processing, (b) Local-only processing, and (c) Local-Cloud framework balance performance and security without leaking IP (proposed by [34]) . . . . . | 7  |
| 2.4  | Basic Single Agent diagram . . . . .                                                                                                                                                                                                   | 8  |
| 2.5  | Comparison of Microservice and Monolithic architecture . . . . .                                                                                                                                                                       | 12 |
| 2.6  | Docling Architecture . . . . .                                                                                                                                                                                                         | 14 |
| 2.7  | KGGen with multi-turn LLM reasoning, grouping generation . . . . .                                                                                                                                                                     | 22 |
| 2.8  | The end-to-end EduKG Pipeline framework for educational knowledge construction and retrieval. . . . .                                                                                                                                  | 22 |
| 2.9  | Hybrid architecture of GraphRAG allow minimal cost and guarantee alignment of generated to original context . . . . .                                                                                                                  | 23 |
| 2.10 | Dynamic Knowledge Graph (DKG) approach tracking knowledge evolution over time. . . . .                                                                                                                                                 | 23 |
| 3.1  | Use case diagram of the five use cases and two actors. . . . .                                                                                                                                                                         | 32 |
| 4.1  | High level visualization of how the system should work . . . . .                                                                                                                                                                       | 37 |
| 4.2  | Entity-relationship diagram of the EKG . . . . .                                                                                                                                                                                       | 38 |
| 4.3  | Ingestion Pipeline . . . . .                                                                                                                                                                                                           | 39 |
| 4.4  | Overlapping window segmentation. Adjacent batches share boundary pages. . . . .                                                                                                                                                        | 40 |
| 4.5  | Dual Phase Extraction . . . . .                                                                                                                                                                                                        | 41 |
| 4.6  | The proposed agent harness and its four concerns. . . . .                                                                                                                                                                              | 43 |
| 4.7  | The four tiers of the Context Builder and their budgets. . . . .                                                                                                                                                                       | 44 |
| 4.8  | Top-level workflow routing across the five intents. . . . .                                                                                                                                                                            | 47 |
| 4.9  | Orchestration of one question and its tracked state. . . . .                                                                                                                                                                           | 47 |
| 4.10 | Modes of one Teach turn and their outcomes. . . . .                                                                                                                                                                                    | 48 |
| 4.11 | Roadmap sub-graph. . . . .                                                                                                                                                                                                             | 49 |
| 5.1  | Services of SmartEdu and the data flows inside them. . . . .                                                                                                                                                                           | 53 |
| 5.2  | Modular Architecture Overview of SmartEdu . . . . .                                                                                                                                                                                    | 54 |
| 5.3  | Deployment view of SmartEdu and its services. . . . .                                                                                                                                                                                  | 55 |
| 5.4  | Ownership graph in lifespan. Shared resources are injected into domain modules . . . . .                                                                                                                                               | 56 |
| 5.5  | MinIO bucket directory structure. . . . .                                                                                                                                                                                              | 62 |

|      |                                                                                                                                                                         |    |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 5.6  | Data model across the five storage engines. . . . .                                                                                                                     | 64 |
| 5.7  | Login protocol with Identity Firewall (represented by the red line). . . . .                                                                                            | 65 |
| 5.8  | Sequence of one course ingestion. . . . .                                                                                                                               | 67 |
| 5.9  | Asynchronous Ingestion Pipeline integrating Queues, Parallel Workers, and Storage                                                                                       | 67 |
| 5.10 | Standard operation of a LangGraph Node . . . . .                                                                                                                        | 69 |
| 5.11 | Sequence of one chat turn. . . . .                                                                                                                                      | 71 |
| 5.12 | Trace data hierarchy. Three nested observation levels. . . . .                                                                                                          | 71 |
| 5.13 | Module view of the frontend. . . . .                                                                                                                                    | 72 |
| 5.14 | States of one chat turn in the interface. . . . .                                                                                                                       | 72 |
|      |                                                                                                                                                                         |    |
| 6.1  | Docker-based infrastructure containerization and deployment . . . . .                                                                                                   | 73 |
| 6.2  | Neo4j Monitor Plugins . . . . .                                                                                                                                         | 74 |
| 6.3  | DBgate monitoring both MongoDB and MySQL . . . . .                                                                                                                      | 74 |
| 6.4  | Database Admin Monitoring and Visualizing . . . . .                                                                                                                     | 74 |
| 6.5  | FastAPI successful lifecycle start . . . . .                                                                                                                            | 74 |
| 6.6  | Registering an existing student ID returns status 409. The same account then signs in, starts a new session, and a teaching task is scheduled for that session. . . . . | 77 |
| 6.7  | Ingestion page with a slide deck and a video selected (left), and the completed run card with its timings and outputs (right). . . . .                                  | 78 |
| 6.8  | Completed course flow of the slide and video run in the Prefect interface. . . . .                                                                                      | 79 |
| 6.9  | Extracted Concept Nodes . . . . .                                                                                                                                       | 79 |
| 6.10 | Completed Knowledge Graph demonstrating no concept fragmentation . . . . .                                                                                              | 80 |
| 6.11 | Share of each measured task in the stage time of each source; other time covers dispatch, process startup, waiting and result transfer. . . . .                         | 81 |
| 6.12 | Lecture answer with inline citations and two citation controls (pages 10 and 12). . . . .                                                                               | 84 |
| 6.13 | Desktop conversation with the cited slide opened on page 10 of 32. . . . .                                                                                              | 85 |
| 6.14 | Original source PDF opened directly on page 12 of 32. . . . .                                                                                                           | 85 |
| 6.15 | Phone Chat and PDF tabs preserving the draft and page. . . . .                                                                                                          | 86 |
| 6.16 | Course-filtered search with no indexed sample-course result. . . . .                                                                                                    | 86 |
| 6.17 | Roadmap with course concepts and prerequisite links, before personal progress exists. . . . .                                                                           | 87 |
| 6.18 | Client-side interface displaying the agentic interaction flow . . . . .                                                                                                 | 87 |
| 6.19 | Two snippets of logged agent workflow results . . . . .                                                                                                                 | 88 |

# List of Tables

|      |                                                                                                                        |    |
|------|------------------------------------------------------------------------------------------------------------------------|----|
| 2.1  | Comparison of LLM Orchestration Frameworks . . . . .                                                                   | 11 |
| 2.2  | Polyglot Persistence Architecture: Strategic allocation of storage engines. . . . .                                    | 17 |
| 2.3  | Comparative analysis of representative educational intelligent systems based on key AI and knowledge features. . . . . | 26 |
| 3.1  | Use case 1 Overview: Register Account . . . . .                                                                        | 32 |
| 3.2  | Use case 2 Overview: Login/Logout . . . . .                                                                            | 33 |
| 3.3  | Use case 3 Overview: Manage Learning Session . . . . .                                                                 | 34 |
| 3.4  | Use case 4 Overview: Ingest Course Material . . . . .                                                                  | 35 |
| 3.5  | Use case 5 Overview: Chat with EQA . . . . .                                                                           | 36 |
| 4.1  | Summary of tools available to the Educational QA Agent. . . . .                                                        | 45 |
| 4.2  | Scope of each agent role. . . . .                                                                                      | 46 |
| 4.3  | Answer and evidence quality on MuSiQue. . . . .                                                                        | 51 |
| 4.4  | Mean workflow latency (ms) by number of hops. . . . .                                                                  | 51 |
| 5.1  | System Module Summary . . . . .                                                                                        | 54 |
| 5.2  | LLM Profile configurations and their engineering rationale. . . . .                                                    | 58 |
| 5.3  | Configuration dataclasses. . . . .                                                                                     | 59 |
| 5.4  | Polyglot persistence strategy overview. . . . .                                                                        | 60 |
| 5.5  | Entity node properties in Neo4j. . . . .                                                                               | 61 |
| 5.6  | Milvus collection schema. . . . .                                                                                      | 61 |
| 5.7  | HNSW index configuration. . . . .                                                                                      | 61 |
| 5.8  | MongoDB student document field schema. . . . .                                                                         | 62 |
| 5.9  | MySQL students table schema. . . . .                                                                                   | 63 |
| 5.10 | Write ownership per database. . . . .                                                                                  | 64 |
| 5.11 | Knowledge Module core libraries. . . . .                                                                               | 66 |
| 5.12 | EQA Module core libraries. . . . .                                                                                     | 68 |
| 6.1  | Measured ingestion time and output per source, one sample each. . . . .                                                | 80 |
| 6.2  | Observed frontend behavior in the local live test. . . . .                                                             | 84 |

# Chapter 1

## Introduction

### 1.1 Motivation

The rapid evolution of Artificial Intelligence (AI) has changed the global socio-economic landscape. To adapt to the new era, self-study and learning methods are becoming more and more important. Traditional education philosophy, such as "practice makes perfect," is no longer entirely true; when AI can automate execution, the value of learning moves from procedural fluency and labour expertise to conceptual mastery and strategic analysis. However, current education systems fail to keep pace with this evolution:

- **Stagnancy of Content:** Legacy Learning Management Systems (LMS) treat knowledge as static artifacts (textbooks, slides). These materials fail to adapt to the varying cognitive baselines of individual students.
- **The Guidance Deficit:** Without a dynamic roadmap, students navigate a "sea of information" where external resources are either too granular (leading to information overload) or too condensed (creating knowledge gaps), with little profound ground truth.
- **The Lack of Interaction:** Current systems lack a feedback loop. Textbooks exceed the average human attention span, while slides provide too condensed context insufficient for novices, leaving no room for real-time, human-like inquiry or mentorship.
- **Outdated Knowledge:** Educational materials and methods cannot catch up with the change of technology. The Internet is an invaluable resource, but its information is chaotic, unstructured and confusing to many new learners.

While Large Language Models (LLMs) offer a glimpse into automated resource synthesis and knowledge base construction, significant technical bottlenecks prevent their reliable application in complex domains:

- **Semantic Fragmentation in RAG:** Traditional Retrieval-Augmented Generation (RAG) frameworks rely on discrete data chunking. This fragmentation disrupts semantic integrity, leading to "contextual fragmentation" where the model captures isolated facts but fails to comprehend the overall hierarchy of the domain.
- **Contextual Saturation vs. Precision:** Expanding the context window of LLMs often introduces diminishing returns. The injection of unrefined data frequently leads to "contextual overhead" and "noise," where the model struggles to distinguish fundamental principles from peripheral details.

- **Relational Limitations of Vector Space:** Relying exclusively on vector embeddings is insufficient for mapping the non-linear, multi-faceted dependencies inherent in academic taxonomies. Similarity-based retrieval cannot substitute for the deterministic logic required to understand prerequisite relationships.
- **The Reasoning Gap in Academic Tasks:** Instruction is a sophisticated, non-linear reasoning task. Current AI systems are largely reactive—proficient in text generation but deficient in long-term goal tracking and proactive guidance. This results in a susceptibility to hallucinations and an inability to execute the "evaluate-and-guide" workflow essential for effective pedagogy.

## 1.2 Goals

To address the current limitations, this project aims to design an intelligent system that goes beyond serving content or a simple QA chatbot: a proactive, mastery-oriented learning platform tracking and adapting to student progress by developing SmartEdu—an intelligent, graph-grounded educational system. This thesis defines the capabilities of such a system as follows:

- **Adaptive Mentorship:** Shifting the educational paradigm from static, linear content delivery to interactive, user friendly workflow that adapts itself to each student's profile and actions.
- **Data Sovereignty and Localized Execution:** A deployment framework that can utilize open-weights Small Language Models (SLMs) and local inference engines, so that student learning data can stay on local infrastructure and recurring API token costs are reduced.
- **Consistent Educational Content:** Automated content adaptation and generation suffer from hallucinations and loss of key information. Mitigating the hallucination risks inherent in generative AI by anchoring the multi-agent reasoning framework within a structured knowledge network to enforce academic truthfulness.

Targeted at university students, this educational platform supports self-learning, one of the most important skills at university. To demonstrate practical utility and fulfill the discussed goals, the application provides the following features:

- **Automated Educational Knowledge Graph (EKG) Engine:** The system automatically ingests unstructured academic materials (raw PDFs, textbooks, and lecture videos) and constructs a structured knowledge graph. It automatically extracts conceptual hierarchies, maps prerequisite dependencies, and groups related topics into organized clusters.
- **Dynamic Student Mastery Mapping:** The application implements a continuous storing-and-tracking engine that tracks the student's real-time comprehension levels, storing historical interaction and cognitive progress securely.
- **Automated Gap Detection and Pathfinding:** Utilizing graph-traversal and heuristic algorithms, the system chooses a study path based on the student's current mastery and state against the EKG backbone. When a target topic is selected, the application automatically identifies missing prerequisite knowledge and dynamically generates a personalized, non-linear learning path.
- **Socratic Dialogue and Document Verification:** The user interface provides an interactive teaching assistant that engages students through multi-step reasoning workflows.

## 1.3 Scope

To fulfill these goals, the research focuses on the following scope:

- **Multimodal Information Extraction:** Knowledge of courses from multiple sources (pdf, videos, textbooks) must be well extracted to consolidate them into a centralized data repository.
- **Domain Specificity:** The knowledge modeling and extraction processes are limited to well-defined academic disciplines with clear hierarchical structures. Unstructured or subjective domains that lack clear prerequisite dependencies are outside the scope of this research.
- **Pedagogical Boundaries:** The conversational assistant is strictly restricted to goal-oriented educational support, specifically for assessing student understanding and explaining course content. Open-domain dialogue, creative writing, and casual chitchat are intentionally disabled.
- **Infrastructure:** The system is designed around open-weights models, so that it can run on local infrastructure when data privacy requires it.
- **Target User Environment:** The system is modeled and evaluated for a single-institution environment, focusing on the interactions between individual learners and a localized knowledge base, rather than a multi-tenant, global-scale educational network.

## 1.4 Thesis Structure

There are 7 chapters in this project's report:

- Chapter 1 introduces the problems, proposes the system's goals, scopes and thesis structure
- Chapter 2 provides the fundamental knowledge needed to understand the project, a literature review of relevant prior research, and an overview of existing educational applications.
- Chapter 3 specifies the requirements.
- Chapter 4 proposes the methodology: what we inferred from previous work, how it is adapted to our use case, and how the retrieval stage is evaluated.
- Chapter 5 details the system: its architecture and components, the settings of each component, and how the implementation puts the proposed methodology into practice.
- Chapter 6 presents the deployed system and its behaviour at runtime, from individual components to the final output.
- Chapter 7 concludes with what the system achieved, its limitations and the future plan.

# Chapter 2

## Fundamental Knowledge & Related Work

### 2.1 Fundamental Knowledge

This section discusses the fundamental techniques and applications foundational to the construction of the proposed system, categorized into 4 pillars:

- **Natural Language Processing (NLP):** The process of transforming raw, unstructured text into structured data formats that machines can process. This forms the basis for document processing and subsequent analysis.
- **Large Language Models (LLMs):** The reasoning components responsible for interpreting user queries, orchestrating tools, and generating responses. This enables the system to perform complex tasks and adapt to user needs.
- **Agentic Orchestration:** The management layer that coordinates specialized AI agents to execute complex, non-linear workflows.
- **System Design:** The architectural structure that ensures scalability and robustness, using design patterns tailored for educational environments.

#### 2.1.1 Natural Language Processing

Natural Language Processing (NLP) converts unstructured document data into a structured representation. The goal is to translate raw text into a semantically precise format, enabling machine-inferable logic and further inferences.

The standard NLP pipeline follows this sequence:

Information Extraction (IE) → Knowledge Modeling → Information Retrieval

**Information Extraction (IE)** Information Extraction isolates relevant portions of text to produce a structured representation [12]. It filters out linguistic noise to capture the atomic units of knowledge. Traditional algorithms extract keywords using two primary methods:

- **Keyword and Feature Extraction:** Uses dependency parsing and statistical methods (e.g., TF-IDF) to identify high-signal terms based on syntactic importance.
- **Named Entity Recognition (NER):** Classifies entities (e.g., academic theories, formulas, historical figures) within a text database [12]. These entities form the "nodes" of the system's knowledge graph.

To capture relationships that rigid rule-based systems miss, this process uses probabilistic modeling. It begins with Tokenization, breaking raw text into atomic units (tokens). Statistical algorithms like Conditional Random Fields (CRF) or Hidden Markov Models (HMM) then calculate probabilities for different interpretations, resolving linguistic ambiguities to predict the most likely semantic structure.

Traditional probabilistic models often struggle with sparse data and lack deep semantic understanding. Deep Learning addresses this by generating Embeddings: these architectures map tokens into a continuous, high-dimensional vector space. In this space, geometrically close vectors represent semantically related concepts, so a model can capture context and nuance beyond surface-level statistics.

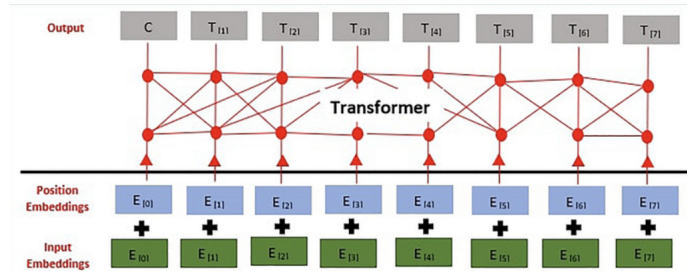


Figure 2.1: Text-to-embedding conversion

The Transformer architecture has become the standard method in NLP, enabling modern Large Language Models (LLMs). Its core innovation is the Self-Attention mechanism. Transformers process the entire sequence at once, not token by token. The attention mechanism calculates a weighted importance score for every token relative to all other tokens, allowing the model to capture global context and long-range dependencies from unstructured texts.

**Knowledge Modeling** Often overlooked in standard Retrieval-Augmented Generation (RAG) systems, Knowledge Modeling is the construction stage where extracted entities are organized into a coherent structure that shapes how the system reasons.

- **Semantic Parsing and Logical Form:** Maps extracted relations into a semantically precise form, such as First-Order Predicate Calculus (FOPC) or Knowledge Graphs (KG). This creates a deterministic "Backbone Graph" of the curriculum.
- **Embedding Space Topology:** Models knowledge within a continuous vector space where geometric proximity represents semantic similarity. This allows the system to handle concepts that are conceptually related but use different vocabulary.

**Information Retrieval (IR)** Information Retrieval is the active execution stage based on the results of IE and Knowledge Modeling. The goal is to identify data points relevant to a specific information need [12].

- **Lexical and Keyword Matching:** Uses exact term overlaps to find specific references or symbolic links within the knowledge base.
- **Vector Retrieval:** The standard modern approach, which uses the embedding space to retrieve context based on semantic similarity, typically calculated via Cosine Similarity:

$$\text{similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} \quad (2.1)$$

- The GraphRAG Paradigm: A retrieval method that navigates the entity-relationship graph established during the modeling phase. Unlike discrete chunk retrieval, GraphRAG performs "multi-hop" reasoning to connect disparate concepts, enabling the system to resolve complex queries that require a global understanding of the curriculum.

### 2.1.2 Large Language Models (LLMs)

Large Language Models (LLMs) are deep learning-based probabilistic models trained on massive datasets to understand and generate human-like text. Mathematically, an LLM performs the task of next-token prediction by modeling the conditional probability distribution of a token  $w_t$  given a sequence of preceding tokens  $(w_1, w_2, \dots, w_{t-1})$ :

$$P(w_t | w_1, w_2, \dots, w_{t-1}) \quad (2.2)$$

The structural foundation of modern LLMs is the Transformer architecture, which relies on the Self-Attention mechanism. This mechanism allows the model to compute the relative importance of different tokens in a sequence regardless of their distance. The attention score is calculated using Query ( $Q$ ), Key ( $K$ ), and Value ( $V$ ) matrices derived from input embeddings:

$$Attention(Q, K, V) = \text{softmax} \left( \frac{QK^T}{\sqrt{d_k}} \right) V \quad (2.3)$$

By utilizing multi-head attention and positional encoding, Transformers can capture complex linguistic patterns and long-range dependencies more effectively than previous sequential architectures like Recurrent Neural Networks (RNNs).

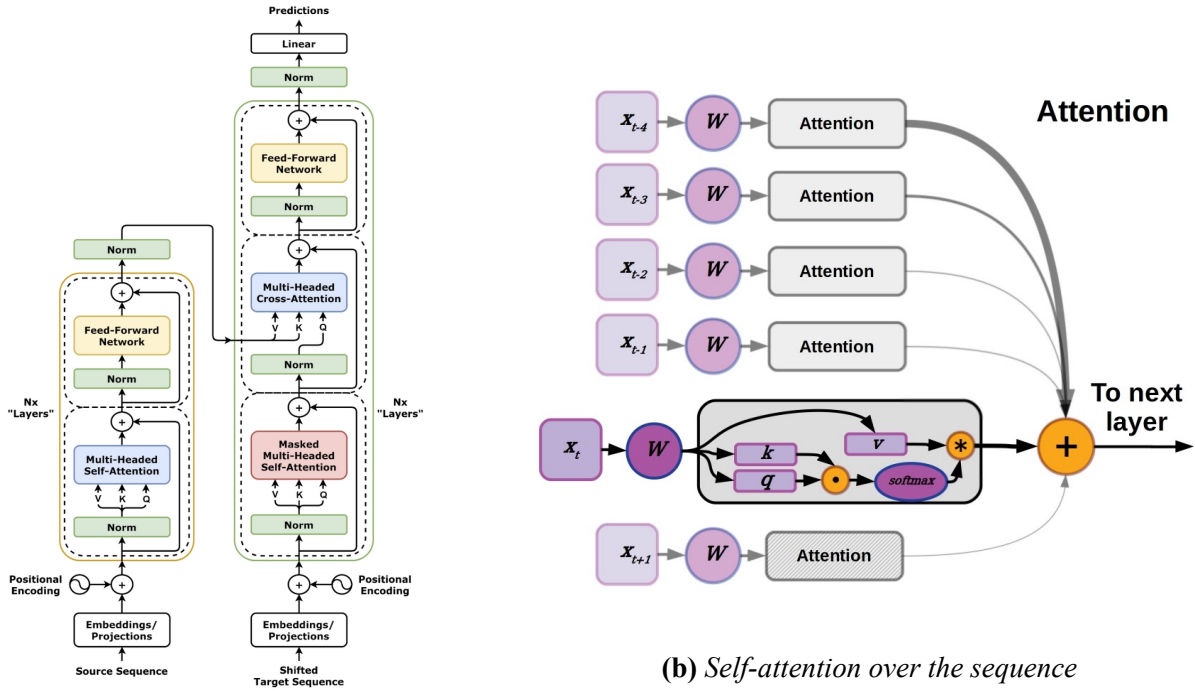
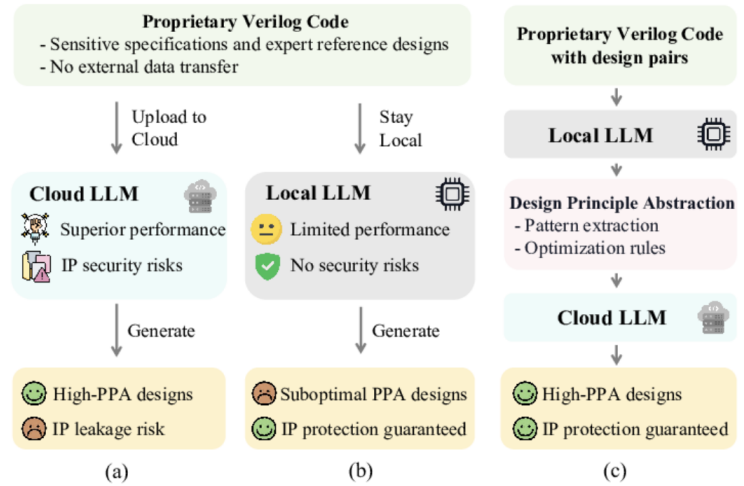


Figure 2.2: Transformer architecture and its attention mechanism

### 2.1.2.1 LLM Deployment: Proprietary vs Open-Weights

The landscape of Large Language Models in the industrial sector is divided by deployment strategy and architectural transparency. Closed-source, cloud-based models like GPT-4 and Claude form the proprietary tier. Provided as a Service (SaaS), these models are accessed via APIs hosted on centralized infrastructure. While they deliver state-of-the-art performance, their internal mechanisms (weights, datasets, and configurations) remain opaque. Additionally, per-token API costs can limit large-scale or iterative research applications.



**Figure 2.3:** LLM deployment strategies: (a) cloud, (b) local, (c) local-cloud hybrid [34]

In contrast, open-weights models enable decentralized execution and architectural transparency. Projects like Meta’s Llama series and Mistral release their pre-trained parameters, allowing deployment on private hardware. This ensures full control over data sovereignty and privacy, as information remains local. These models serve as baselines for academic research and industrial applications without relying on external APIs or subscription fees.

Building on these general-purpose baselines, researchers have fine-tuned models for specialized tasks. Coding specialists like Qwen-coder or CodeLlama are optimized for source code synthesis and logical reasoning, often rivaling larger general-purpose models in programming contexts. Beyond text, models have been extended into multimodal domains. By integrating vision encoders, developers enable visual comprehension and cross-modal reasoning.

### 2.1.2.2 LLM Hyper-parameters and Inference Control

This subsection explains the utility of LLM inference parameters in controlling the generation process. During inference, the next-token probability distribution predicted by the model can be dynamically altered to balance determinism and creativity.

**Temperature** Temperature ( $\tau$ ) controls the sharpness or randomness of the predicted probability distribution. Given the logits  $z_i$  output by the final layer, the adjusted probability  $P(w_i)$  for each candidate token is computed using the modified softmax function:

$$P(w_i) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \quad (2.4)$$

A lower temperature ( $\tau < 1$ ) sharpens the distribution, making the model more confident in its top choices and producing highly deterministic text, which is ideal for rigid tasks like JSON extraction. A higher temperature ( $\tau > 1$ ) flattens the distribution, encouraging diversity and creativity.

**Top-k Sampling** Top- $k$  sampling restricts the model from considering the entire vocabulary, instead keeping only the  $k$  highest-probability candidates. This approach is conceptually similar to beam search, where only the most promising paths are kept active:

$$V_k = \arg \max_{V' \subset V, |V'|=k} \sum_{w \in V'} P(w) \quad (2.5)$$

The probability mass is then redistributed among these  $k$  tokens. This truncation prevents the model from generating highly improbable and potentially nonsensical words (the "long tail" of the distribution).

**Top-p (Nucleus) Sampling** Top- $p$  (nucleus) sampling provides a dynamic alternative to Top- $k$ . Instead of keeping a fixed number of tokens, it keeps the smallest prefix subset of the vocabulary whose cumulative probability mass meets or exceeds a threshold  $p$ :

$$\min \left\{ |V'| : V' \subset V, \sum_{w \in V'} P(w) \geq p \right\} \quad (2.6)$$

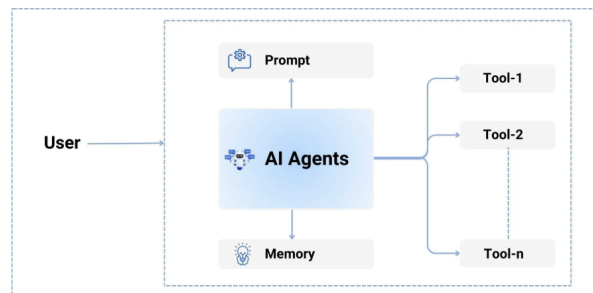
This allows the candidate pool to expand or contract organically based on the model's confidence. When the model is very confident, the nucleus contains only a few tokens; when uncertain, the nucleus expands to include more options.

**Repetition Penalty** Repetition penalty modifies the logits of tokens that have already appeared in the generated sequence, mathematically reducing their likelihood of being selected again. This prevents the model from entering infinite generation loops or excessively repeating specific phrases, ensuring a more natural text flow.

## 2.1.3 Agentic Systems

### 2.1.3.1 AI Agents

An Artificial Intelligence (AI) Agent is a computational system capable of autonomous task execution through the dynamic design of workflows using available tools. Beyond language processing, AI agents can make decisions, solve problems, and interact with external environments.



**Figure 2.4:** Basic Single Agent diagram

Unlike standard LLMs, which rely solely on static training data and fixed knowledge limits, agentic technology uses tool calling on the backend to obtain up-to-date information. Agents optimize workflows and create subtasks autonomously, translating high-level natural language queries into executable workflows that complete real-life tasks.

What defines the Action Capabilities of Agents is the harness. This involves constructing and integrating systems that boost capabilities, enabling the agent to navigate the complex, multi-step processes required for deep reasoning.

**LLM: The Logical Core** Large Language Models (LLMs) serve as the brain of the agent, representing its thinking capacity. They provide the reasoning capabilities necessary to interpret vague user queries and orchestrate the appropriate tools. The LLM is responsible for parsing intent, identifying missing information, and deciding on the next actions. System decision-making depends heavily on the model used:

- Reasoning Models: Models like OpenAI o1 or DeepSeek R1 are optimized for Chain of Thought, decomposing difficult queries into executable multi-step plans.
- Function Calling Models: Models such as GPT-4o or Claude 3.5 Sonnet are fine-tuned to output structured data (e.g., JSON) for tool configuration and output formats.
- Multimodal Models: Models such as Gemini 1.5 Pro natively understand both text and visual inputs.
- Lightweight Efficiency Models: Smaller models like Llama 3 8B or GPT-4o-mini are faster and cheaper to run, making them ideal for narrow, repetitive tasks.

**Context Engineering** Context engineering represents the memorization capacity. It involves crafting the environment in which the LLM operates to ensure accurate and consistent outputs. This field includes several topics that govern how the model interacts with data and users:

- Prompting Strategies: Prompting is the primary method to instruct the agent, helping it understand the goals and constraints it is dealing with. Advanced methods like Chain of Thought (CoT) and Language of Thought (LoT) prompt the agent to expand its reasoning process step-by-step, ensuring a meaningful thought process before a final answer is generated.
- State Management: To maintain a consistent workflow, systems must store chat history and the current state. This allows the agent to understand the conversation context and the results from tools or other agents. State management ensures the agent knows exactly where it is in a multi-step workflow process, preventing redundant actions or confusion.
- Retrieval-Augmented Generation (RAG): Agents cannot remember all the information contained in massive data repositories, so RAG is used to retrieve external information that is not stored within the LLM's internal memory. RAG ensures the agent gets the needed information by querying relevant data segments or metadata to ground its answers in factual evidence.

**Tool Integration** Representing the action capacity, tool integration is the process of connecting external resources (systems, software, hardware) or APIs to the agent, allowing it to perform specific functions that are beyond the capabilities of the LLM alone. It lets the agent act on the external world, not only on static text. It also carries risk: a wrong judgement by the agent becomes a real action.

### 2.1.3.2 Agent Frameworks

Retraining a Large Language Model for every rule or task is expensive. Agent Frameworks are software architectures that control and improve the behavior of AI without changing the model itself. They connect the generative model, as the neural core, with ordinary software logic. Modern frameworks cover three main aspects of agentic design and orchestration:

**Multi-agent Orchestration** In complex operations, giving a single agent too much information and too many different tasks often causes errors. A single agent has a limited attention span and context window. Frameworks solve this by using Multi-agent Systems. By breaking a large job into smaller parts handled by different agents, developers can assign specific agents to each task (e.g., one for visual analysis, one for audio processing, and one for synthesis), ensuring each agent stays focused and accurate.

**Agent Memorization** Intelligent systems need to remember information over time, but standard LLMs treat every question as a new conversation (stateless). Frameworks solve this with hierarchical memory objects, such as sessions, that agents query for previous context at run-time. Memory management usually follows two paradigms:

- **Structured Memory:** Information is organized so that the agent reads only the relevant context, not every past detail.
- **Context Window Optimization:** Memory lives in an external database, and tool results or conversation topics are loaded lazily when needed. This prunes and injects context dynamically, keeping the agent within its limits and saving computation.

**Tool Abstraction** Frameworks turn standard code functions into Tools that agents can use on their own. The key difference between a normal function and a Tool lies in its data structure:

- **Description:** Instructions on what the tool does and when to use it. The agent reads this description when needed without overwhelming itself with internal logic.
- **Input/Output Schema:** A defined structure for input and output parameters, often in JSON format, specifying the type and format of data the tool expects and returns. This allows the agent to call the tool correctly.
- **Logging memory:** Function results and errors are logged into memory for agent inference, context injection, and debugging.

### 2.1.3.3 Langchain and LangGraph

The specific use case of an educational system necessitates a flexible, nonlinear, and highly orchestrated approach. While numerous frameworks exist to harness the capabilities of Large Language Models (LLMs), the intricate requirements of educational guidance require a high-level, state-aware framework.

Unlike sequential or data-centric frameworks like LlamaIndex and SDK, LangGraph is built upon a *State Graph Architecture* where the system is modeled as a directed graph. In this paradigm, each node represents a specialized agent or a discrete computational step, while the edges dictate the transition logic based on the system's current state.

This granular control over the workflow enables non-linear branching, allowing the system to perform multipath reasoning for complex, non-repetitive tasks. In an educational context, because the workflow is globally state-aware, it can gracefully navigate diverse use cases without losing track of the student's long-term learning objectives. This is statistically difficult to achieve with stateless or purely sequential agentic frameworks.

| Framework         | Core                                                                                                                  | Characteristic                                                  | Strength in Education                                                                                 | Limitation in Education                                                                                                                                                                       |
|-------------------|-----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>LlamaIndex</b> | Data-Centric: Optimizing the interface between LLMs and private data.                                                 | Retrieval-heavy; focus on indexing and query engines.           | Very powerful when it comes to multistep input output manipulation of tools, LLMs and states          | Often prioritizes data retrieval over complex, multi-step pedagogical reasoning.                                                                                                              |
| <b>LangGraph</b>  | Explicit state management via cyclic graphs. Supporting edges, nodes creation making full control of logic, branching | <b>Stateful, non-linear branching</b> with fine-grained control | Suitable for complex nonlinear task, with stateaware and context, logic localization and independence | Branching acts as a selling point and a potential logical error, when applying complex flow to simple tasks. Most of all, due to its high level abstraction, input output of LLM get unstable |
| <b>Google SDK</b> | Tool-Centric: Direct, low-latency access to model capabilities.                                                       | Functional/Linear; focus on tool-calling and result passing.    | Deliver stable tool results, state and worker result if the pipeline is well designed                 | Becomes unstable and difficult to maintain when including branching workflows.                                                                                                                |
| <b>CrewAI</b>     | Role-Centric: Simulating human-like collaborative teams.                                                              | Autonomous, task-based collaboration between predefined roles.  | High level, well LLM and human interaction facilitation, capable of complex workflow                  | Can lack the deterministic, full control orchestration required for strict academic knowledge modeling.                                                                                       |

**Table 2.1:** *Comparison of LLM Orchestration Frameworks*

The survey of various frameworks shown in Table 2.1 highlights LangGraph as the top framework for an educational system. While LlamaIndex offers stable operation with full data control, its data-centered nature often obscures the complex logic and asynchronous operations required for student-teacher interaction. The Google Generative AI SDK, though efficient for direct tool integration, lacks the complex state-management infrastructure necessary to handle unstable workflows compared to state-graph frameworks. The ability of LangGraph and CrewAI to maintain a persistent state and manage non-linear transitions provides the deterministic backbone required to facilitate an adaptive and reliable learning experience.

## 2.1.4 System Design

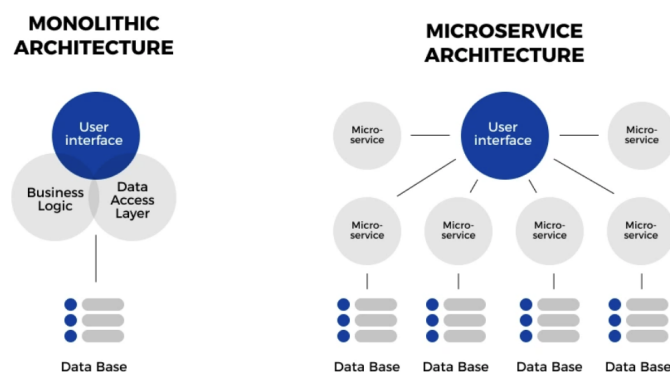
This section explains the software engineering principles, architectural patterns, and technological foundations used to construct scalable AI-integrated systems.

Object-Oriented Programming (OOP) is a paradigm based on "objects" that contain data (attributes) and code (methods). In System Design, OOP is the industry standard because core architecture components are inherently stateful. Unlike simple functional scripts that execute and exit, system components must maintain persistent internal states, including conversation history and active tool configurations. OOP provides the natural structure to manage this state lifecycle through

four pillars:

- **Abstraction:** Allows the system to model complex entities by hiding internal implementation details and exposing only relevant operations. This lets outside workflow logic interact with high-level methods rather than raw data from lower components.
- **Encapsulation:** Groups related variables and functions into a single unit, protecting the internal state from unintended interference.
- **Inheritance:** Promotes code reusability by allowing new classes to derive properties from existing ones. Specialized classes inherit these foundational capabilities while implementing unique behaviors, significantly reducing code duplication.
- **Polymorphism:** Allows objects of different classes to be treated as objects of a common superclass. An orchestrator can iterate over different components, such as retrieval engines or analysis modules, and run them through one interface without knowing their concrete type.

System architecture defines the overall structure of a software system, organizing its major components and how they communicate. Major applications often start with a Modular Monolithic Architecture. Unlike a traditional monolith where code is heavily tangled, a modular monolith compiles the application as a single unified program but keeps clear logical boundaries between different functions. This approach offers practical early-stage benefits: it simplifies deployment, ensures fast internal communication, and makes the codebase easier to test and debug.



**Figure 2.5:** *Comparison of Microservice and Monolithic architecture*

However, as system demands grow, combining heavy processing tasks with basic web routing in a single program can cause performance bottlenecks. Microservices take the next step by turning logical modules into physical, independent services. Each service runs in its own process, often with its own separate database, and communicates with other services using standard protocols like REST APIs or message brokers. This architecture inherits the organizational advantages of the modular monolith while providing several additional merits:

- **Independent Scalability:** Microservices allow different parts of the system to scale separately. For example, a heavy processing service can be upgraded with more computing power or duplicated without needing to scale the lightweight web server, ensuring efficient hardware use during high traffic.
- **Fault Isolation:** Software failures are contained. If a heavy processing service crashes due to memory limits, it does not bring down the entire system. Other services, such as user login and basic web browsing, remain fully operational while the crashed service restarts.

- **Technology Flexibility:** Microservices allow developers to choose the best programming language or tool for each specific task. For instance, data science components can be built using Python for its machine learning libraries, while high-speed API routing can be written in Go or Node.js.

Alongside these benefits, microservices possess some disadvantages. Internal function calls in a monolith are extremely fast. In microservices, these are replaced by network requests over the internet or local network, adding delay and processing overhead. Moreover, managing, monitoring, and deploying multiple independent services with separate databases is significantly harder than deploying a single program. It also requires complex strategies to keep data consistent across independent services.

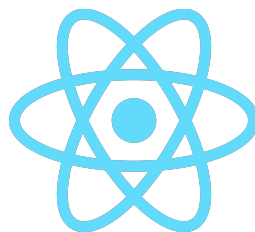
### 2.1.5 Technology Stack and Frameworks

The selection of frameworks is driven by the need for high-performance systems with direct AI integration. The following frameworks are effective building blocks for implementing the application layer.



**Backend Development** FastAPI is a modern web framework for building APIs with Python. Older frameworks like Django were built on the synchronous WSGI standard, while FastAPI runs on the Asynchronous Server Gateway Interface (ASGI) and handles Python's await syntax natively. An I/O-bound task, such as waiting for an LLM response or a database query, releases the CPU to serve other incoming requests. Its key features are:

- **High Concurrency:** By utilizing the ASGI standard, FastAPI can handle thousands of simultaneous connections (such as multiple users uploading videos or chatting with agents) without the blocking issues inherent in traditional synchronous frameworks.
- **Data Validation:** It integrates tightly with Pydantic, a data validation library. This ensures that the complex JSON outputs generated by AI agents are rigorously validated against strict schemas before being sent to the frontend, preventing runtime type errors.
- **Performance:** Built on top of Starlette, FastAPI offers execution speeds comparable to compiled languages like Go, which is critical for minimizing the latency overhead in the educational content processing pipeline.



**Frontend Development** React is a JavaScript library for building user interfaces from reusable components. Each component holds its own state and re-renders when that state changes, which suits screens that update often, such as a chat that receives new messages while the user reads.

Next.js is a framework built on React that adds routing, server rendering, and a production build pipeline. Its App Router maps folders to URL routes, and its React Server Components render on the server and send finished HTML, so only the interactive parts of a page ship JavaScript to the browser. Next.js can also run server-side route handlers and a request proxy, which lets the browser talk to a single origin while the frontend server forwards calls to backend services and keeps tokens out of client scripts.

- **Server and Client Components:** Static layout renders on the server; forms, viewers, and chat widgets run in the browser.
- **File-based Routing:** Each folder under the app directory is a route with its own layout, loading, and error states.
- **Built-in Optimization:** Code splitting, image optimization, and caching are handled by the framework.

**UV: High-Performance Package Management** Traditional Python dependency management with pip and virtualenv introduces significant overhead in complex AI projects, where dependency resolution across dozens of GPU-accelerated libraries can take several minutes. uv is a Rust-based Python package manager developed by Astral, which consolidates virtual environment creation, dependency resolution, and lockfile generation into a single binary.



UV achieves dependency resolution 10 to 100 times faster than pip by implementing a custom resolver in Rust. Its deterministic lockfile mechanism ensures that all team members and deployment environments operate with identical dependency versions, eliminating the "works on my machine" class of bugs.

**Docling: Document Conversion** Plain PDF text extraction returns one stream of characters in which headings, captions, tables, and body text look the same. Docling is a document conversion toolkit that rebuilds the layout of a file into a typed document model: it recognizes headings, paragraphs, tables, figures, and reading order, and keeps the heading hierarchy the author wrote.

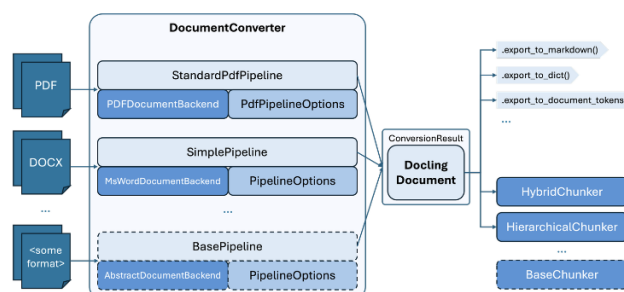


Figure 2.6: Docling Architecture

From this model, a document can be split along its own sections, not fixed-size character windows. Each chunk keeps its heading path and page number, so the table of contents becomes usable structure without a language model having to reconstruct it.

**Automatic Speech Recognition** Lecture recordings hold content that often exists nowhere in writing, and a text pipeline cannot use them until they are transcribed. Whisper is an open speech recognition model from OpenAI trained on large multilingual audio data. faster-whisper reimplements it on the CTranslate2 inference engine, running faster and with less memory than the reference code at the same accuracy. It also supports reduced numerical precision, so the same model can run on a GPU or fall back to a CPU.

**Prefect: Workflow Orchestration** A long data pipeline that runs as one background job is hard to observe and fragile: when a late step fails, the whole job restarts and repeats every expensive step before it. Prefect is an open-source workflow orchestrator for Python in which ordinary functions are declared as flows and tasks. A server records every run with its state, timing, and logs, and presents them in a web interface.

- **Subflows and Deployments:** A flow can call other flows as subflows, and a flow registered as a deployment can be executed by a worker process polling a work pool, so heavy steps run in separate processes or on separate machines.
- **Retries and Result Caching:** Tasks declare their own retry policy, and a task result can be persisted under a cache key, so a rerun reuses completed results instead of computing them again.
- **Observability:** Every flow and task run has a state (e.g., Running, Completed, Failed) that can be tracked from the interface or its API while the pipeline is still running.

**Ollama: Local Model Inference and Optimization Infrastructure** Ollama is an advanced abstraction layer designed for managing and executing LLMs on local infrastructure. It is built upon the llama.cpp core, providing an efficient environment for model orchestration and hardware acceleration.



A critical feature of local inference is Quantization, the process of reducing the precision of model weights from high-bit floating-point formats (e.g., FP16 or BF16) to lower-bit integers (e.g., 4-bit or 8-bit). This technique significantly reduces the VRAM footprint and memory bandwidth requirements:

$$w_q = \text{round} \left( \frac{w}{\Delta} \right) \quad (2.7)$$

where  $w$  is the original weight and  $\Delta$  is the scaling factor. Quantization enables large-scale models to run on consumer-grade hardware with minimal degradation in perplexity and reasoning capabilities.

Moreover, it utilizes the GGUF (GPT-Generated Unified Format), a binary format designed for rapid model loading and efficient metadata storage. Key technical characteristics include:

- **mmap Compatibility:** Allows models to be mapped directly into memory, enabling nearly instantaneous startup times.
- **Unified Execution:** Supports flexible memory offloading, where layers of the model can be distributed between the GPU (VRAM) and CPU (System RAM) based on available resources.
- **Modelfile Abstraction:** Provides a structured way to define system prompts, temperature settings, and context window sizes within a single configuration file, ensuring deterministic behavior across different local environments.

**The Langchain Framework** Mentioned in Subsection 2.1.3.3, Langchain [13] is one of the most popular agentic frameworks in the industry, with a large community and rich documentation. The accumulated contributions to the framework and agentic research using Langchain have provided us with a wealth of tools and integrations:

- **Pre-built Agents:** Customizable agents for question answering, summarization, and multi-step reasoning; in 2026 it published DeepAgent for long-running workflows.
- **Langfuse:** Along with LangSmith, an observability platform that traces every model and tool call of an agent run, so developers can follow its decisions and debug failed workflows.
- **Context Management:** Utilities for memory over multiple external databases, conversation history, and dynamic context injection across long interactions.

## 2.1.6 Database System Design & Management

A database is a structured system where applications store, read, write, update, and delete necessary information. In the domain of modern software engineering, data handling has become increasingly complex. To address this, many architectures adopt a Polyglot Persistence strategy, which uses distributed storage technologies optimized for different data modalities.

**Table 2.2:** *Polyglot persistence architecture.*

| Storage Engine           | Data Modality            | System Role                                        | Theoretical Justification                                                                                                                                                |
|--------------------------|--------------------------|----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Relational (MySQL)       | Structured Data          | User Authentication, Billing, Access Logs          | Enforces ACID properties for transactional integrity and supports complex relational joins for administrative analytics.                                                 |
| NoSQL (MongoDB)          | Semi-Structured Data     | Semantic Metadata, Analysis Tags, JSON Outputs     | Provides schema polymorphism to handle variable outputs from different AI agents (e.g., OCR vs. Object Detection) without sparse tables.                                 |
| Object Storage (MinIO)   | Unstructured Binary      | Lecture Materials, Educational Assets, Audio Clips | Decouples storage from compute to prevent database bloat; utilizes Erasure Coding for high-throughput parallel streaming during ingestion.                               |
| Vector Database (Milvus) | High-Dimensional Vectors | Embeddings Index, Semantic Search                  | Optimized for Approximate Nearest Neighbor (ANN) algorithms (e.g., HNSW), enabling efficient similarity search in latent spaces where traditional B-Tree indexing fails. |
| Graph Database (Neo4j)   | Multi-hop Graph Data     | Knowledge Graph, Prerequisite Relationships        | Optimized for traversing complex relationship networks; enables recursive queries and multi-hop reasoning to connect disparate educational concepts.                     |

**Relational Database (RDBMS)** Relational Databases are mathematically founded on Set Theory and First-Order Predicate Logic. They organize data into tuples (rows) and attributes (columns) within strictly defined relations (tables). The defining characteristic of this model is the enforcement of ACID properties (Atomicity, Consistency, Isolation, Durability), which guarantees that database transactions are processed reliably.

RDBMS is typically utilized for data that requires high integrity and deterministic relationships, such as User Management and Billing. It acts as the source of truth for entity relationships, enabling complex SQL JOIN operations (e.g., selecting all learning materials uploaded by a specific role within a timeframe). The adoption of RDBMS for core administrative data offers specific advantages to system stability.

MySQL is one of the most widely used open-source Relational Database Management Systems. It typically handles core entities such as user accounts, roles, and administrative logs, ensuring that all structured interactions follow strict relational integrity.

- **Transactional Integrity:** In a multi-user environment where credits or permissions are updated frequently, RDBMS ensures that race conditions do not occur. For instance, if a user purchases a subscription while simultaneously uploading an educational asset, the ACID properties ensure the account balance is updated accurately before the upload permission is granted.
- **Data Normalization:** By structuring data into normalized tables (e.g., Users, Roles, Logs), redundancy is minimized. This ensures that a change in a user's details is reflected system-wide instantly without needing to update thousands of duplicate records.
- **Complex Querying:** The SQL engine allows administrators to perform audit trails and analytics efficiently, such as tracking system usage patterns across different user groups, which is computationally expensive in non-relational systems.

**Non-Relational Database (NoSQL)** To address the limitations of rigid RDBMS schemas, Non-Relational databases emerged, often adhering to the CAP Theorem (Consistency, Availability, Partition Tolerance). Unlike RDBMS, which prioritizes strong Consistency, NoSQL databases like MongoDB often prioritize Partition Tolerance and Availability, offering a flexible schema-less design.

Document stores are typically the primary engine for storing semantic metadata derived from educational content analysis. Metadata derived from curriculum analysis is highly variable; one resource might contain OCR text, while another contains only Audio Transcripts or Face Recognition data. Document stores handle these as dynamic BSON (Binary JSON) documents, allowing fields to be added on the fly without the sparse matrices typical of rigid SQL tables.



MongoDB is a high-performance document store suited to the semi-structured metadata generated by AI agents. It can hold diverse analysis results—such as transcripts, OCR results, and agentic reasoning logs—within a single, flexible schema.

- **Schema Agnosticism:** AI agents are constantly evolving. If an "Object Detection" agent is upgraded to output new attributes (e.g., "Confidence Score" or "Bounding Box Coordinates"), document stores allow the new data to be saved immediately without migrating the entire database schema or breaking existing application logic.
- **Read Performance for Aggregates:** In a Knowledge-QA scenario, the system needs to retrieve the Video Title, Description, Transcripts, and Vector IDs simultaneously. In an RDBMS, this might require joining multiple tables. In a NoSQL system, this data can be stored as a single nested document, allowing the application to retrieve the entire context of a lesson in a single, high-speed read operation.

- **Horizontal Scalability:** As archives grow to thousands of hours of lecture content, metadata storage requirements increase. Native sharding capability allows data to be distributed across multiple servers economically, ensuring lookup times remain low even as the dataset grows.

**High-Performance Object Storage** While RDBMS and NoSQL excel at storing alphanumeric data, they are fundamentally inefficient for storing massive binary large objects (BLOBs) like lecture recordings, leading to database bloat and performance degradation. Object Storage systems like MinIO treat data not as a file hierarchy or a table, but as discrete objects stored in a flat address space.

High-performance architectures utilize Object Storage to handle the high-volume nature of raw educational media. It utilizes Erasure Coding—a mathematical algorithm that splits data into fragments and expands them with redundant data pieces—allowing for data recovery even if multiple drives fail. This approach decouples storage from compute; the heavy educational artifacts are offloaded to Object Storage, while the databases store only the lightweight reference pointers (URLs) to these objects.



MinIO is a high-performance, S3-compatible object storage server. It commonly hosts raw multimedia assets, including lecture videos, extracted images, and audio segments used for AI training and inference.

- **Decoupling Storage and Compute:** By separating the raw files from the metadata databases, "database bloat" is prevented. This ensures that the RDBMS remains lightweight and responsive for user queries, while the Object Store handles the heavy I/O load of content streaming.
- **Parallel Streaming for AI Ingestion:** When an agent analyzes a lecture recording, it requires high-speed access to the file to extract frames. Object Storage supports parallelized byte-range requests, allowing the AI to read multiple parts of a media file simultaneously. This significantly reduces the time required for the ingestion and preprocessing phase compared to reading from a standard file server.
- **Cost-Efficiency and Redundancy:** Erasure Coding allows the system to survive hardware failures without data loss, while consuming less storage space than traditional replication methods. This is vital for maintaining a massive archive of media files economically.

**Vector Database** Traditional databases are designed for structured data and are inefficient for handling high-dimensional vector data generated by AI models. Vector databases are optimized for storing and querying high-dimensional vectors, which represent the semantic content of lecture segments. They utilize Approximate Nearest Neighbor (ANN) algorithms, such as Hierarchical Navigable Small World (HNSW) graphs, to enable efficient similarity search in latent spaces where traditional indexing methods fail.

The Vector Database is the backbone of the Retrieval-Augmented Generation (RAG) paradigm, allowing the system to perform semantic search based on the embeddings generated by the AI agents. When a user query is processed, it is transformed into an embedding vector, which is

then compared against the vectors stored in the database to retrieve the most semantically relevant knowledge segments. The adoption of a Vector Database is critical for enabling the advanced retrieval capabilities required for educational analysis.

Milvus is an open-source vector database engine. It indexes high-dimensional embeddings generated from course documents and lecture transcripts, providing sub-second retrieval times for semantic similarity queries across millions of vectors.

- **Efficient Similarity Search:** Traditional databases cannot efficiently query high-dimensional vectors. Vector databases provide specialized indexing structures to handle this efficiently.
- **Scalability for Large Datasets:** As the number of educational segments grows, the vector database can scale horizontally to maintain low latency for similarity searches, ensuring that retrieval times remain fast even as the dataset expands.
- **Integration with AI Workflows:** Vector databases are designed to integrate directly with AI pipelines, supporting real-time insertion and retrieval of embeddings.
- **Support for Dynamic Data:** As new educational materials are ingested and analyzed, their embeddings can be added to the vector database in real-time, allowing the system to continuously evolve and improve its retrieval capabilities without downtime.

**Graph Database** Neo4j is a native graph database well suited to storing the "Backbone Graph" of a knowledge base. It represents educational entities (concepts, lessons, modules) as nodes and their pedagogical relationships as directed edges.

- **Multi-hop Reasoning:** Unlike relational databases that require expensive JOIN operations, Neo4j allows for efficient traversal of deep relationship chains. This is critical for detecting knowledge gaps that may span multiple prerequisite levels.
- **Cypher Query Language:** The use of Cypher enables the system to express complex pedagogical patterns, such as "Find all prerequisite concepts of concept X that the student has not yet mastered," in a readable and performant manner.
- **Dynamic Schema:** The graph model allows the knowledge structure to evolve as new educational materials are added, without needing to redefine rigid table structures.

**Database Management and Observation** Managing multiple database engines requires a unified observation layer to ensure data consistency and facilitate debugging during development. DbGate is an open-source, multi-database manager. It provides a unified web interface to inspect and manipulate data across MySQL and MongoDB instances simultaneously.

- **Unified Interface:** Reduces the overhead of switching between multiple specialized management tools.
- **Schema Visualizer:** Helps developers understand the evolving structure of NoSQL documents and Relational tables in real-time.
- **Query Benchmarking:** Allows for testing and optimizing complex queries before they are integrated into the application's repository layer.

### 2.1.7 Deployment and Containerization

Deployment is the process of making a software application available for real-world usage. This section discusses the deployment strategy and the use of containerization to ensure that the system can be reliably and efficiently deployed for the AI pipeline and high-performance stacks discussed above.

Docker [6] is a containerization platform that enables developers to bundle applications and their required components into isolated units called containers. Each container acts as a lightweight, virtualized environment that includes all the necessary elements for the application to run. Because containers are portable and consistent, they work reliably across various environments, such as on-premise systems, cloud platforms, and clustered infrastructures.

- **Portability:** A container packages the application with its dependencies, so it behaves identically on a local machine or a cloud server.
- **Isolation:** Each container runs independently, so applications on the same host do not interfere with each other.
- **Resource Efficiency:** Containers share the host operating system kernel, so they use far less memory and disk than virtual machines.
- **Scalability:** Docker supports efficient scaling of applications. Docker Compose declares a multi-container system, with its services, networks, and volumes, in one file that starts with one command, and orchestration tools such as Kubernetes or Docker Swarm run many container instances across machines.
- **Faster Development and Deployment:** Developers work in reproducible environments identical to production, and deployment can be automated.

### 2.1.8 Security: Token-Based Authentication and Authorization

OAuth 2.0 is the standard authorization framework for web backends. Its password grant (Resource Owner Password Credentials) lets a client send a username and password straight to the application's own token endpoint, with no external identity provider. It fits an institution that is the only authority over its accounts. The issued credential is usually a JSON Web Token (JWT).

- **Stateless Bearer Tokens:** A JWT carries its claims, such as the user identifier and expiry time, together with a signature. The client sends it in the Authorization header of each request, and the server checks the signature without a session store, so any server instance can handle any request.
- **Signing and Expiry:** Tokens are signed with a server-held secret, commonly with HMAC-SHA256, so a tampered token fails verification. A short-lived access token is paired with a longer-lived refresh token, and keeping the refresh token in an HTTP-only cookie hides it from page scripts.
- **Password Storage:** Passwords are stored only as one-way hashes, so a leaked database does not expose them directly. Salted, deliberately slow functions such as bcrypt make brute-force guessing more expensive.

The verified token subject then identifies the user in every downstream data store, so each user reaches only their own records.

## 2.2 Related Work

### 2.2.1 Document-Knowledge Modeling

Converting documents into structured knowledge representations has long been a critical task in both industry and educational domains. To replace manual curation in educational contexts, which is time-consuming and not scalable, prior attempts have focused on entity extraction, prerequisite relation reasoning, and Retrieval-Augmented Generation (RAG). This has proven to be an effective strategy, demonstrated by early supervised models like PREREQ [18], which utilized latent topic representations with Siamese networks to infer course dependencies and achieved over 70% Precision@100 on the University Course and NPTEL MOOC datasets. However, traditional methods of entity extraction and relation reasoning often rely on rule-based systems or pure statistical machine learning techniques that struggle to capture the complexity and nuances of educational content relationships. Moreover, recent research [37, 20] demonstrates that models struggle to access relevant information when documents exceed a certain length. Consequently, although prior studies achieved impressive results on medium-scale retrieval benchmarks, they are difficult to apply to massive and unstructured enterprise datasets.

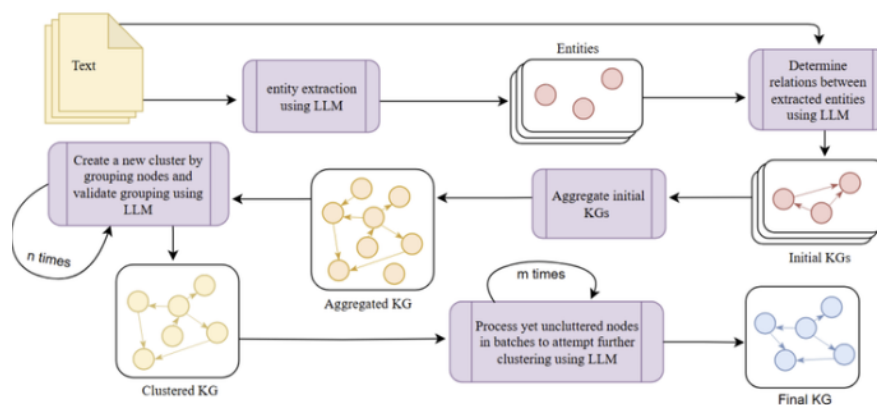


Figure 2.7: KGGen architecture [24]

To address these limitations, LLM-based frameworks have been widely proposed. A prime example is KGGen [24], an automated system that utilizes a clustering algorithm to group synonymous entities and relations, resulting in denser and better-connected graphs that scored an 81.03% average fact accuracy on the MINE-2 (Measure of Information in Nodes and Edges) benchmark. The EDC [39] framework extends this capability by extracting, automatically defining schemas, and canonicalizing knowledge triples without predefined schemas, consistently surpassing state-of-the-art baselines on complex datasets like WebNLG, REBEL, and Wiki-NRE.

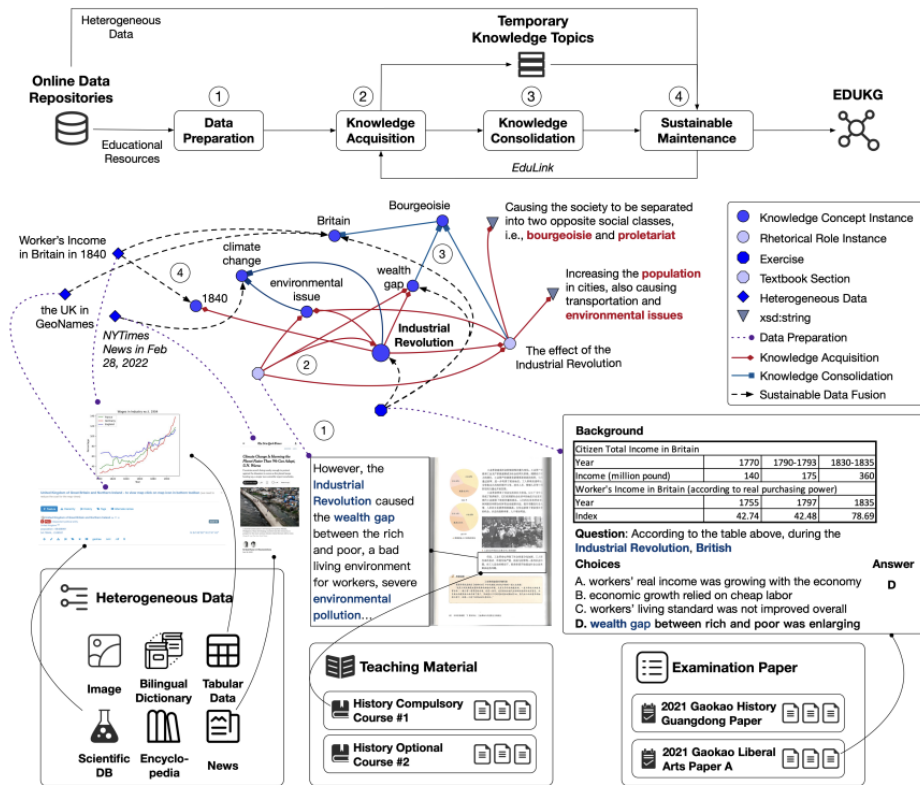


Figure 2.8: EduKG Pipeline framework [1]

However, LLMs have significant drawbacks. First and most popular of all is the "hallucinations problem", leading to inaccurate and poorly structured extractions. To mitigate this, GraphJudge [11] proposes using a fine-tuned open-source LLM as a "judge" to score and filter out incorrect knowledge triples generated by naive LLMs. Validated on general (REBEL-Sub, GenWiki) and domain-specific benchmarks (SCIERC, Re-DocRED), it achieved over 90% judgment accuracy. For large-scale educational and enterprise systems that must optimize cost and latency, hybrid pipelines have proven highly effective in practice. For instance, EduKG Pipeline [1] establishes an end-to-end framework fusing traditional text extraction with transformer-based weighting, demonstrating high precision on educational transcripts from the CCI dataset.

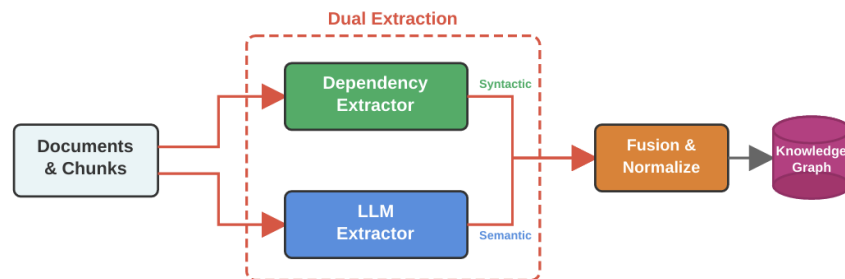
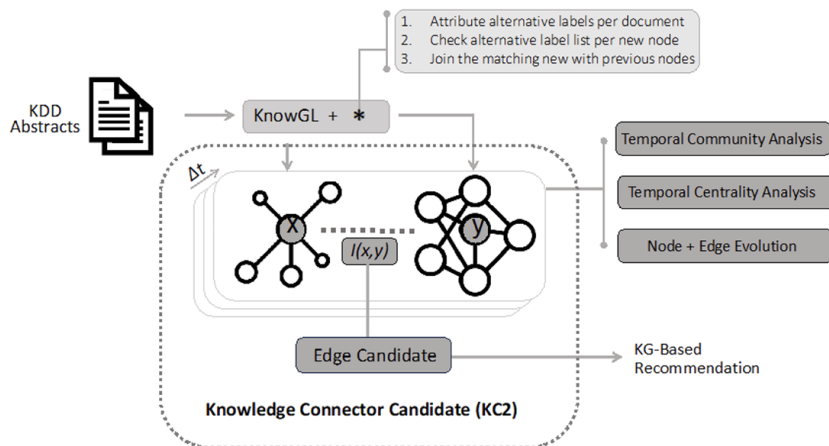


Figure 2.9: Hybrid GraphRAG architecture [22]

Similarly, efficient GraphRAG architectures [22] substitute expensive LLM extractions with classical dependency parsing to maintain scalability, retaining 94% of LLM-based performance on RAGAS metrics when evaluated on real-world enterprise datasets like CCM (Custom Code Migration). Finally, CoDe-KG [3] utilizes syntactic decomposition to process complex sentences, thereby significantly enhancing information recall by an 8-point gain on rare relations across the REBEL and WebNLG2 benchmarks.

Another inevitable error lies in the LLMs' limited context length, making them unable to gen-

eralize broad knowledge. They perform well when discovering short-dependent context, entities, or relationships in small sections, but fail for long dependencies in extremely long documents and course information. Trying to inject an excessive amount of information even when cut by windows, until now with the current level, only leads to redundant costs from multiple calls, information overload, and decreased accuracy. To uncover hidden relationships and deduce learning paths beyond explicit textual representations, several studies have shifted toward graph-algorithmic approaches.



**Figure 2.10:** *Dynamic Knowledge Graph (DKG) approach [4]*

The Dynamic Knowledge Graph (DKG) [4] is a prominent representative of this approach, analyzing macroscopic structures within Knowledge Graphs. Its KC2 (Knowledge Connector Candidate) algorithm uses closeness centrality to suggest connections between disconnected knowledge islands, and the Leiden [32] algorithm optimizes the Constant Potts Model quality function to cluster knowledge into well-connected chapters, avoiding a known limitation of Louvain. Splitting the graph into time snapshots additionally lets DKG track structural change, regroup clusters, and detect emerging knowledge communities.

A parallel line of work treats the graph itself as the engineering artifact. Graph Engineering [8] has advanced and gained popularity rapidly, driven by steadily improvement of frontier models and context-management techniques such as caching and context compaction. Together they let an agent read long documents and build its own knowledge structure, as in LLM-Wiki [23], without the context fragmentation that limited earlier graph construction. Yet an LLM residual error and costs always scale linearly with document length and complexity, and once complexity surpass a threshold, performance spiralling down. For this reason the state of the art in industry remains hybrid architecture such as GraphRAG [22] and DKG [4].

BookRAG [35] has recently emerged strongly in industry when it comes to long document ingestion. It does not let an LLM analyze full documents on its own as Graph Engineering does; instead it makes ingestion more structured and efficient by utilizing features that long documents already carry: layout, headings, and the table of contents, to build a hierarchical index (BookIndex) and to trace entity relations across its levels. The retriever gains granularity control and relational traversal without asking an LLM to rediscover the structure. The ordering of retrieval arms evaluated later in this report follows from this reasoning.

## 2.2.2 Generative AI in Interactive Teaching

Bridging the gap between raw document extraction algorithms and end-user educational platforms is the integration of Generative AI and Large Language Models (LLMs). The increasing accessibility and reasoning capabilities of Generative AI tools have led to widespread exploration in

educational settings, fundamentally transforming traditional teaching and learning paradigms [26]. Modern LLM agents have proven capable of autonomous reasoning, tool manipulation, and orchestrating complex academic tasks.

Recent surveys highlight the versatility of LLM agents in automating multifaceted educational workflows, ranging from curriculum design to generating highly contextualized feedback [5]. These agents operate dynamically, leveraging multi-hop reasoning and continuous interaction to adapt to a student's cognitive state. A qualitative study involving educators demonstrated that the core impact of generative models spans three critical pillars: active teaching and learning, administrative automation, and personalized assessments [26].

A prominent large-scale empirical example of this shift is the integration of AI within Harvard University's CS50 course. By deploying a suite of specialized AI software, the course approximated a 1:1 teacher-to-student ratio. Crucially, the system was designed with pedagogical boundaries, guiding students toward conceptual understanding rather than merely offering outright code solutions. This constrained autonomy proved highly effective in code explanation, style improvement, and administrative querying, demonstrating the practical viability of AI as a ubiquitous teaching assistant [21].

Despite these opportunities, deploying LLM agents in educational systems introduces substantial systemic and ethical challenges. The probabilistic nature of LLMs frequently leads to hallucination, where the model generates factually incorrect or flawed information. Theoretical analyses of LLM representations reveal phenomena such as "semantic collapse," where the continuous manifold of the model's activations struggles to maintain stable, logically sound ontologies over continuous, long-term reasoning tasks [38].

To address this limitation, research in educational AI applications has shifted from static next-token prediction to autonomous agentic systems, aiming for frameworks that tightly navigate semantic boundaries in an academic context to ensure the reliability, alignment, and transparency of AI outputs. Architectures like the Adaptive Evaluation Multi-Agent (AEMA) framework have been proposed to address this gap. AEMA provides an auditable, multi-step evaluation mechanism operating under human oversight, ensuring that agents' internal decision-making processes, tool selections, and collaborative strategies remain verifiable and transparent [16]. Implementing such verifiable frameworks is essential to mitigate the risks of unconstrained autonomy and ensure that AI integration in education remains trustworthy and pedagogically effective.

### **2.2.3 Educational Intelligent System**

Intelligent educational systems have transitioned from pure research into commercial applications. Many intelligent commercial platforms have emerged recently, alongside research institutions proposing full end-to-end pipelines beyond standard benchmark studies.

In the research domain, systems such as [40] (along with variants like K12EduKG and MOOC-KG) have been developed to uniformly model knowledge from textbooks and online courses. The [1] framework automates KG construction from unstructured PDF documents by extracting keywords via KeyBERT and performing semantic linking with LLMs. Furthermore, MEduKG pushes beyond text limitations by integrating multi-modal data into the graph, thereby enhancing the system's comprehensiveness and engagement for students.

In the commercial sector, popular platforms such as Khanmigo leverage generative LLMs to provide personalized tutoring and real-time feedback across extensive curriculum resources. In Vietnam, platforms such as VioEdu utilize manually curated knowledge trees to structure curricula, while EdMicro, specialized for enterprise training, employs skill matrices to map learning objectives and prerequisites. These systems represent distinct approaches to AI integration in education, each with unique strengths and limitations regarding scalability, personalization, and con-

tent structuring. Based on practical surveys and evaluation criteria, current educational systems exhibit diverse characteristics in AI adoption and knowledge structuring. A comparative analysis of these platforms is presented in Table 2.3.

| Criteria                                                                                                                                                           | EduKG | Khan Academy | Docebo | VioEdu (FPT) | EdMicro |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|--------------|--------|--------------|---------|
| <b>Adaptive learning:</b> Personalize the learning process based on needs, abilities, and preferences.                                                             |       | X            | X      | X            | X       |
| <b>Intelligent Recommendations:</b> Some personalized suggestions courses, videos based on preferences, and areas of improvement.                                  |       | X            |        | X            |         |
| <b>Natural language processing for queries:</b> Based on the questions from learners, it can give out the answer based on the content the LMS has in its DATABASE. | X     | X            |        |              |         |
| <b>Learning analytics/Progress Tracking:</b> AI-based analytics track and analyze user progress. Can visualize using tracking board, and leaderboard.              |       | X            | X      | X            | X       |
| <b>Text-to-audio content:</b> Provide audios for text content.                                                                                                     |       | X            |        |              |         |
| <b>Interactive learning method:</b> Interactive content, such as quizzes, coding exercises, and simulations, can engage learners and reinforce learning.           | X     | X            | X      | X            | X       |
| <b>Dynamic Roadmap:</b> Automatically extract concepts and reason long-term prerequisite learning paths without manual expert input.                               | X     |              |        |              |         |

**Table 2.3:** Comparative analysis of educational intelligent systems.

## 2.2.4 Conclusion and Research Gaps

Based on an extensive survey of the representative literature, including peer reviews and author self-evaluations, this report pinpoints the following core research gaps in the current landscape:

**Limitations in Automation and Cost Optimization:** Existing large-scale systems heavily depend on manually constructed knowledge trees or skill matrices. While course content is typically structured and ordered by domain expertise indexing, the lack of deep internal semantic modeling renders these systems less adaptive to individual student needs. This often results in effective high-level course recommendations but poor micro-level concept recommendations. Consequently, learning paths remain linear and rigid, with minimal real-time adaptation or automated content revision. This lack of automation restricts system scalability, as manual labeling becomes increasingly impractical as content repositories grow.

**Lack of Connectivity and Transparency in Lesson Structures:** Traditional text-mining and clustering methods often produce disconnected or fragmented knowledge groups. Existing learning platforms typically lack a standardized, mathematically verified network algorithm for modularizing study topics, resulting in disjointed learning paths that disrupt learners' cognitive flow.

**Static Roadmap Reasoning and Gap Detection:** While current Learning Management Systems (LMS) and AI tutors provide automated guidance, they rely primarily on traditional document

retrieval or standard RAG, lacking the network structure and deterministic reasoning required to map prerequisites dynamically. The identification of critical "knowledge gaps" remains static and reactive, failing to automatically trace bottlenecks back to fundamental foundational concepts using network centrality metrics.

To date, frameworks such as [1] and its lineages have made significant attempts to address these issues through an end-to-end pipeline that fuses traditional text extraction with transformer-based weighting. Conversely, approaches like [4] propose what [1] lacks: an efficient mathematical community detection method that resolves the connectivity and transparency issues in lesson structures encountered by earlier pipelines. However, both methodologies exhibit distinct limitations. The former lacks deterministic knowledge mapping, causing knowledge disruption even within unfragmented graphs; the latter is less efficient in multi-step extraction, making it valuable for static knowledge modeling and analysis but suboptimal for real-time inference and recommendation. Finally, many theoretical proposals remain unadopted due to the absence of a complete, production-grade platform capable of integrating these research advancements.

# Chapter 3

## Requirements Specifications

The limitations discussed in Subsection 2.2.4 have shaped the technical vision for the proposed system. To address these gaps and deliver a robust, scalable educational platform following our goal (Section 1.2), this section outlines the explicit Functional and Non-Functional Requirements, as well as visualizing use—case design. These requirements are derived directly from the systemic constraints and the pedagogical needs of learners for a more personalized, diversified and comprehensive learning assistance system.

### 3.1 Functional Requirements

#### 3.1.1 Knowledge Modelling and Management

As a key objective feature, knowledge modelling construct knowledge base, foundation to all other key features. This functional category encompasses the capabilities required to process raw educational inputs into a structured knowledge base.

- **Data Ingestion and Parsing:**

- The system must be able to ingest unstructured, text-dense academic formats (e.g., PDF textbooks, syllabus guidelines).
- The system must accurately parse text into discrete chunks while retaining crucial positional metadata (e.g., source file origin, exact page numbers).

- **Semantic Extraction and Normalization:**

- The system must extract fundamental educational concepts and their explicit pedagogical relationships from the ingested text.
- The system must normalize extracted entities to a canonical form to ensure conceptual consistency and prevent any form of semantic fragmentation (even of different course) across the knowledge base.

- **Hierarchical and Relational Organization:**

- The system must organize extracted entities into a standardized hierarchical topology.
- The system must automatically group highly related concepts into cohesive educational clusters or chapters.
- The system must explicitly define and store directional "prerequisite" relationships between concepts to enable automated learning path generation.

- **Storage Capability:**

- The persistence mechanism must guarantee a database structure that facilitates high-speed, complex relational querying and subgraph retrieval during live interactions.
- To ensure diversity of learning input, storage design must ensure efficient storing of enormous data with reasonable latency.

### 3.1.2 Intelligent Teaching Assistance

To utilize modeled knowledge base as well as guarantee satisfactory learning experience and result, the design of agentic system that serves as our Teaching Assistance is crucial. This category defines the required behaviors and reasoning capabilities of the automated teaching assistant.

- **Intent Recognition and Context Orchestration:**

- The system must classify user intents (e.g., Retrieve information, Generate roadmap, Clarify concepts) and autonomously route the request to the appropriate internal workflow.
- The system must dynamically gather necessary context before responding, which includes retrieving the student's personal mastery history and the relevant subset of the educational knowledge base.

- **Intelligent Reasoning and Supervision:**

- The system must support a Socratic teaching interaction; it must be capable of generating probing questions to assess student comprehension rather than merely providing direct answers.
- The system must objectively evaluate the student's response to determine whether to authorize progression to the next prerequisite topic.
- The system must be able to autonomously detect when it lacks sufficient context to answer a question and execute a fallback strategy that synthesizes information from the broader knowledge base.

- **Response Verifiability and Fallback:**

- To guarantee grounded responses, the system must prioritize explicit document referencing, citing exact source pages for its generated answers.
- The system must execute a broader knowledge synthesis fallback only when direct contextual concepts are missing from the immediate document.

### 3.1.3 Student Interaction and System Navigation

This category defines the functional requirements concerning the end-user's experience, accessibility, and direct interaction with the intelligent system.

- **Identity and Access Security:**

- The system must provide secure authentication capabilities (Login/Logout) to uniquely identify users.

- The system must maintain isolated, tamper-proof session states for each user to securely record individual learning progress without cross-user data exposure.
- **Interactive Dialogue and Visualization:**
  - The system must provide a natural language chat interface that supports continuous, multi-turn conversational interactions and real-time streaming responses.
  - The system must dynamically render visual learning roadmaps, empowering students to make informed, user-centric choices regarding their educational progression rather than following a static curriculum.
  - The interface must consolidate essential learning tools, providing seamlessly integrated functional components such as conversational history logs, a synchronized document viewer, and progress summarization dashboards.
- **Advanced Navigation and Content Discovery:**
  - The system must implement an intuitive, multi-level navigation structure (e.g., breadcrumb trails) to help students orient themselves within complex knowledge hierarchies.
  - The system must incorporate a powerful semantic search utility, allowing users to rapidly query and locate specific educational materials, graph concepts, or historical interactions.
- **Contextual and Responsive Feedback:**
  - The system must be capable of emitting explicit interface triggers alongside its textual responses to automatically synchronize the user's interface (e.g., auto-scrolling the document viewer to the exact referenced page).
  - Notifications and intelligent pedagogical prompts must be delivered in a non-intrusive, accessible manner that captures user attention while preserving the fluidity of the learning experience.

## 3.2 Non-Functional Requirements

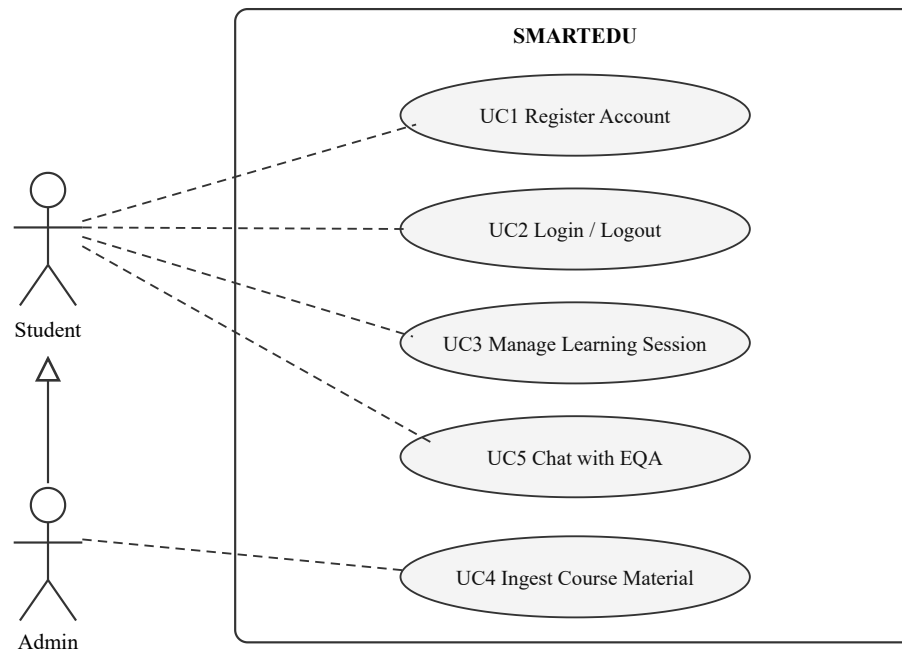
To create a powerful, reliable, and user-friendly educational platform, the Smart Learning System cannot stop at being functional, but need focusing on efficiency and overall system performance. Thus, it must also adhere to a set of non-functional requirements that govern its performance, scalability, security, and overall user experience.

- **Scalability and Modularity:**
  - The backend architecture must be designed as stateless, relying exclusively on external persistence layers for state management to ensure readiness for future distributed scaling.
  - System design must be modular, with clear separation of components responsibility and dependency to facilitate future enhancements, maintenance, and potential integration of additional features or third-party services.
- **Resource Optimization and Latency Minimization:**

- Operating under the constraint of localized execution, the system must optimize memory and computational overhead, loading resource-heavy models into memory only when explicitly demanded by an active workflow.
  - All resource must be efficiently released immediately after use to minimize the system's memory footprint and ensure responsiveness.
  - System must be scheduled to be constantly running, minimizing downtime and waiting time for any task.
  - All resource must be manageable, specifically for developers and system administrators, but crucial for maintaining system stability and performance.
- **Reliability and Determinism:**
    - The system must mitigate the inherent unpredictability of generative AI by enforcing strict, programmatic data schemas for all internal system communications and decision-making logic.
    - Complex analytical tasks, such as calculating knowledge gaps, must be executed via deterministic programmatic logic rather than relying on probabilistic generative approximations.
  - **Observability and Traceability:**
    - The system must maintain comprehensive telemetry across all automated workflows.
    - Database state changes, including knowledge graph updates and student progress logs, must be monitored and saved to enable historical analysis and rollback capabilities.
    - Every automated decision, context retrieval action, and resource consumption metric must be actively logged to provide a transparent audit trail for both system debugging and pedagogical evaluation.
  - **User Experience and Interface Requirements:**
    - User interface is focus on the simplicity and intuitiveness, ensuring that users of varying technical proficiency can navigate the system effectively.
    - User interface must be responsive and accessible, adhering to modern web standards to ensure compatibility across a wide range of devices and screen sizes (phones, tablets, desktops).
    - Client side shouldn't be pulled by heavy backend response latency. It should maintain a fluid, real-time interaction experience, minimizing latency in response generation and interface updates. Minimizing "user experience latency" regardless of system performance is a strategic design move to preserve conversational engagement as well as learning interest.

### 3.3 Use Cases

Two actors use the system: the student, who registers, signs in, manages a learning session and chats with the Educational QA Agent (EQA), and the administrator, who is a student account with the additional right to ingest course material. Figure 3.1 places the five use cases in one diagram, and the tables that follow specify each of them.



**Figure 3.1:** Use case diagram.

### 3.3.1 Use case 1: Register Account

|                         |                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name</b>    | Register Account                                                                                                                                                                                                                                                                                                                                    |
| <b>Actor</b>            | User (Student)                                                                                                                                                                                                                                                                                                                                      |
| <b>Description</b>      | The user registers a new account in the system to access personalized learning features.                                                                                                                                                                                                                                                            |
| <b>Preconditions</b>    | The user does not already have an account with the chosen student ID/username.                                                                                                                                                                                                                                                                      |
| <b>Postconditions</b>   | A new user account is successfully created in the database.                                                                                                                                                                                                                                                                                         |
| <b>Normal Flow</b>      | <ol style="list-style-type: none"> <li>1. User are navigated to the registration page.</li> <li>2. User enters student ID and password.</li> <li>3. System validates that the student ID is not already taken.</li> <li>4. System creates the account and securely stores credentials.</li> <li>5. User is redirected to the login page.</li> </ol> |
| <b>Alternative Flow</b> | <b>Username taken:</b><br>A1. User submits an already registered student ID.<br>A2. System displays an error message asking the user to choose another ID or log in.                                                                                                                                                                                |
| <b>Exceptions</b>       | If the backend database is unreachable, show a “Service temporarily unavailable” message.                                                                                                                                                                                                                                                           |

**Table 3.1:** Use case 1 Overview: Register Account

### 3.3.2 Use case 2: Login/Logout

|                         |                                                                                                                                                                                                                                                                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name</b>    | Login/Logout                                                                                                                                                                                                                                                                                                                                 |
| <b>Actor</b>            | User                                                                                                                                                                                                                                                                                                                                         |
| <b>Description</b>      | The user logs in to the system to access authorized features and logs out to end the session.                                                                                                                                                                                                                                                |
| <b>Preconditions</b>    | The user has a valid account.<br>The system is connected to the authentication service.                                                                                                                                                                                                                                                      |
| <b>Postconditions</b>   | User is successfully logged in (receiving a valid access token) or logged out.                                                                                                                                                                                                                                                               |
| <b>Normal Flow</b>      | <b>Login:</b><br>1. User navigates to the login page.<br>2. User enters username and password.<br>3. System validates credentials and generates JWT token.<br>4. User is redirected to the dashboard/homepage.<br><b>Logout:</b><br>5. User clicks the logout button.<br>6. System clears the session token and redirects to the login page. |
| <b>Alternative Flow</b> | <b>Invalid login attempt:</b><br>A1. User submits incorrect credentials.<br>A2. System displays an error message and asks user to retry.                                                                                                                                                                                                     |
| <b>Exceptions</b>       | If the authentication service is unavailable, show a “Service temporarily unavailable” message.                                                                                                                                                                                                                                              |

**Table 3.2:** Use case 2 Overview: Login/Logout

### 3.3.3 Use case 3: Manage Learning Session

|                         |                                                                                                                                                                                                                                                                                                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name</b>    | Manage Learning Session                                                                                                                                                                                                                                                                                                                                               |
| <b>Actor</b>            | User (Student)                                                                                                                                                                                                                                                                                                                                                        |
| <b>Description</b>      | The user starts a new learning session to begin interacting with the curriculum, and ends it upon finishing.                                                                                                                                                                                                                                                          |
| <b>Preconditions</b>    | The user is logged in and authenticated.                                                                                                                                                                                                                                                                                                                              |
| <b>Postconditions</b>   | A unique session state is initialized or terminated for tracking the student's progress and history.                                                                                                                                                                                                                                                                  |
| <b>Normal Flow</b>      | <b>Start Session:</b><br>1. User clicks to start studying.<br>2. System initializes a new session ID and prepares memory context.<br>3. System returns the session ID to the client interface.<br><b>End Session:</b><br>4. User leaves the learning interface or clicks end session.<br>5. System clears the active session context from memory but retains history. |
| <b>Alternative Flow</b> | <b>Session Timeout:</b><br>A1. User is inactive for an extended period.<br>A2. System automatically ends the session and prompts the user to start a new one upon return.                                                                                                                                                                                             |
| <b>Exceptions</b>       | If the session database fails to respond, gracefully fall back and notify the user to refresh the page.                                                                                                                                                                                                                                                               |

**Table 3.3:** *Use case 3 Overview: Manage Learning Session*

### 3.3.4 Use case 4: Ingest Course Material

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name</b>    | Ingest Course Material                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Actor</b>            | Administrator / Instructor                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Description</b>      | The actor uploads course slides and textbooks so the system can extract concepts and build the Educational Knowledge Graph.                                                                                                                                                                                                                                                                                          |
| <b>Preconditions</b>    | The actor has valid administrative or instructional privileges.<br>The raw PDF files are accessible.                                                                                                                                                                                                                                                                                                                 |
| <b>Postconditions</b>   | The knowledge base (Neo4j, Milvus) is populated with parsed nodes, embeddings, and relationships.                                                                                                                                                                                                                                                                                                                    |
| <b>Normal Flow</b>      | <ol style="list-style-type: none"><li>1. Actor submits the course name and target files via the ingestion endpoint.</li><li>2. System immediately returns a 200 SUCCESS status, indicating the task is running in the background.</li><li>3. System parses PDFs, segments windows, and extracts concepts (Dual-Phase Extraction).</li><li>4. System constructs the final Knowledge Graph and Vector store.</li></ol> |
| <b>Alternative Flow</b> | <b>Invalid file format:</b><br>A1. Actor uploads corrupted or unsupported file types.<br>A2. System aborts ingestion and logs an error without modifying the database.                                                                                                                                                                                                                                               |
| <b>Exceptions</b>       | If the LLM provider for extraction times out, the ingestion task pauses or retries based on internal middleware logic.                                                                                                                                                                                                                                                                                               |

**Table 3.4:** *Use case 4 Overview: Ingest Course Material*

### 3.3.5 Use case 5: Chat with EQA

|                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Use Case Name</b>    | Chat with EQA                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>Actor</b>            | User (Student)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Description</b>      | The user submits a learning query to the Educational QA Agent and receives an educational response or pedagogical guidance.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Preconditions</b>    | The user is logged in and has an active session ID.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Postconditions</b>   | The user receives an appropriate textual answer, and UI actions (like PDF page scrolling) are triggered.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| <b>Normal Flow</b>      | <ol style="list-style-type: none"><li>1. User submits a query (e.g., “What is Machine Learning?”) in the chat interface.</li><li>2. System immediately accepts the request and returns a unique <code>task_id</code>.</li><li>3. Client continuously polls the status of the <code>task_id</code> in the background.</li><li>4. EQA orchestrates the response using the workflow (e.g., Teach, Retrieve, Roadmap).</li><li>5. Upon completion, the polling mechanism receives the final output (text and UI actions).</li><li>6. Client renders the response and updates the UI (e.g., highlights document).</li></ol> |
| <b>Alternative Flow</b> | <b>EQA Workflow Failure:</b><br>A1. The EQA encounters an internal error during orchestration.<br>A2. The polling endpoint returns a ‘Fail’ status with error details.<br>A3. Client notifies the user that the EQA encountered a problem.                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Exceptions</b>       | Polling timeout: If the EQA takes too long to respond (e.g., > 300s), the client aborts polling and alerts the user.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

**Table 3.5:** *Use case 5 Overview: Chat with EQA*

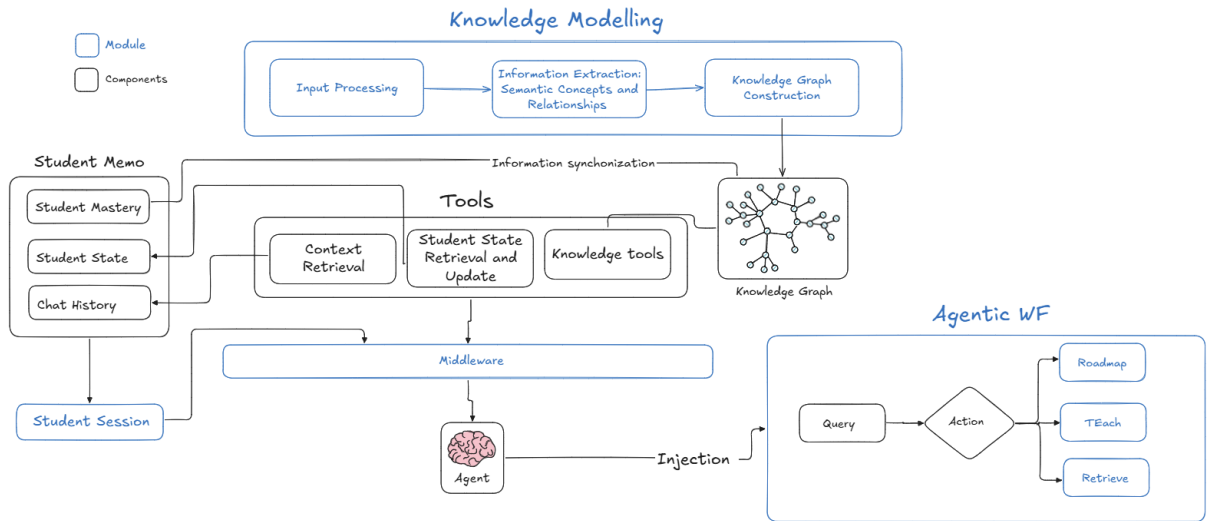
# Chapter 4

## Proposed Methodology

Founded on the vision outlined in the requirements specification (Chapter 3), this research aims to propose a novel system that combines the strengths of classical Graph-based architectures with the generative capabilities of Large Language Models (LLMs). This chapter details the foundational methodology of our system.

### 4.1 Overview

The proposed paradigm can be visualized as Figure 4.1.



**Figure 4.1:** High-level system workflow

A key security boundary is established at the system's entry point. Upon user authentication, the student's Session and Memo contexts are directly injected into the computational tools. As a result, the agents use these pre-configured tools instead of interacting directly with the database. This approach ensures strict data isolation and prevents the LLM from making unauthorized or destructive database transactions.

### 4.2 Knowledge Modeling

SmartEdu anchors the generative flexibility of LLMs to a deterministic structure by modeling the educational domain as a property graph, combining semantic richness with logical consistency.

### 4.2.1 Graph Definition

The graph holds an ontology of educational concepts and the relationships between them, such as prerequisites. This subsection gives its formal representation and the rules that govern it (Figure 4.2).

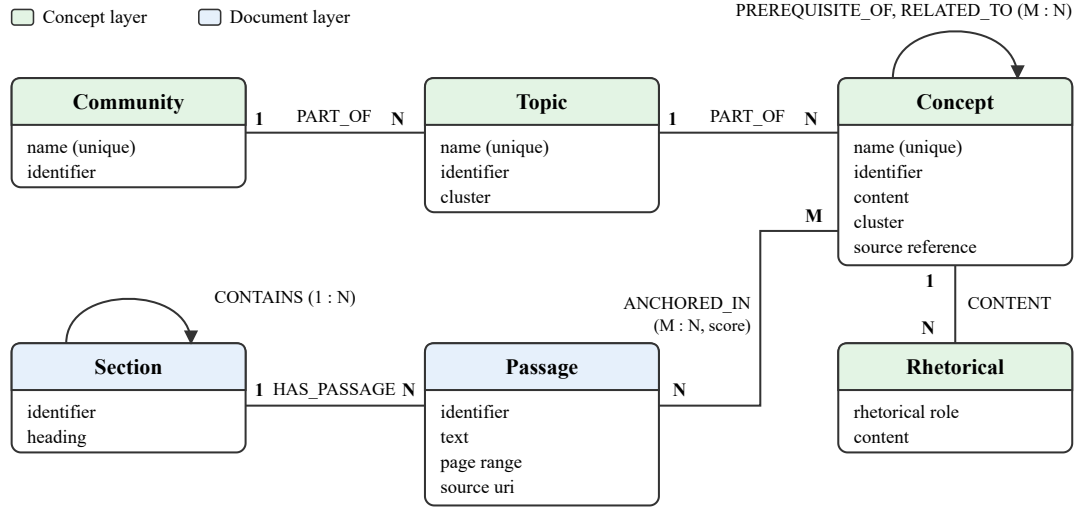


Figure 4.2: Entity-relationship diagram of the EKG

**Formal Property and Topological Definition** A property graph lets both vertices and edges carry properties, so a concept can hold definitions, formulas, and context rather than being a bare node. The Educational Knowledge Graph (EKG) is a heterogeneous, directed property graph  $G = (V, E, \Lambda, \Pi)$ , where:

- $V$ : educational entities at several levels of abstraction.
- $E \subseteq V \times V$ : directed semantic and structural relationships between them.
- $\Lambda$ : a finite, predefined label set that classifies nodes and prevents semantic drift.
- $\Pi$ : key-value properties  $\pi_{key} = value$  on nodes and edges, holding metadata, raw text, and dynamically updated metrics.

Treating the curriculum as a set  $V$  and a mapping  $E$  lets graph algorithms such as topological sorting and shortest-path routing run directly on it.

**Ontological Hierarchy formulation** From whole disciplines down to single explanatory sentences, the label set is split into a concept layer and a document layer,  $\Lambda = \Lambda_c \cup \Lambda_d$ :

- **CommunityNode**: a discipline or course-level cluster (e.g., “Artificial Intelligence”).
- **TopicNode**: a module or chapter nested in one community (e.g., “Neural Networks”).
- **ConceptNode**: the atomic unit of instruction, an indivisible idea, theory, or algorithm (e.g., “Backpropagation”).
- **RhetoricalNode**: the instructional function of a text span, given by its role  $\pi_{rrole}$  (e.g., Definition, Application, Theorem), separating what a concept is from how it is explained.

- **SectionNode**: a textbook heading taken from the document's own table of contents, not inferred by a model (Section 4.3.1).
- **PassageNode**: the text under one section with its page range, returned as evidence and cited by page.

**Edge Semantics and Constraints** Each edge is an ordered triple  $e = (v_i, v_j, \lambda_e)$ , typed by layer:

- **Concept layer**: the semantic types **PART\_OF**, **RELATED\_TO**, and **PREREQUISITE\_OF**, plus **CONTENT** from a concept to its rhetorical nodes.
- **Document layer**: **CONTAINS** for the heading tree and **HAS\_PASSAGE** from a section to its passages.
- **Across layers**: **ANCHORED\_IN**, from a concept to the passages that explain it.

Only the semantic types enter centrality and path computations. Knowledge is sequential (Algebra before Calculus), so the prerequisite sub-graph  $G_{prereq} = (V, E_{prereq})$ , with  $E_{prereq} = \{e \in E \mid \lambda_e = \text{PREREQUISITE\_OF}\}$ , must be a Directed Acyclic Graph (DAG):

$$\forall v \in V, \nexists \text{ path } P = (v, \dots, v) \text{ in } G_{prereq} \quad (4.1)$$

A cycle would leave the student unable to start any of its concepts and would break the topological sort that generates learning roadmaps.

### 4.3 Content Processing and Ingestion

A major challenge in building an automated knowledge base is extracting structured relationships from unstructured educational text. Single-pass LLM extractions—where the model tries to identify concepts, classify them, and link them all at once—are highly prone to failure. Multiple frameworks propose multi-turn LLM synthesis, but it is costly and still does not yield a hallucination-free graph, as each additional turn increases entropy and the total context. I propose a hybrid method that combines Dual-Phase LLM extraction with NLP-based filtering and normalization to obtain a consistent graph.

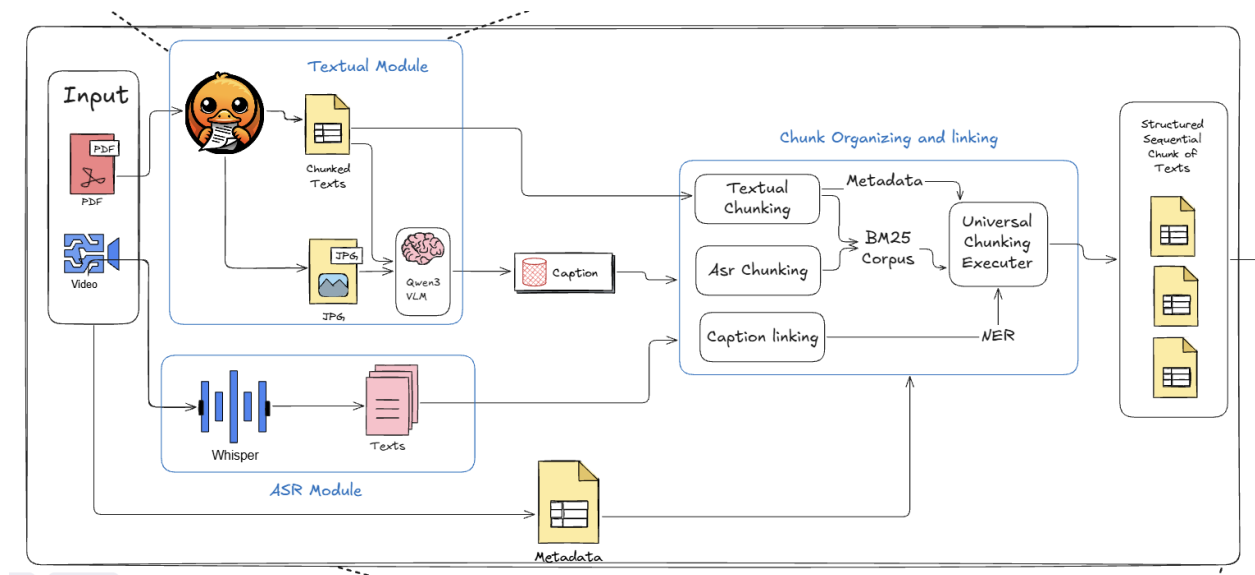


Figure 4.3: Ingestion Pipeline

The two kinds of course material need different treatment. Lecture slides are sparse: a language model has to recover the concepts behind their bullet points. Textbooks already carry an authored structure of chapters and nested headings, so paying a model to rediscover it would only add cost and hallucination risk. The pipeline therefore has two routes: a textbook route, described first, and a slide route that runs through the remaining subsections of this section.

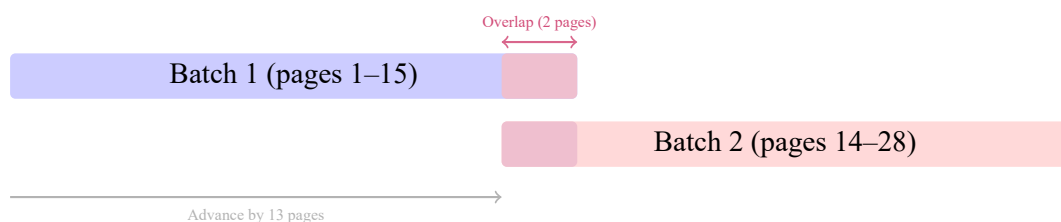
### 4.3.1 BookRAG and Docling Integration

Following the BookRAG principle [35], a hierarchy that a document already exposes is treated as ground-truth structure and is not re-inferred by a language model. Docling (Section 2.1.5) converts a textbook into a document model that keeps the heading tree, and the content is split along the book's own section boundaries. Authored sections become structural nodes, and the text under each section becomes passage nodes attached to them. The textbook's structure is thus obtained by deterministic parsing, not by generation.

Slides play the opposite role. The vocabulary of a course is what the instructor chose to teach, so concepts are created from slides, through the extraction pipeline of the next subsections, while textbooks supply the detailed evidence. Once both routes finish, each concept is linked to the textbook passages that explain it (an ANCHORED\_IN edge), by embedding similarity restricted to the same course and cut off at a minimum score. A textbook's wide coverage therefore cannot flood the graph with concepts outside the course, and the tutoring agent can still retrieve full-length explanations instead of a slide's compressed fragments. Language-model calls are spent only on relating concepts.

### 4.3.2 Input Ingestion

The slide route starts by cutting each deck into batches that fit the extraction model's context budget without splitting an explanation at a page break. A document parser converts the PDF into page-indexed text blocks, dropping non-informational elements such as lecturer contact details, rendering artifacts, and fragments shorter than five characters. Pages are grouped into batches of fifteen pages by default, and the window steps forward by thirteen, so adjacent batches share two boundary pages.



**Figure 4.4:** *Overlapping window segmentation.*

The main purpose of this design is cross-boundary continuity: an explanation that spans a page break is captured intact in at least one batch, so the extraction model never receives truncated input and the overlap keeps knowledge islands connected. The redundancy also degrades gracefully: if the LLM fails on one batch, its neighbour still holds the boundary content.

### 4.3.3 Dual Phase LLM Extraction

Dual Phase Extraction Pipeline is one that methodologically decouples top-down hierarchical decomposition from lateral relational mapping. Let  $T$  represent the continuous corpus of unstructured

educational text, ingested from raw documents. The overall extraction objective is to discover an approximation function  $\Phi : T \rightarrow (V, E)$  that maximizes semantic fidelity. We decompose  $\Phi$  into two sequential, deterministic mappings.

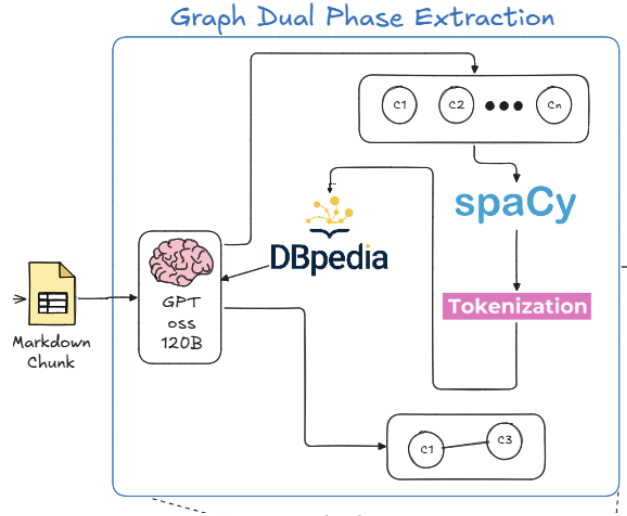


Figure 4.5: Dual Phase Extraction

### Phase 1: Top-Down Hierarchical Extraction (Skeleton Phase)

The primary objective of the first phase is to establish the taxonomic structure of the knowledge domain without the cognitive burden of cross-referencing. The LLM's action space is strictly confined to identifying entities and their subsumption (parent-child) relationships.

We define the skeleton extraction function  $\Phi_{\text{skeleton}}$  as:

$$\Phi_{\text{skeleton}} : T \rightarrow (V_{\text{tree}}, E_{\text{tree}}) \quad (4.2)$$

where  $E_{\text{tree}} \subset V_{\text{tree}} \times V_{\text{tree}}$  contains strictly hierarchical edges (e.g., *PART\_OF*).

To eliminate ontological drift, the LLM is forced to output data conforming to a rigid, predefined syntactic schema (e.g., strict JSON schema validation). The system parses this structural output to instantiate *CommunityNode* and *TopicNode* entities, ensuring that every atomic concept is firmly anchored to a macroscopic curriculum node.

Mathematically, this phase yields disjoint taxonomic trees. Because the LLM is only tasked with hierarchical assignment, the spatial complexity is bounded to  $O(N)$  parent-child assignments. This drastic reduction in the problem space significantly mitigates the probability of generation errors. To complete the graph, phase 2 is executed subsequently.

### Phase 2: Lateral Relational Mapping (Relation Phase)

With the hierarchical skeleton established as the logically structured frame, the second phase focuses on inferring the non-hierarchical, semantic linkages across different branches of the tree. Given the isolated, immutable entity set  $V_{\text{tree}}$ , the relational extraction function  $\Phi_{\text{relation}}$  is defined as:

$$\Phi_{\text{relation}} : (V_{\text{tree}}, T) \rightarrow E_{\text{lateral}} \quad (4.3)$$

where  $E_{\text{lateral}}$  encompasses cross-cutting educational dependencies such as *PREREQUISITE\_OF* or *RELATED\_TO*.

By presenting the LLM with a predefined, validated list of concepts ( $V_{\text{tree}}$ ) and tasking it only with edge generation, the problem space is reduced to a binary classification task over possible

pairs. The final Educational Knowledge Graph is the formal union of these two sub-spaces:

$$G = (V_{\text{tree}}, E_{\text{tree}} \cup E_{\text{lateral}}) \quad (4.4)$$

To summarize, candidate concepts are first extracted as taxonomy trees, which are then combined into a graph. This paradigm improves the consistency of the generated graph. It bridges the gap between unstructured educational data and a structured property graph by constraining the LLM’s action space, which partially prevents hallucinations. Partially, because a problem remains: Phase 2 relies on Phase 1 for a structured and stable output, but Phase 1 itself may produce a poor skeleton. To reconstruct and verify the skeleton, we use the deterministic techniques described next.

#### 4.3.4 NLP-Based Concept Filtering and Verification

Dual-Phase extraction constrains each step with the result of the previous one, but the raw output of Phase 1 is still inconsistent: one concept appears under several lexical forms, some concepts do not fit the course, and some are too granular because they are mentioned only once. Using an LLM to correct an LLM has its limits, so a deterministic NLP pipeline, inspired by BPE tokenization, runs between the two phases. It turns the candidate set  $V_{\text{tree}}$  into a verified set  $V^* \subseteq V_{\text{tree}}$  in four steps.

**Step 1 — Normalization.** Each label is lowercased, stripped of punctuation, lemmatized and cleared of stop words  $S$ , then split into unigrams and bigrams:

$$N(v_i) = \text{tokenize}(\text{lemmatize}(\text{lower}(v_i))) \setminus S \quad (4.5)$$

Later steps compare these token sets instead of raw strings.

**Step 2 — Wikidata Validation.** Each concept is searched in Wikidata, and the  $k$  candidates  $L_k$  are scored by how well their descriptions match a weighted list of subject keywords (e.g., “algorithm”, “programming”). The concept is matched only if the best candidate passes the threshold  $\theta$ :

$$l^* = \arg \max_{l \in L_k} \text{relevance}(l, v_i), \quad \text{relevance}(l^*, v_i) \geq \theta \quad (4.6)$$

A matched concept takes the Wikidata identifier and label, so the same idea from different documents resolves to one node; a parent topic inferred in Step 3 is also accepted only if it has a match. An unmatched concept may be a hallucination, a pseudo-entity, or a recent term Wikidata does not cover yet. It is kept as a local, course-specific concept if its label passes a lexical check (at least two words, ending in a noun, no long digit runs), and pruned otherwise.

**Step 3 — Implicit Clustering.** Retained concepts are compared pairwise by Jaccard similarity:

$$J(v_i, v_j) = \frac{|N(v_i) \cap N(v_j)|}{|N(v_i) \cup N(v_j)|} \quad (4.7)$$

Pairs with  $J \geq \tau_{\text{merge}}$  are merged into one canonical node. Pairs that share tokens below that threshold form an implicit cluster whose shared tokens name a parent TopicNode; “Machine Learning Algorithms” and “Machine Learning Optimization” yield “Machine Learning”. These topics confirm or complete the skeleton of Phase 1.

**Step 4 — Node Caching.** Each verified concept is cached under its token set  $N(v_i)$ . The same concept met in another document resolves to the same node without re-running the pipeline, new material only verifies new concepts, and the Relation Phase receives a fixed, pre-validated entity list  $V^*$ .

## 4.4 Agentic System

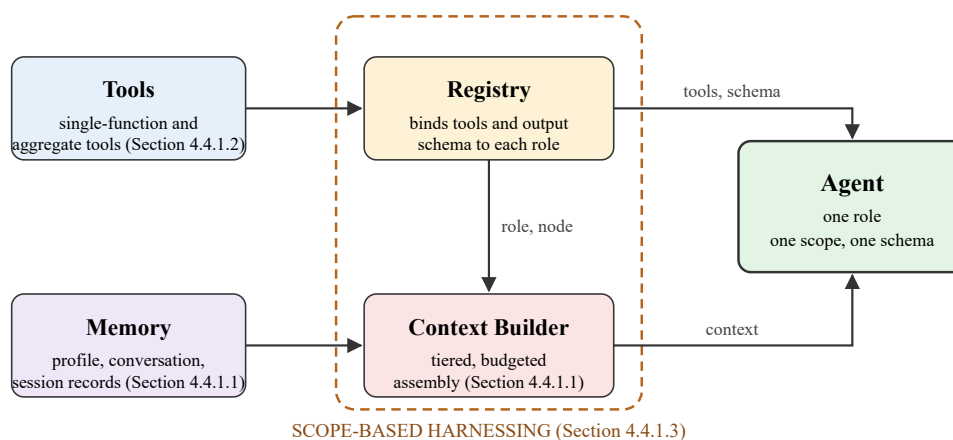
The knowledge graph  $G$  serves as the static, structural foundation of the system. However, real-time interaction with a human learner demands a dynamic, adaptive reasoning engine capable of navigating  $G$  to produce personalized educational responses. A single monolithic language model, no matter how capable, cannot reliably handle the diverse reasoning tasks involved in tutoring—from classifying student intent, to retrieving factual content with minimal cost and latency, to generating structured lectures, to evaluating learning outcomes. Each of these tasks requires a different workflow design.

The proposed system therefore decomposes tutoring into a multi-agent system in which every agent is a language model wrapped in its own harness. Section 4.4.1 defines the harness: what each agent remembers, sees, and can call. Section 4.4.2 defines the orchestration: how work is routed and delegated among agents.

### 4.4.1 Agent Harnessing

An agent is more than its language model. The Agent Harness is everything around the model: what it remembers, what it is shown at each step, which tools it may call, and in what format it must answer. Common practice binds every agent to one fixed prompt, one fixed tool list, and the whole chat history, which fails in tutoring in the ways listed below.

The proposed harness follows the Single Paradigm: one responsibility per agent, tied to one instruction set, one context, one tool set, and one output schema. It separates four concerns (Figure 4.6). Memory keeps what happened. The Context Builder decides how much of it an agent reads. Tools define what an agent can do. The Registry binds tools and output schema to each role, so prompts carry no format instructions. Sections 4.4.1.1 and 4.4.1.2 describe how the harness obtains this material; Scope (Section 4.4.1.3) decides which of it each role receives.



**Figure 4.6:** The proposed agent harness and its four concerns.

Each benefit of the separation answers one failure of common practice:

1. Bounded context: history and tool output grow every turn until noise drowns the signal.

Every slice now has a budget, so what the model sees is deterministic and can be logged and audited.

2. Narrow roles: a classifier, a lecturer, and a search worker receiving the same tools and information are each tempted by options they should never use. A search worker now never sees the conversation, and the answerer sees compact, cited evidence instead of raw results.
3. Independent change: when storing, selecting, and acting live in one prompt, changing one silently breaks the others. The history policy is now swapped without touching tools, and a tool added without touching prompts.
4. Central registry: agents hard-coded in workflow nodes scatter their behaviour across the code. Declared in one place, it is inspected and debugged there.

#### 4.4.1.1 Context Engineering and Management

The same memory can show an agent too little to stay coherent or too much to reason well. The Context Builder assembles every context from four tiers, each with its own source, priority, and budget (Figure 4.7). Assembly is deterministic code, not a model call, so its cost is predictable and its output can be logged.

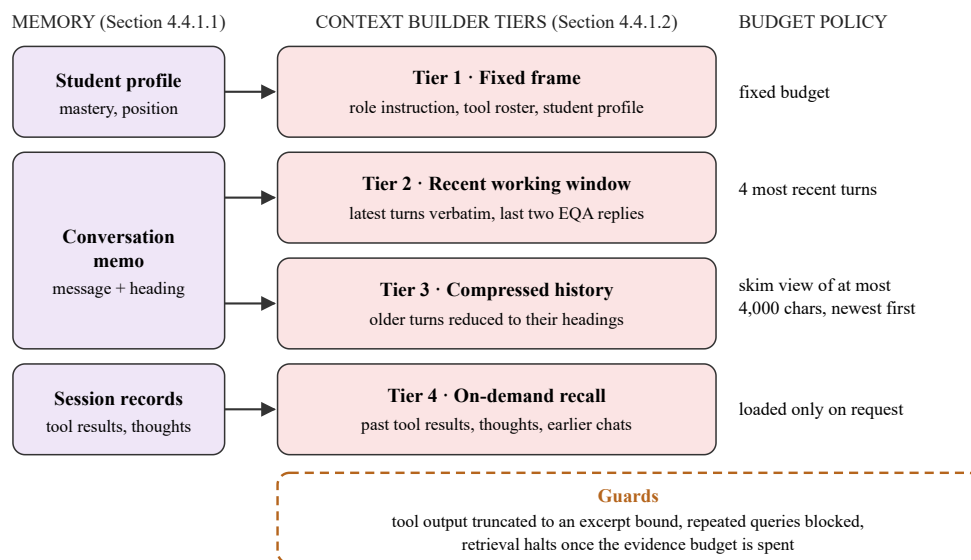


Figure 4.7: Context Builder tiers and budgets.

- Tier 1, Fixed frame: role instruction, tool roster, and a compact student profile; fixed budget.
- Tier 2, Recent window: the latest turns verbatim plus the tutor's last two replies; the workflow entry carries four turns in full.
- Tier 3, Compressed history: older turns reduced to headings, capped at 4,000 characters and filled newest first; the turn being answered is excluded.
- Tier 4, On-demand recall: past tool results, reasoning, and earlier chats stay out of context until an agent asks for them.
- Guards bound what the agents themselves produce: tool output is cut to fixed-length excerpts, repeated queries are refused, and retrieval halts once the next round could exceed the evidence budget.

#### 4.4.1.2 Tools

Direct LLM access to the system’s databases (Neo4j, Milvus, MongoDB, MinIO) risks malformed queries, unauthorized access, and destructive writes. A tool abstraction layer therefore sits between each agent and the underlying infrastructure: each tool exposes exactly one well-defined function, and agents have no direct access to raw query languages.

These tools are pre-configured with the student’s session before any agent invocation: they carry only the session identifier, and the student’s identity is resolved behind the Identity Firewall (Section 5.4), so each call is a simple, self-contained operation from the agent’s perspective. This design simultaneously simplifies the agent’s decision space and enforces strict data isolation across sessions.

Some needs are not a single lookup. Gathering evidence means querying several stores that index the same material differently and merging the results. Leaving the model to sequence and merge those calls is slow and inconsistent, so the harness offers aggregate tools: one call fans out to several specialized tools concurrently and returns one fused, ranked result. The agent decides what to ask and which sources to use, not how results are combined. Table 4.1 lists the tools.

| Tool Name                      | Group                 | Description                                                                                      | Parameters               | Database           |
|--------------------------------|-----------------------|--------------------------------------------------------------------------------------------------|--------------------------|--------------------|
| Context tools                  |                       |                                                                                                  |                          |                    |
| Student roadmap                | Student context       | Retrieve current student learning path and progress.                                             | session_id               | MongoDB            |
| Student chat history retrieval | Student context       | Retrieve recent conversation history for context injection.                                      | session_id, limit        | MongoDB            |
| Mastery Mapping                | Student context       | Fetch student mastery scores for concepts.                                                       | session_id               | MongoDB            |
| Recall Tool Results            | System context        | Retrieve recent tool execution results to prevent redundant calls.                               | limit, agent_type        | Logger (In-memory) |
| Recall Thoughts                | System context        | Retrieve recent agent reasoning traces for self-correction.                                      | limit                    | Logger (In-memory) |
| Knowledge tools                |                       |                                                                                                  |                          |                    |
| Recommend New                  | Graph Analytics       | Find top hub nodes ranked by out-degree for a starting point.                                    | course_filter, from_node | Neo4j              |
| Course Backbone                | Graph Analytics       | Extract course skeletal structure via top hub concepts and relations.                            | course_name, max_hubs    | Neo4j              |
| Optimal Path                   | Graph Analytics       | Find the shortest weighted path between two concepts.                                            | start_node, end_node     | Neo4j              |
| Concept Retrieval              | Graph Retrieval       | Resolve entity names to canonical IDs using graph or Wikidata.                                   | query                    | Neo4j              |
| Edge Explorer                  | Graph Retrieval       | Explore multi-hop semantic relationships (excluding CONTENT edges).                              | node_id                  | Neo4j              |
| Rhetorical retrieval           | Graph Retrieval       | Retrieve educational content nodes by rhetorical role.                                           | node_id, role, limit     | Neo4j              |
| Get Page                       | Full content reading  | Extract literal text from specific pages of a course document.                                   | pages, destination       | MinIO              |
| Get Concept Page               | Full content reading  | Quickly locate page numbers referencing a specific concept.                                      | concept                  | Neo4j              |
| Retrieval tools                |                       |                                                                                                  |                          |                    |
| Semantic Search                | Retrieval             | Search course concept content by vector similarity.                                              | query                    | Milvus             |
| Textbook Search                | Retrieval             | Hybrid dense and lexical passage search, extended along document-graph links and entity anchors. | query, seed evidence     | Neo4j              |
| Aggregate Search               | Retrieval (aggregate) | Run the permitted sources concurrently and fuse their rankings into one result.                  | query, sources           | Milvus, Neo4j      |

**Table 4.1:** Tools available to the EQA.

#### 4.4.1.3 Scope-based Harnessing

Scope decides which of the material above a role receives, along four dimensions: context tiers, tool set, output schema, and model. It is fixed per role, not per prompt, so roles can be compared and changed one at a time (Table 4.2).

| Role             | Context received                                        | Tools                                    | Output                           |
|------------------|---------------------------------------------------------|------------------------------------------|----------------------------------|
| Intent router    | Fixed frame, skim view of history, the query            | None                                     | One intent label                 |
| Retrieval worker | The query or one focused follow-up only, no history     | Graph / Vector / LayoutTree search tools | Ranked candidate passages        |
| Orchestrator     | Question, permitted sources, decisions so far, evidence | None                                     | Stop, or one delegated follow-up |
| Finalizer        | Question, decisions, complete evidence                  | None                                     | Claims, each citing its evidence |
| Answerer         | Question, finalizer synthesis, compiled evidence slice  | None                                     | A short answer                   |
| Tutor            | Tiers 1–4, retrieved content, student profile           | Page reading, recall                     | Structured lecture               |

**Table 4.2:** *Scope of each agent role.*

Two strategies follow from the table:

- Workers, stateless and task-scoped: see the task and its evidence, never the student or the conversation; narrow, read-only tools make them safe to run in parallel.
- Tutor, stateful and student-scoped: sees the profile and history; its tools reach the learner’s records and the course documents, and its content comes from page lookup or the retrieval stage.

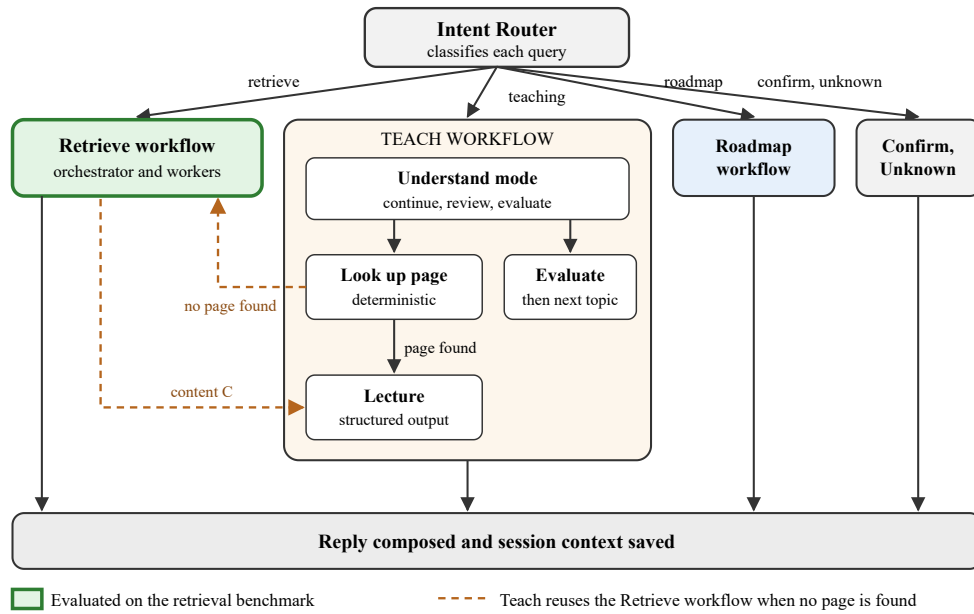
Scope narrows toward the leaves of the orchestration: the further an agent is from the student, the less of the student it needs to know. It is also enforced at run time: a delegation that names a source outside the permitted set, or cites evidence absent from the evidence, is rejected and repaired once before any search runs.

### 4.4.2 Agent Orchestration

The Educational QA Agent is a hierarchical state machine. A router classifies each query into one of five intents and dispatches it to a sub-workflow; inside a workflow, an orchestrator may delegate to workers, and a final synthesis node formats the response and persists the session state (Figure 4.8). The router is a few-shot classifier with boundary-case examples. Given the learning state, a skim-mode chat summary and the query, it returns one label from { retrieve, roadmap, teaching, confirm, unknown} at temperature zero, with no tools and no reasoning chain.

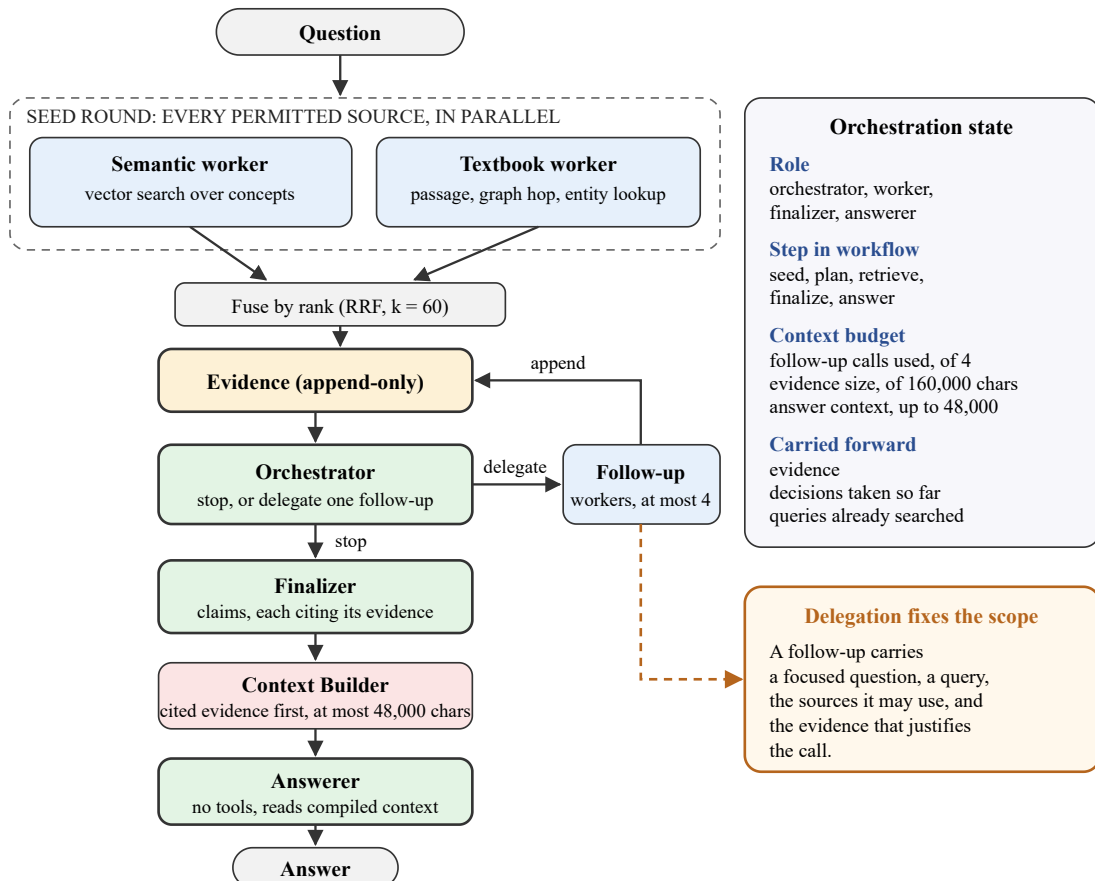
#### 4.4.2.1 Workflow: Retrieve (GraphRAG)

Retrieval answers factual questions from the course material. A single tool-calling agent is slow once a question needs several documents, since latency and context grow with every hop, and it is a



**Figure 4.8:** *Top-level workflow routing.*

single judge, so a passage discarded early is lost. The workflow is therefore an orchestrator–worker system (Figure 4.9) that tracks the evidence, the decisions taken and the queries already searched.



**Figure 4.9:** *Orchestration of one question and its tracked state.*

Four workers search the material: Semantic (vector search over slide concepts), Textbook (hybrid dense and lexical search over passages), Graph hop (walks document-graph links from evidence already found) and Entity (fetches the passages anchored to entities in the query). The orchestrator addresses two sources, semantic and textbook, the latter covering the last three workers. Sources run concurrently and are fused by Reciprocal Rank Fusion ( $k = 60$ ). The RAG preset

of Section 4.5 permits the semantic source only; the BookRAG preset permits both. Three rules govern the orchestration:

- Append-only evidence: fusion orders evidence but never removes it.
- Bounded delegation: at most four follow-ups, each one focused query citing its justifying evidence; repeated searches are refused.
- Separated answering: the finalizer binds each claim to its evidence, the Context Builder compiles it (Section 4.4.1.1), and a tool-free answerer replies.

Delegation is sequential across hops, so latency grows with the number of hops (Table 4.4).

#### 4.4.2.2 Workflow: Teach (Agentic Long Memory)

The Teach workflow is a Socratic loop: deliver content, elicit a response, assess understanding, then advance or reinforce. An entry classifier chooses one of three modes from the message and the session state: *continue* (advance), *review* (revisit) or *evaluate* (assess). Only *evaluate* changes, or deliberately keeps, the student's position (Figure 4.10).

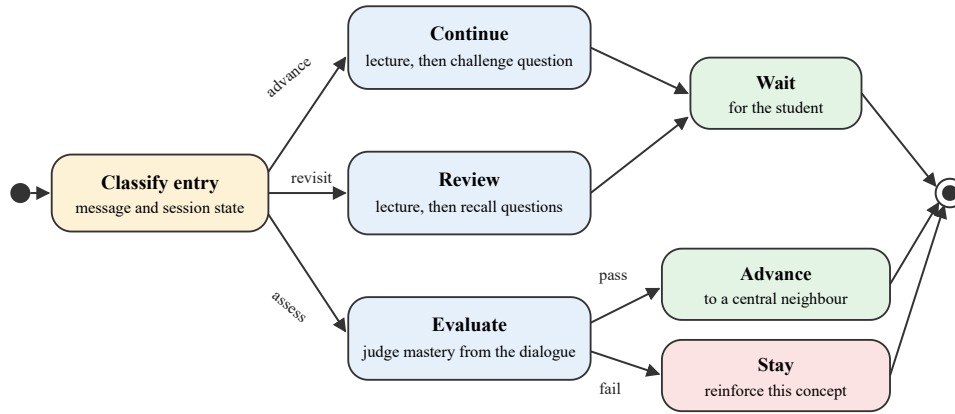


Figure 4.10: Modes of one Teach turn and their outcomes.

Lecture content is collected without a model call: the current concept is resolved to its source page, and when it has none the flow falls back to the Retrieve workflow. The lecture is calibrated by the recorded mastery, from definitions and examples to application problems, and ends with a challenge question (*continue*) or recall questions (*review*). Assessment is tool-free and judges understanding from the conversation. A pass auto-applies an advance to a high-centrality neighbouring concept ranked by  $C_{deg}$ ; a fail keeps the current position.

#### 4.4.2.3 Workflow: Roadmap (Heuristic Graph)

The Roadmap workflow answers what the student should learn next, and in what order. All computations use only the semantic edges  $E_{sem}$  (Section 4.2.1), so they reflect prerequisite dependency rather than rhetorical attachment.

**Backbone.** Inspired by [4], central concepts are ranked by semantic out-degree, a simplified centrality that the graph database computes natively:

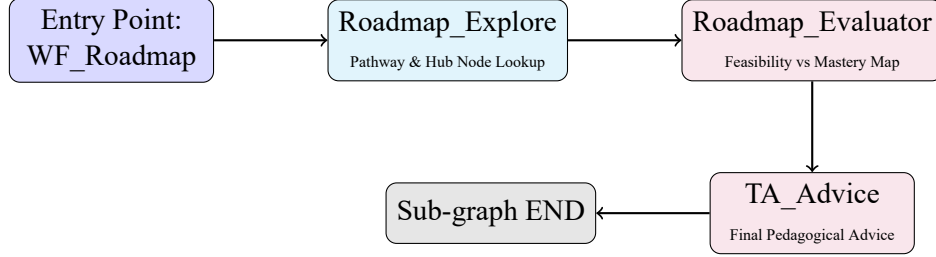
$$C_{deg}(v) = |\{w \in V \mid (v, w) \in E_{sem}\}| \quad (4.8)$$

High- $C_{deg}$  nodes form the course backbone; Teach reuses the same ranking to choose the next concept.

**Knowledge gap.** Mastery  $\mu(v) \in [0, 1]$  is the recorded score of an explored concept and 0 otherwise. For a backbone node  $b$  with immediate prerequisites  $P(b)$  and mastery threshold  $\tau$ , the gap is

$$\Delta(b) = \{u \in P(b) \mid \mu(u) < \tau\} \quad (4.9)$$

and a non-empty gap must be closed before  $b$ .



**Figure 4.11:** Roadmap sub-graph.

**Path.** From the current position  $v_{\text{cur}}$ , the shortest prerequisite path to each backbone hub is computed. The paths are merged, topologically sorted and cut into segments  $W_i$ , and only the top- $k$  nodes by  $C_{\text{deg}}$  of each segment are kept:

$$P_{\text{roadmap}} = \text{top-k}(W_1) \oplus \dots \oplus \text{top-k}(W_m) \quad (4.10)$$

so prerequisites always precede the concepts that depend on them.

**Critique and consent.** An evaluator checks each backbone node for unresolved gaps and the sequence for cognitive load, and an advisory agent turns the critique into guidance. The resulting Learning Proposal changes the session state only after the student confirms it, unlike the auto-applied advance of Teach.

## 4.5 Self-Evaluation Methodology

### 4.5.1 Benchmark Selection

The system is evaluated on MuSiQue [33], a multi-hop question answering benchmark (released under CC BY 4.0) constructed by composing single-hop questions so that each question requires two to four connected reasoning steps. MuSiQue was chosen over simpler multi-hop datasets for two properties. First, its construction deliberately suppresses shortcuts: questions cannot be answered by matching a single paragraph, so retrieving only the most similar passage is insufficient. Second, every question ships with distractor paragraphs alongside the supporting ones, which directly tests whether the retriever ranks gold evidence above plausible but irrelevant text. The scope of this benchmark is therefore limited: MuSiQue measures corpus coverage, retrieval ranking, multi-hop evidence assembly, and latency.

### 4.5.2 Metrics

Metrics fall into two groups: answer metrics check what the system says, and evidence metrics check what it read. Both are computed deterministically from stored traces.

**Answer metrics.** Answers are normalized (lower-casing, removal of punctuation and articles, whitespace collapsing) and compared with the gold answer and its aliases, keeping the best score. Exact Match is

$$EM = \begin{cases} 1, & \hat{a} = a \\ 0, & \text{otherwise} \end{cases} \quad (4.11)$$

and is strict: a correct answer with one extra word scores zero. Token F1 treats the prediction  $\hat{a}$  and gold answer  $a$  as multisets of tokens:

$$P = \frac{|\hat{a} \cap a|}{|\hat{a}|}, \quad R = \frac{|\hat{a} \cap a|}{|a|}, \quad \text{Ans F}_1 = \frac{2PR}{P+R} \quad (4.12)$$

Its precision penalizes every extra or fabricated token, and its recall gives partial credit where EM gives none.

**Evidence metrics.** Let  $R_{ctx}$  be the set of paragraph identifiers placed in the final answer context,  $R_{sup}$  the set the reasoning chain cites as support, and  $R_{gold}$  the gold supporting paragraphs:

$$CP = \frac{|R_{ctx} \cap R_{gold}|}{|R_{ctx}|}, \quad CR = \frac{|R_{ctx} \cap R_{gold}|}{|R_{gold}|} \quad (4.13)$$

$$P_s = \frac{|R_{sup} \cap R_{gold}|}{|R_{sup}|}, \quad R_s = \frac{|R_{sup} \cap R_{gold}|}{|R_{gold}|}, \quad \text{Sup F}_1 = \frac{2P_sR_s}{P_s+R_s} \quad (4.14)$$

Context precision (CP) measures how much of the context is noise, and context recall (CR) whether every hop of the question was found at all; a multi-hop answer is impossible when CR misses a hop. Support F1 scores the evidence the answer actually relies on, so a high answer F1 with a low support F1 flags an answer that is right for the wrong reasons.

Latency is recorded end to end per workflow with a monotonic clock. Every run stores the code revision, model configuration, and policy digest, and two results are compared only when those match.

### 4.5.3 Results

Table 4.3 compares two retrieval presets of the system with two reference groups. The first is recent retrieval-augmented systems, each cited to its original paper, all read by the same general-purpose model (Llama-3.3-70B-Instruct); their EM and F1 on a 1,000-question MuSiQue sample come from the shared evaluation in the HippoRAG 2 study [10]. The second is readers trained on MuSiQue from the public MuSiQue-Ans leaderboard [2], which date from 2023 and report only answer and support F1. Plain RAG uses vector search only; BookRAG adds the textbook hierarchy and its passages. Cells a source does not report are empty. The comparison is indicative, not like-for-like: the sources differ in question sample, corpus, and reader model, while the presets here run on a 500-question STEM subset.

| System                     | EM (%) | Ans F1 (%) | CP   | CR   | Sup F1 (%) |
|----------------------------|--------|------------|------|------|------------|
| Ours: BookRAG              | 25.03  | 60.45      | 0.57 | 0.81 | 81.2       |
| Ours: RAG                  | 20.07  | 47.13      | 0.33 | 0.61 | 56.4       |
| HippoRAG 2 [10]            | 37.2   | 48.6       | —    | —    | —          |
| NV-Embed-v2 (7B) [15]      | 34.7   | 45.7       | —    | —    | —          |
| GritLM-7B [25]             | 33.6   | 44.8       | —    | —    | —          |
| GTE-Qwen2-7B-Instruct [17] | 30.6   | 40.9       | —    | —    | —          |
| SeqGraph [29]              | 59.2   | 56.9       | —    | —    | 78.4       |
| Select+Answer [33]         | —      | 52.3       | —    | —    | 75.2       |
| Human [33]                 | —      | 78.0       | —    | —    | 93.9       |

**Table 4.3:** Answer and evidence quality on MuSiQue.

Adding the textbook structure improves every metric: answer F1 rises by 13 points, context recall from 0.61 to 0.81, and support F1 from 56.4 to 81.2, so most of the gain comes from finding the right evidence. Plain RAG (47.13 F1) matches strong dense retrievers (NV-Embed-v2 45.7, GritLM 44.8), and BookRAG exceeds HippoRAG 2 in answer F1 (60.45 vs 48.6) but not in EM (25.03 vs 37.2), because the answerer explains rather than returning the bare gold entity: the extra words keep recall but cost precision and exact match. BookRAG’s support F1 of 81.2 is close to readers trained on MuSiQue (75.2 to 78.4), though none of its components was trained on that dataset and those readers select support from a given paragraph set rather than a whole corpus.

**Table 4.4:** Mean workflow latency (ms) by number of hops.

| Preset  | 2 hops | 3 hops | 4 hops  |
|---------|--------|--------|---------|
| BookRAG | 17 128 | 42 367 | 115 413 |
| RAG     | 18 343 | 29 499 | 47 812  |

The quality gain has a cost that grows with depth (Table 4.4). At two hops both presets take about 17–18 seconds; at four hops BookRAG takes almost two minutes against under fifty seconds for RAG, because each extra hop is another sequential delegation round over a larger evidence set.

# Chapter 5

## System design and implementation

This chapter presents the concrete design and implementation of the SmartEdu system, translating the methodology outlined in Chapter 4 into a functioning software implementation. The following sections present the implementation, design paradigms, and structural layout, covering these aspects:

I. System Architecture and Design Patterns: A high-level description of the system’s architecture, components, their responsibilities and interactions. It underlies design choices and principles that ensure modularity, scalability, and maintainability.

II. Core Module: The foundational utility supporting all other modules, including LLM inference engine management, database clients, centralized configuration handling, and shared data schemas.

III. Database Design: An overview of the polyglot persistence DB design, database cross reference and detailed explanation of the ”One-Write” policy solving the constraint.

IV. Student Module: The implementation of authentication, session management, and the Student Tracker façade, which abstracts away persistence details while enforcing strict privacy.

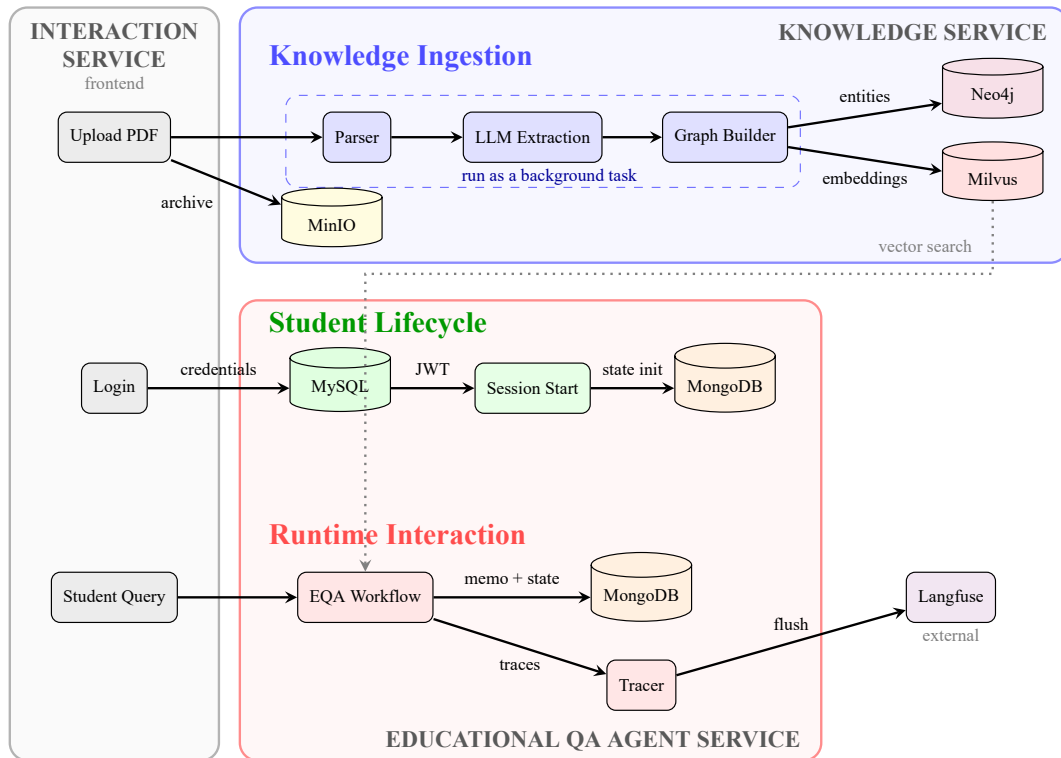
V. Knowledge Module: The offline ingestion pipeline responsible for constructing the educational knowledge graph from raw course materials, covering extraction, transformation, and load operations.

VI. Educational QA Agent (EQA) Module: Explain how LangGraph framework are used in the implementation of agentic workflow.

VII. Frontend Module: The navigation, document reader, session continuity and learning views that expose the three backend modules to students.

## 5.1 System Architecture

### 5.1.1 Overall Architecture and Module Responsibilities



**Figure 5.1:** Services of SmartEdu and the data flows inside them.

SmartEdu follows a microservice architecture of three services. Figure 5.1 draws their boundaries around the data flows, and Figure 5.2 shows the modules inside them.

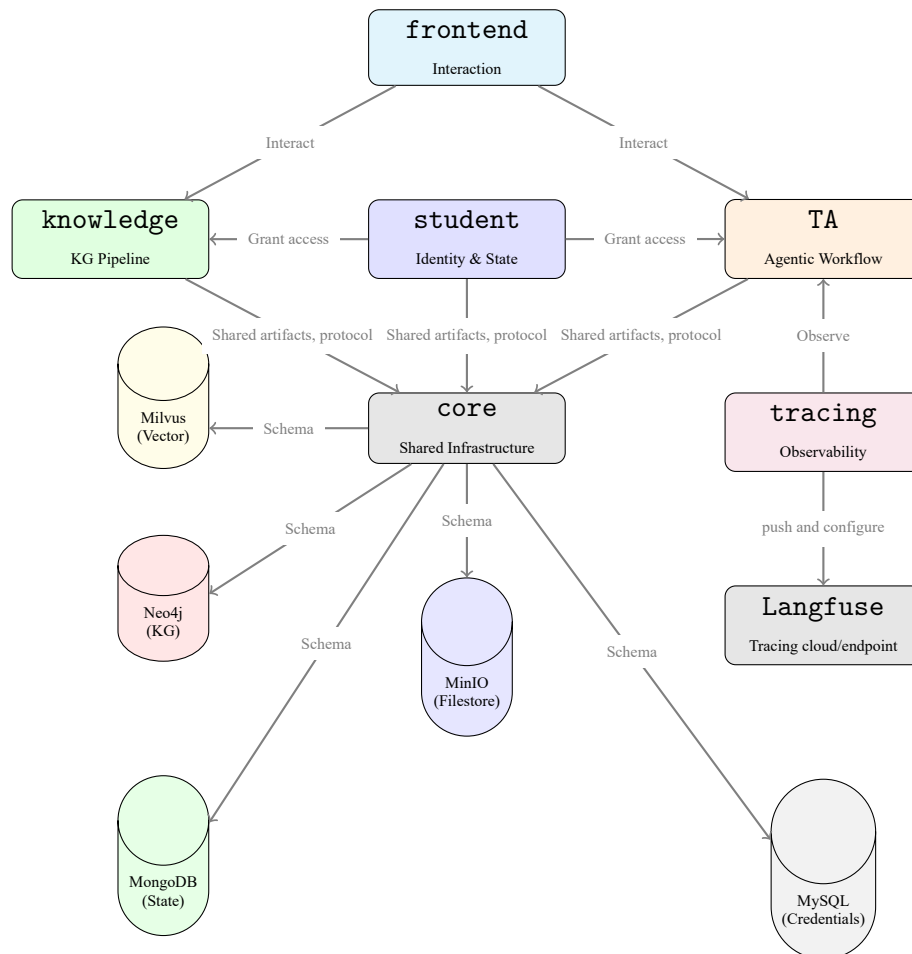
- Interaction service: the frontend, the single entry point where students sign in, chat and read documents, and where administrators upload course material.
- Knowledge service: the ingestion interface and the graph construction pipeline, which runs on the Prefect orchestrator; it owns the knowledge graph, the vectors and the source material.
- Educational QA Agent service: the Student module (accounts, token issuing, sessions, learning state), the agentic workflow and its tracing; it owns credentials, learning state and conversations.

All services share a core library for configuration, storage clients, schemas and a few foundational functions; sharing logic this way is efficient, though it couples the services to one library version. Every request carries the access token issued by the Student module at sign-in, and the Knowledge service checks it before serving course data or accepting an ingestion. The backend services are deployed together in one application process, which avoids network latency between the steps of the multi-step agentic workflow and keeps the deployment footprint small; the storage engines run in their own containers.

**Table 5.1:** *System Module Summary*

| Module    | Primary Responsibility                                                                           | Representative Components                 |
|-----------|--------------------------------------------------------------------------------------------------|-------------------------------------------|
| core      | Shared infrastructure: LLM engine, all DB repository clients, configuration, shared schemas.     | CoreLLMEngine, GraphDB, MilvusDB, MinioDB |
| student   | Authentication, session lifecycle, unified learning-state façade.                                | Student_Tracker, StudentSession, Memo     |
| knowledge | Offline EKG construction pipeline from raw course materials.                                     | KnowledgeModule, CourseIngestionService   |
| TA        | Real-time agentic workflow; generates pedagogically grounded responses.                          | TAModule, SmartEdu, AgentInjector         |
| frontend  | Authenticated study interface, conversation history, source document reader, search and roadmap. | Session manager, chat panel, PDF viewer   |

The core library is the shared foundation used by all three modules, and the Student module depends on nothing else. The Knowledge module uses the Student module for authentication, and the Educational QA Agent module uses the Student module for learning state and authentication, both through direct calls. Concept-name normalization, used by both the Knowledge and the Educational QA Agent modules, lives in the core library. The tracing component is consumed exclusively by the Educational QA Agent. This topology is illustrated in Figure 5.2.



**Figure 5.2:** *Modular Architecture Overview of SmartEdu*

## 5.1.2 Technical Setup: Docker Infrastructure

The system's five database engines and their supporting services are deployed as Docker containers (Section 2.1.7), orchestrated through a Docker Compose configuration. Infrastructure includes:

- **neo4j-core**: Knowledge graph database with APOC and GDS plugins
- **mongo-core**: Student state, chat memo, and session context persistence
- **milvus-standalone**: Vector similarity search engine for concept embeddings
- **minio-core**: S3-compatible object storage for PDF materials and text chunks
- **mysql-core**: Student credentials and account metadata
- **etcd**: Metadata store required by Milvus for distributed coordination
- **attu**: Web-based GUI for Milvus collection inspection and debugging
- **dbgate**: Unified database administration interface for MongoDB and MySQL
- **prefect-server**: Prefect orchestrator recording every ingestion run; its three process work pools, for document parsing, LLM extraction and speech recognition, are served by worker processes on the host

Figure 5.3 shows how the services and the storage engines are deployed.

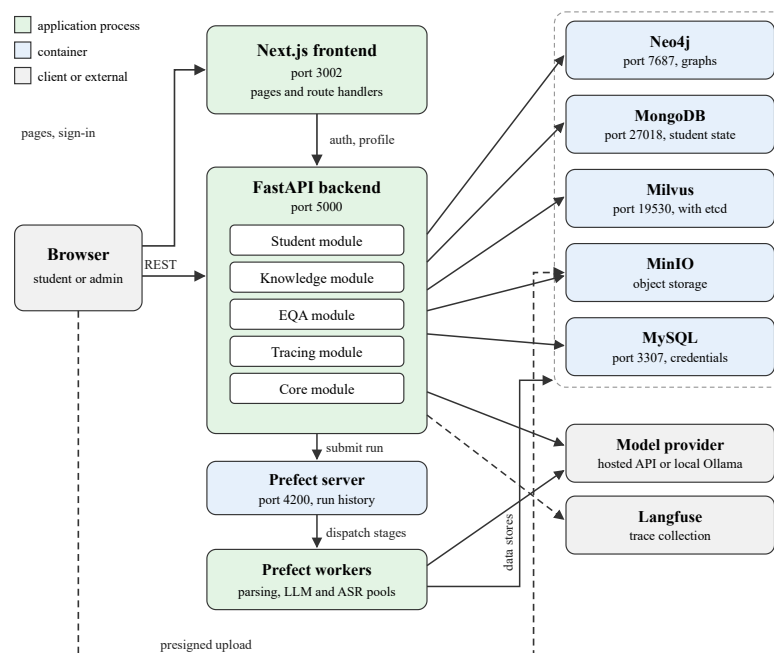
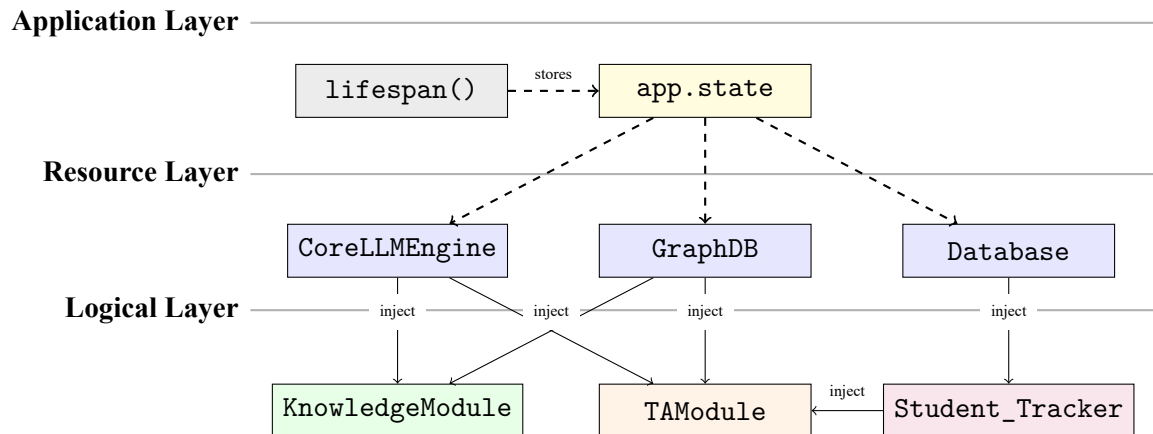


Figure 5.3: Deployment view of SmartEdu.

## 5.1.3 Design Paradigm

The patterns below apply inside each service. In the current co-located deployment one instance serves all backend services; a service deployed on its own holds its own instance.

**Singleton** Long-lived resources such as LLM model instances and database connection pools are expensive to create. Constructing them per request would add latency and exhaust memory, so they need to be managed centrally instead of being rebuilt again and again across the code base.



**Figure 5.4:** Ownership graph in lifespan.

Figure 5.4 shows how the application is constructed and how the resources and logic modules are then initialized. The app lifecycle and its modules are managed through FastAPI's lifespan context manager ( `core/api/life_span.py` ), which implements the Singleton pattern at the application scope. During startup, lifespan performs sequentially these actions:

1. Configuration import: all configuration (database URIs, model names, API keys) is imported and centralized at startup, so that the entire application operates on one configuration for its whole lifetime.
2. Resource initialization: resource managers receive the configuration and construct themselves with the methods needed to create and manage resource instances such as models and connection pools.
3. The lifespan allocates these managers on the application state, global and unique across the whole application. Resources are therefore easy to access and, whatever logic runs below, cannot be duplicated: every reference points to the global instance, not a copy.
4. On shutdown, it performs explicit disconnection and releases the resources. Since all resource managers are singletons owned by the lifespan, no stray instances or connections are left when the application stops.

**Dependency Injection** To complement the Singleton pattern, the system adds a dedicated Dependency Injection (DI) layer in `core/dependencies.py`. This file defines thin provider functions — one per injectable resource — that each return the corresponding singleton from `app.state`. Route handlers declare their dependencies as typed parameters annotated with `Depends()`, and FastAPI's IoC container resolves and injects the correct costly instance at request time.

This indirection yields 4 engineering benefits:

- Decoupling of business logic: Route handlers are pure functions of their typed inputs, completely unaware of how their dependencies are constructed.
- Independence across all modules: Handlers receive resources without importing them, so downstream modules cannot create duplicates, which reinforces the Singleton pattern and avoids circular dependencies between shared components. Each part can be refactored or debugged in isolation.

- Flexible implementations: Replacing even the most sensitive backbone infrastructure requires modifying only the provider function, and changes are effective across all systems.
- Testability: The test suite overrides any provider via `app.dependency_overrides`, enabling full unit testing without live database or LLM connections.

**MVC-Aligned Module Structure** The system is designed to support complex stateful workflows while remaining strictly modular. Each domain module follows an MVC-aligned internal layout that enforces a clean separation of concerns:

- Model (Schema): Pydantic models defined in `core/schema/` act as the system-wide data contract. All inter-module data is typed and validated through shared classes. This shared schema layer is what makes each module independently works yet produce consistent, reusable results by other modules.
- Controller (API Router): Each module exposes its functionality through a dedicated FastAPI router (e.g., `TA/api/route.py`). The router handles only HTTP-level concerns: parsing requests, invoking the service layer, and serializing responses. It contains no business logic.
- Service (Domain Module): Domain classes (`TAModule`, `KnowledgeModule`) encapsulate all business logic. They receive infrastructure dependencies exclusively via constructor injection and communicate with the controller only through protocols defined in shared Pydantic schemas.

This layering keeps the service boundaries narrow: modules exchange only shared schemas from `core` and a few entry points, namely the Student Tracker, the memo and the authentication of the Student module, and the concept-name normalization of the Knowledge module. These entry points are the contracts between the services, with the Pydantic schemas as their API contracts, and the domain logic behind them does not depend on where the other services run.

## 5.2 Core Module

The core package is the dependency foundation of the entire system. It owns no domain business logic; instead, it supplies the primitive, reusable services and shared data contracts upon which all domain modules are built. As established in Section 5.1.3, all core resources are constructed once during the lifespan phase and injected by reference. Injecting by reference ensures that all modules share the same instance of each resource, which is crucial for expensive resources such as LLM models and database connection pools.

### 5.2.1 LLM Engine

`CoreLLMEngine` is the central gateway for all LLM interactions. It implements the Multiton pattern — a generalization of Singleton where a fixed, named pool of instances is managed. Internally it maintains a named registry of model connectors keyed by a profile name. When an agent issues a model retrieval request, the engine performs a cache lookup: if the profile is absent, it reads the corresponding `LLMProfile` from `core/llm/config.py`, constructs a model connector object, stores it in the registry, and returns it; subsequent calls for the same profile return the cached object directly without reconstruction.

**Lazy Instantiation and GPU Memory Lifecycle** An important property of this design is its zero-cost initialization: the engine’s internal registry is empty at application startup, and a model connector is instantiated only on the first retrieval request for a given profile, loading only that specific model. GPU memory is therefore occupied exclusively by models that have been actively requested.

Memory retention after a request is governed by an idle retention timeout configured per profile and forwarded directly to the backend runtime, which holds the model’s weights in GPU memory for that duration after the last inference call and then evicts them. The registered connector object persists in the cache, so the next invocation transparently triggers an automatic weight reload; profiles without an explicit timeout inherit the runtime’s global default retention policy. The pairing of lazy instantiation with per-profile retention windows creates a tiered memory strategy: high-frequency agents are kept warm for the full duration of a tutoring session, moderate-frequency agents are evicted after a shorter idle period, and lightweight classification workers are evicted aggressively since their small size makes cold-start reloads negligibly cheap.

**Profile Configuration** Each LLMProfile dataclass encapsulates the full operational parameter set for one agent role. Table 5.2 lists the six profiles defined in the current deployment, along with the rationale for each configuration choice. All profiles are served through Ollama (Section 2.1.5). When the monthly quota of the Ollama cloud models is exhausted, the reasoning-heavy profiles are temporarily pointed to an equivalent open-weights model on an OpenAI-compatible provider (OpenRouter); only the profile configuration changes, and the agents are unaffected.

**Table 5.2:** LLM Profile configurations and their engineering rationale.

| Profile   | Model              | Temp | Context (tok) | GPU Retention | Role and Rationale                                                                                                                                                       |
|-----------|--------------------|------|---------------|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| graph     | gpt-oss:120b-cloud | 0.2  | 8 192         | —             | Knowledge graph extraction. Large context window accommodates full slide batches; low temperature enforces deterministic structured JSON output.                         |
| ta        | gpt-oss:120b-cloud | 0.67 | 4 096         | 1 h           | Conversational tutor. Higher temperature yields more natural dialogue; kept hot in GPU memory for the full duration of a tutoring session.                               |
| rag       | qwen3:8b           | 0.0  | 2 048         | 30 min        | Retrieval reasoning agent. Temperature zero ensures deterministic tool-calling decisions; a fixed output token cap prevents runaway generation during structured output. |
| evaluator | qwen3:8b           | 0.3  | 2 048         | —             | Assessment nodes. Low temperature maintains a consistent scoring rubric across evaluations.                                                                              |
| generator | gpt-oss:120b-cloud | 0.4  | 4 096         | —             | Lecture generation. Moderate temperature balances creative expression with factual grounding.                                                                            |
| worker    | gemma3:1b          | 0.0  | 768           | 5 min         | Topic naming during ingestion. Smallest available model; minimal context and aggressive eviction keep resource cost negligible.                                          |

## 5.2.2 Configuration Management

All system configuration is expressed as Python `@dataclass` objects in `core/config.py`. Each sub-system has a dedicated configuration class — Neo for the knowledge graph database, `Mil_conf` for the vector store, and so on. Environment variables are read exactly once at module import time via `python-dotenv`, which loads values from a centralized `.env` file. The values are then bound to these objects, which keeps the configuration consistent across the application. All dataclass fields default to sensible values if the corresponding variable is absent.

**Table 5.3:** *Configuration dataclasses.*

| Infrastructure                           | Key Parameters                                                              | Purpose                                                                                                          |
|------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <i>Infrastructure Connection Configs</i> |                                                                             |                                                                                                                  |
| Neo4j                                    | Connection URI, credentials, target database name                           | Graph database connection for the knowledge graph.                                                               |
| Milvus                                   | Service URI, collection name, vector dimensionality (768), retry budget     | Vector store connection for embedding and similarity search.                                                     |
| HuggingFace                              | Model identifier, vector dimensionality (768), maximum token length         | Embedding model parameters for scientific text vectorisation (default: SciBERT).                                 |
| MinIO                                    | Service endpoint, access credentials, TLS flag                              | Object storage connection for raw PDF course materials and extracted text chunks.                                |
| MongoDB                                  | Host address, credentials, target database name                             | Document store connection for student state, chat history, and session persistence.                              |
| <i>Module Logic Configs</i>              |                                                                             |                                                                                                                  |
| Knowledge pipeline                       | Source data path, pages per textbook batch (10), pages per slide batch (15) | Controls chunking and batch sizes tuning the trade-off between LLM context utilization and extraction precision. |
| LLM Engine                               | Extraction LLM profile name                                                 | Initiate LLM (see Table 5.2) handles knowledge graph extraction tasks.                                           |
| EQA pipeline                             | Agent name, system prompt, and output schema                                | Declares and register the set of agents instantiated at startup.                                                 |
| Langfuse                                 | Enable flag, secret and public API keys, host URL                           | Optional observability configuration.                                                                            |

## 5.3 Database Design

The system employs the polyglot persistence strategy of Section 2.1.6, distributing data across five specialised storage engines. Each engine is selected for the data modality it serves, so each store can be managed on its own.

### 5.3.1 Polyglot Persistence Design

**Table 5.4:** *Polyglot persistence strategy overview.*

| Engine  | Entities of the system                                                                                                        | Purpose                                                                                             |
|---------|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Neo4j   | Concept, topic, community and rhetorical nodes; textbook section and passage nodes; prerequisite, hierarchy and anchor edges. | Multi-hop traversals through rerequisite paths, backbone hubs and document-graph hops               |
| Milvus  | Embeddings of concept content, keyed by concept identifier.                                                                   | Concept retrieval over embeddings, and the shared identifier leads straight back to the graph node. |
| MongoDB | Per-student document: learning state and mastery, chat memo, session context and cached tool results.                         | Non rigid storage for semantically rich agentic outputs.                                            |
| MinIO   | Original slide and textbook PDFs, and the extracted text chunks.                                                              | High Volume, Velocity and Variety for Heavy Resources such as PDF and Videos                        |
| MySQL   | Student credentials and account meta-data (username, name, email, administrator flag).                                        | Structured ACID DB for sensitive data.                                                              |

**Neo4j: Knowledge Graph** The database stores the Educational Knowledge Graph constructed by the Knowledge Module, following the property graph definition of Section 4.2.1. Each entity node carries the standardised properties of Table 5.5, which support both structural navigation and semantic retrieval. Four directed edge types connect the concept layer: *PART OF* (hierarchical containment), *RELATED TO* (semantic association), *PREREQUISITE OF* (learning dependency), and *CONTENT* (rhetorical elaboration from a concept to its explanatory nodes). The document layer adds Section and Passage nodes, joined by *CONTAINS* and *HAS PASSAGE*, and *ANCHORED IN* links a concept to the passages that explain it. Videos add Video and Segment nodes, joined by *HAS SEGMENT*, and *ANCHORED IN* also links a concept to the video segments that discuss it. The database enforces uniqueness constraints on both node identifiers and names, maintains a full-text search index across both fields for keyword-based retrieval, and keeps full-text and vector indexes over passages for the textbook search. A dedicated index on the rhetorical role property enables the teaching workflow to perform role-filtered queries (e.g., retrieving only Definition-type nodes for explanatory responses).

| Property         | Description                                                                                       |
|------------------|---------------------------------------------------------------------------------------------------|
| Name             | Canonical concept label, enforced unique across the database                                      |
| Identifier       | Machine-generated unique key for programmatic reference                                           |
| Content          | Full textual description or definition of the concept                                             |
| Node Type        | Ontological category: Community, Topic, Concept, or Rhetorical                                    |
| Rhetorical Role  | Instructional function (Definition, Theorem, Application), assigned only to Rhetorical-type nodes |
| Cluster          | Community identifier assigned during graph construction                                           |
| Source Reference | Serialised pointer to the original material location in object storage                            |

**Table 5.5:** *Entity node properties in Neo4j.*

**Milvus: Vector Embedding Index** The Milvus vector database stores 768-dimensional SciBERT embeddings generated during the knowledge ingestion pipeline. Each record in the collection consists of three fields described in Table 5.6.

**Table 5.6:** *Milvus collection schema.*

| Field     | Data Type          | Constraint  | Description                                                                  |
|-----------|--------------------|-------------|------------------------------------------------------------------------------|
| id        | VARCHAR            | Primary Key | Unique entity identifier, shared with Neo4j for cross-database reference     |
| text      | VARCHAR            | —           | Concatenation of concept name and content, used as the embedding source text |
| embedding | FLOAT_VECTOR (768) | Indexed     | SciBERT-generated dense vector for semantic similarity search                |

The collection employs a Hierarchical Navigable Small World (HNSW) index optimised for the L2 (Euclidean) distance metric. Table 5.7 details the index and search parameters.

**Table 5.7:** *HNSW index configuration.*

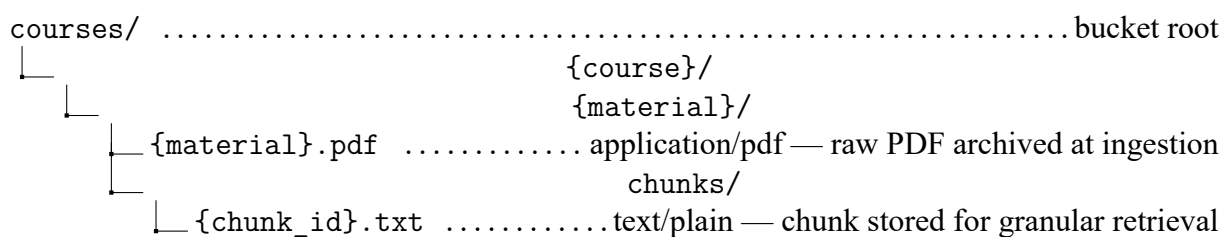
| Parameter                | Value          | Purpose                                                                                 |
|--------------------------|----------------|-----------------------------------------------------------------------------------------|
| M (bi-directional links) | 8              | Maximum number of neighbours per graph node during index construction                   |
| efConstruction           | 64             | Search depth during index build; higher values improve recall at the cost of build time |
| ef (query-time)          | 64             | Search depth during query; balances retrieval accuracy against latency                  |
| Metric Type              | L2 (Euclidean) | Distance metric for approximate nearest-neighbour search                                |

**MongoDB: Student State and Session Context** MongoDB serves as the primary persistence layer for all student-related runtime data. Each student is represented as a single document keyed by their unique identifier, containing three nested structures: the learning state, the chat memo, and the session context. Table 5.8 provides the detailed field-level schema of the student document.

**Table 5.8:** MongoDB student document field schema.

| Field Path                                                | Type          | Description                                                |
|-----------------------------------------------------------|---------------|------------------------------------------------------------|
| <i>state (Learning State)</i>                             |               |                                                            |
| finished_communities                                      | Array         | List of completed knowledge graph communities              |
| current_pos                                               | Object / null | Currently active concept node in the learning path         |
| active_resource                                           | String / null | Currently loaded material reference                        |
| mastery_map                                               | Object        | Key-value map of concept name to mastery score             |
| previous_nodes                                            | Array         | Ordered history of recently visited concept nodes          |
| upcoming_nodes                                            | Array         | Queue of next recommended concepts from roadmap            |
| summary                                                   | String        | Compressed natural-language summary of learning progress   |
| pending_proposal                                          | Object / null | Unconfirmed learning path update awaiting student approval |
| <i>memo (Chat History) — nested by session_id</i>         |               |                                                            |
| role                                                      | String        | Speaker identifier: “student” or “TA”                      |
| heading                                                   | String        | Topic heading for memo display                             |
| message                                                   | String        | Full message content                                       |
| timestamp                                                 | DateTime      | Server-side UTC timestamp                                  |
| <i>session_context — nested by session_id and chat_id</i> |               |                                                            |
| tool_results                                              | Array         | Cached tool execution outputs for within-request recall    |
| thoughts                                                  | Array         | Agent reasoning traces for debugging and observability     |

**MinIO: Object Storage** MinIO provides S3-compatible object storage for all binary educational assets. All objects reside in a single bucket named “courses”. The storage hierarchy follows a deterministic three-level path convention detailed in Figure 5.5, enabling the teaching workflow to reconstruct any object path programmatically from the concept’s source reference property.



**Figure 5.5:** MinIO bucket directory structure.

The deterministic naming convention ensures that any component in the system can reconstruct an object path from two known values: the course name and the material name. The EQA workflow exploits this property to serve PDF page references directly to the student frontend without querying a separate metadata index.

**MySQL: Authentication Store** Student authentication is managed through a MySQL 8 database running as a container. Table 5.9 describes the schema of the single “students” table.

**Table 5.9:** *MySQL students table schema.*

| Column     | Type         | Constraint  | Description                                |
|------------|--------------|-------------|--------------------------------------------|
| id         | VARCHAR(255) | PRIMARY KEY | Unique student identifier                  |
| username   | VARCHAR(255) | UNIQUE      | Login credential, enforced unique          |
| name       | VARCHAR(255) | —           | Student display name                       |
| email      | VARCHAR(255) | —           | Contact email address                      |
| password   | VARCHAR(255) | —           | SHA-256 hashed password                    |
| is_admin   | TINYINT      | DEFAULT 0   | Administrator flag; gates course ingestion |
| created_at | TIMESTAMP    | DEFAULT NOW | Account creation timestamp                 |

A relational engine with a unique constraint on the username gives ACID guarantees for credential operations, and keeps credentials apart from the document store that holds learning state. Password hashing uses SHA-256 at the application layer before storage.

### 5.3.2 Cross-Database Reference and Consistency

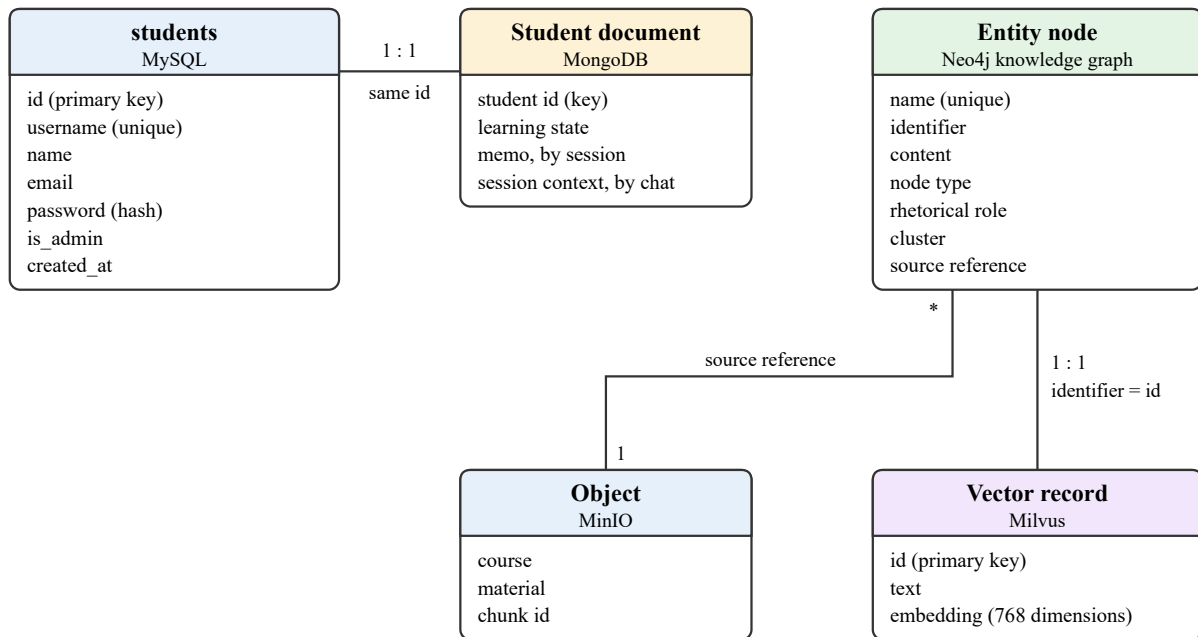
Several modules access several databases, and data from different sources is aggregated and updated at run time. Two architectural rules govern how the five storage engines coexist without data duplication or race conditions.

**Unique Key Cross-Database Reference** Entity identifiers serve as the primary cross-reference between Neo4j and Milvus: the same string identifier stored as a node property in Neo4j functions as the primary key in the Milvus collection, so the retrieval agents can move directly between graph traversal results and vector similarity searches. When an agent performs a semantic search in Milvus, the returned identifiers are directly usable as lookup keys in Neo4j Cypher queries, eliminating the need for an intermediate translation layer.

Student identifiers bridge MySQL (authentication) and MongoDB (learning state). A single login event propagates the authenticated identity consistently across both stores: MySQL validates the credential and returns the student identifier, which MongoDB uses as its document primary key. This shared key lets the Student Tracker read the learning state from MongoDB without any synchronisation protocol beyond the identifier itself.

Figure 5.6 summarizes these keys. Two of them carry the whole design: the student identifier joins the credential table and the learning-state document, and the concept identifier joins the knowledge graph, the vector index and the source material in object storage.

**One-Write Policy** With consistent cross-database references, modules can easily access many databases, a convenience that brings a risk of race conditions. To prevent this, a "One-Write Policy" is applied: each database collection or table is assigned exactly one write-owning module, with strictly governed method. All other modules are restricted to read-only access. Table 5.10 enumerates the write ownership assignments.



**Figure 5.6:** Data model across the five storage engines.

**Table 5.10:** Write ownership per database.

| Database          | Collection               | Writer           | Consumers  |
|-------------------|--------------------------|------------------|------------|
| Neo4j (Knowledge) | Entity nodes, edges      | Knowledge Module | EQA Module |
| Milvus            | Concept embeddings       | Knowledge Module | EQA Module |
| MongoDB           | students collection      | Student Module   | EQA Module |
| MongoDB           | learning_logs collection | Student Module   | EQA Module |
| MinIO             | courses bucket           | Knowledge Module | EQA Module |
| MySQL             | students table           | Student Module   | —          |

This strict ownership eliminates the possibility of two modules concurrently writing to the same field. Within a single write-owning module, every write uses MongoDB’s atomic field-level update operators (\$set and \$push) rather than full document replacement, so concurrent sessions of the same student update disjoint document paths (e.g., one session updating the mastery map while another appends to the chat memo) without overwriting each other’s changes.

## 5.4 Student Module

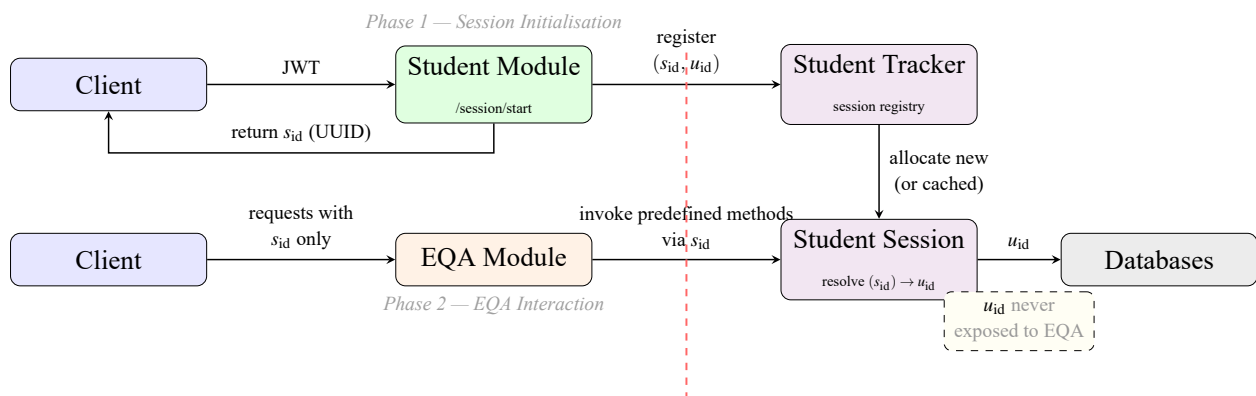
**Authentication: OAuth2 + JWT** The student module implements the OAuth2 password flow of Section 2.1.8, using PyJWT for JWT minting. The /student/login endpoint validates credentials against SHA-256 hashed passwords in MySQL and returns a short-lived access JWT (sub = student\_id) together with a longer-lived refresh token, which the frontend proxy keeps in an HTTP-only cookie. All subsequent requests attach this token as a Bearer credential; get\_current\_student() (a FastAPI dependency) decodes and validates it on each call.

**Session Lifecycle and the Identity Firewall** The purpose of this module is to open access to sensitive components and resources, and to enable the logic modules, only after login and in a

controlled way. Access must also stay abstracted: the application layer cannot see the underlying identity and can only act through monitored request protocols.

These principles are put into practice through the session lifecycle in Figure 5.7. The process is as follows:

1. After login, the student calls POST /student/session/start with their JWT. The server generates a random UUID ( session\_id) and registers it in Student\_Tracker.
2. All subsequent EQA interactions supply only session\_id. The EQA Module never receives student\_id.
3. Inside Student\_Tracker.\_resolve(), the private mapping session\_id → student\_id translates transparently before any database call.



**Figure 5.7:** Login protocol with Identity Firewall (represented by the red line).

This pattern is analogous to a hotel key-card system: the key (session\_id) grants access to resources, but revealing it does not expose the guest’s credentials (student\_id). Modules and tools gain abstracted access to student metadata only after login, which keeps the AI and outside parties away from sensitive databases and resources.

The Student Tracker acts as a Singleton manager that provides and caches session resources and cleans up those of unused sessions. In the current implementation, it manages two distinct layers of state:

1. Session-Specific State ( StudentSession): Keyed by session\_id, this layer isolates chat history and session-specific context (like recent PDF pages). This prevents ”Identity Race Conditions” where multiple browser tabs contaminate each other’s chat logs.
2. Shared Learning State: Keyed by student\_id, this layer caches the student’s learning progress (current position, mastery map, etc.) and is shared across all active sessions of the same student.

The tracker writes the shared state through the atomic field updates of the One-Write Policy (Section 5.3), so one session cannot revert progress made in another concurrent session.

## 5.5 Knowledge Module

The Knowledge Module operates as an offline administrative pipeline responsible for transforming raw educational documents into the structured Educational Knowledge Graph consumed by the

Educational QA Agent at runtime. This module is accessible exclusively through dedicated REST endpoints, invoked by administrators upon uploading new course materials, and its pipeline runs as Prefect flows.

**Technical Setup** Table 5.11 summarises the principal libraries employed by the Knowledge Module. All dependencies are resolved through the *uv* package manager, ensuring deterministic, lockfile-driven builds across development and deployment environments.

**Table 5.11:** *Knowledge Module core libraries.*

| Library        | Role in Pipeline                                                                                                                                            |
|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Docling        | PDF parser (Section 2.1.5) that converts each page into structured text blocks with provenance metadata such as page number and bounding region.            |
| spaCy          | Natural language processing library providing lemmatisation, synonym resolution, and stopword filtering for the post-extraction deduplication stage.        |
| Pydantic       | Schema validation framework that enforces strict structural contracts on every LLM extraction output, rejecting malformed responses before graph insertion. |
| SciBERT        | Domain-specific sentence transformer (768 dimensions) trained on scientific corpora, generating embeddings that capture academic semantic similarity.       |
| faster-whisper | Speech recognition (Section 2.1.5) that transcribes lecture videos and audio into timed segments.                                                           |
| Prefect        | Workflow orchestrator (Section 2.1.5) that runs the pipeline as flows and tasks, dispatches the heavy stages to worker processes, and caches their results. |

### 5.5.1 Ingestion Workflow

An administrator starts ingestion from the browser in two steps (Figure 5.8). The backend first hands out one presigned upload address per file, so the browser sends the files straight to object storage and the backend never carries the file bytes. A second request then names the uploaded files; the backend checks that they exist and that the required workers are available, submits a course flow run to the orchestrator, and answers immediately with its run identifier. The run reads the materials, writes the graph and the vectors, and keeps a report in object storage that the browser polls until the run ends.

**Orchestration** The pipeline runs as a hierarchy of Prefect flows. A root course flow runs one subflow per source file in a fixed order: textbooks first, then slides, then the linking of slide concepts to textbook passages, and videos last, since anchoring a video segment searches the concept vectors that the slides have just written. Files of the same type run concurrently. The heavy stages, document parsing, LLM extraction and speech recognition, are deployed as separate flows on three process work pools, so each runs in a worker process of its own and a crash in one of them does not bring down the backend. Their results are persisted under a cache key built from the revision of the source object and the configuration of the stage, such as the model, the prompts and the parsing parameters; when a later step fails, a manual rerun reuses these results instead of parsing, extracting or transcribing the file again. The lighter tasks that publish chunks and write to the stores declare their own retries. The root flow writes the run report when it starts and again when it ends, with the duration of each stage, the outputs of each file and any errors.

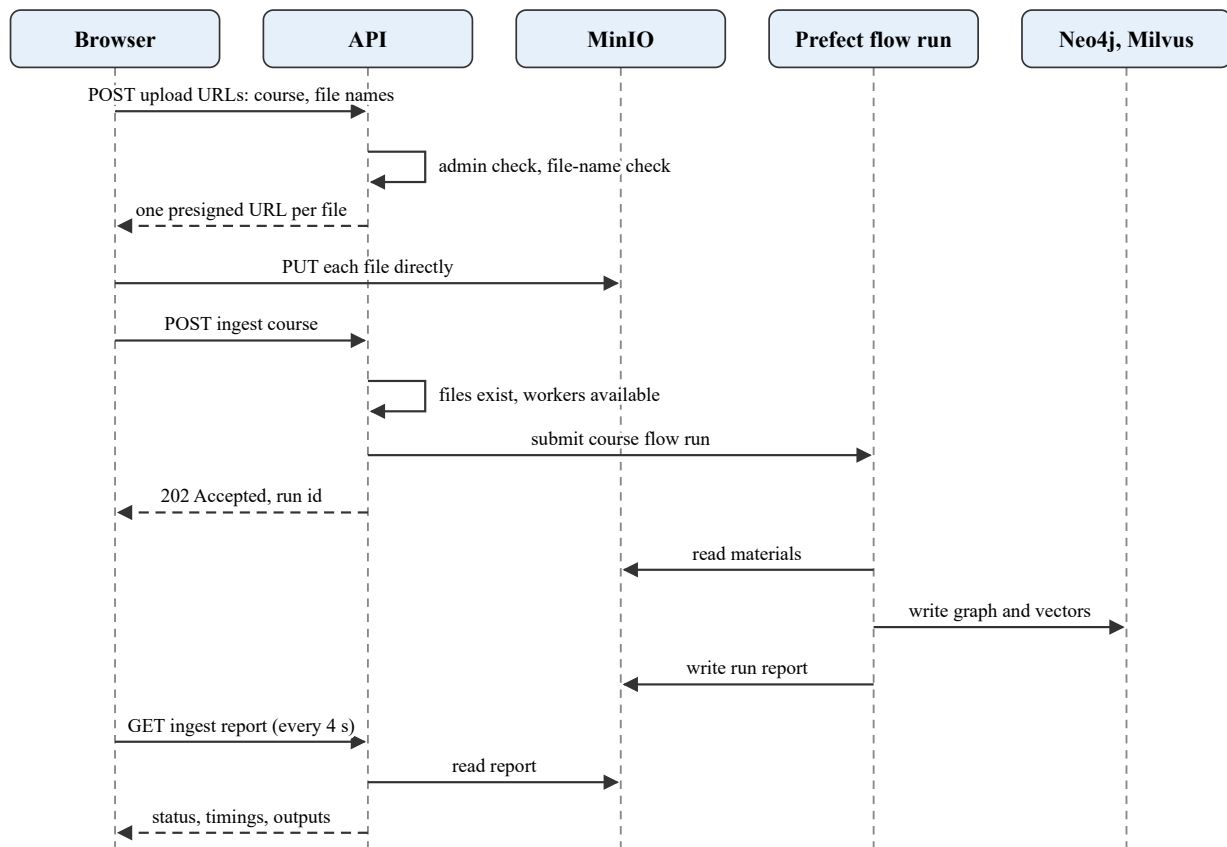


Figure 5.8: Sequence of one course ingestion.

**Material Routes** The pipeline supports three material types. Lecture slides and textbooks share the document parsing entry stage and diverge at knowledge extraction: slides are cut into overlapping windows (Section 4.3.2), while textbooks are split along their own sections (Section 4.3.1). Videos take a separate route: speech recognition transcribes the audio into timed segments, each segment is embedded and matched against the concept vectors of the course, and the video and its segments are written to Neo4j with anchor edges from the matched concepts.

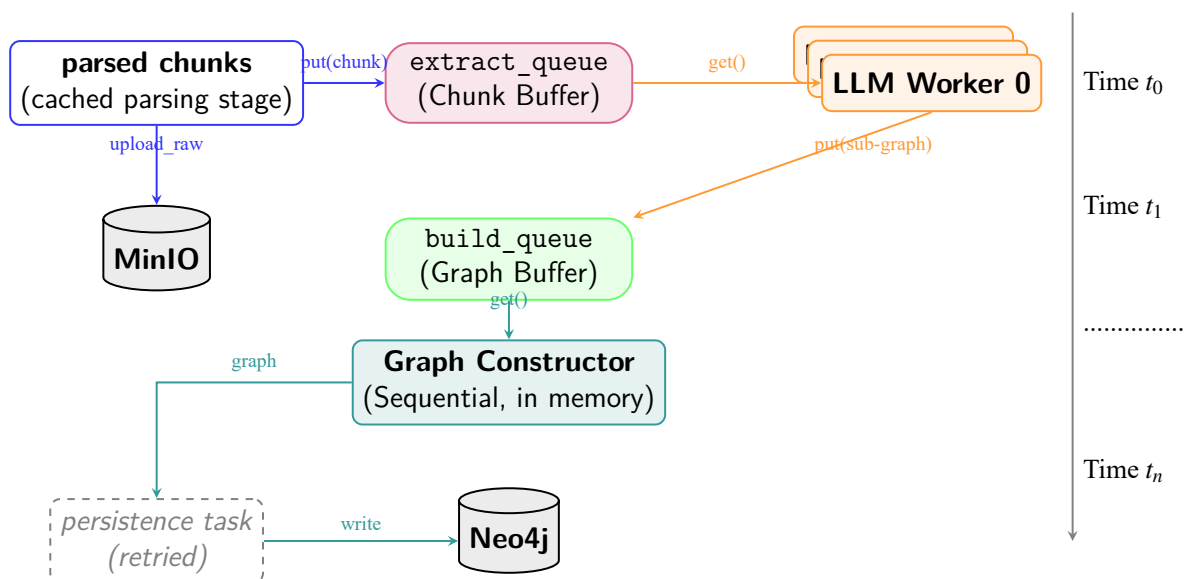


Figure 5.9: Asynchronous ingestion pipeline.

Lecture slides follow the primary extraction path. Because slide extraction involves complex, interdependent LLM operations and strict database insertion rules, it is architected as a fully asyn-

chronous Three-Station Producer-Consumer pipeline interconnected by queues. Neither heavy I/O nor LLM inference blocks the stream, so extraction runs continuously without being bottlenecked by slower downstream operations.

- Station 1 (Producer): The parsing stage converts the deck into chunks once and caches the result. The chunks are uploaded to MinIO and pushed into the Extraction Queue.
- Station 2 (LLM Workers): A pool of concurrent worker coroutines consumes items from the Extraction Queue. These workers operate independently, forwarding chunks to the LLM for the Dual-Phase Extraction (concept mapping and relation discovery). Each asynchronous task yields while it waits for the LLM result, which costs little memory and overlaps the inference waiting times. Once the LLM returns the structural representation, the result is forwarded to the Build Queue.
- Station 3 (Graph Builder): A single sequential consumer takes the extracted nodes and edges from the Build Queue and assembles the Knowledge Graph in memory. Keeping this station single-threaded intentionally prevents race conditions and topological conflicts during the graph assembly phase, while the heavier workload (LLM inference) remains aggressively parallelized upstream. The assembled graph is then written to Neo4j and Milvus by a separate persistence task that is retried on failure.

Backpressure is enforced by a maximum capacity on the Build Queue, so memory usage remains stable even if the LLM workers outpace the graph builder. In the measured run of Section 6.3, three workers extracted a 14-page deck in 33.8 s, about a fifth of its 151.9 s stage. Run one at a time, the same calls would take up to about three times as long; this estimate assumes the three workers stayed busy and was not measured separately.

## 5.6 Educational QA Agent Module

The Educational QA Agent (EQA) Module is the real-time component of the system, responsible for processing student queries and generating pedagogically grounded responses by orchestrating tool-calling agents across a state-driven workflow. Unlike the Knowledge Module which operates offline, the EQA Module handles concurrent student sessions with sub-second latency requirements.

### 5.6.1 Technical Setup

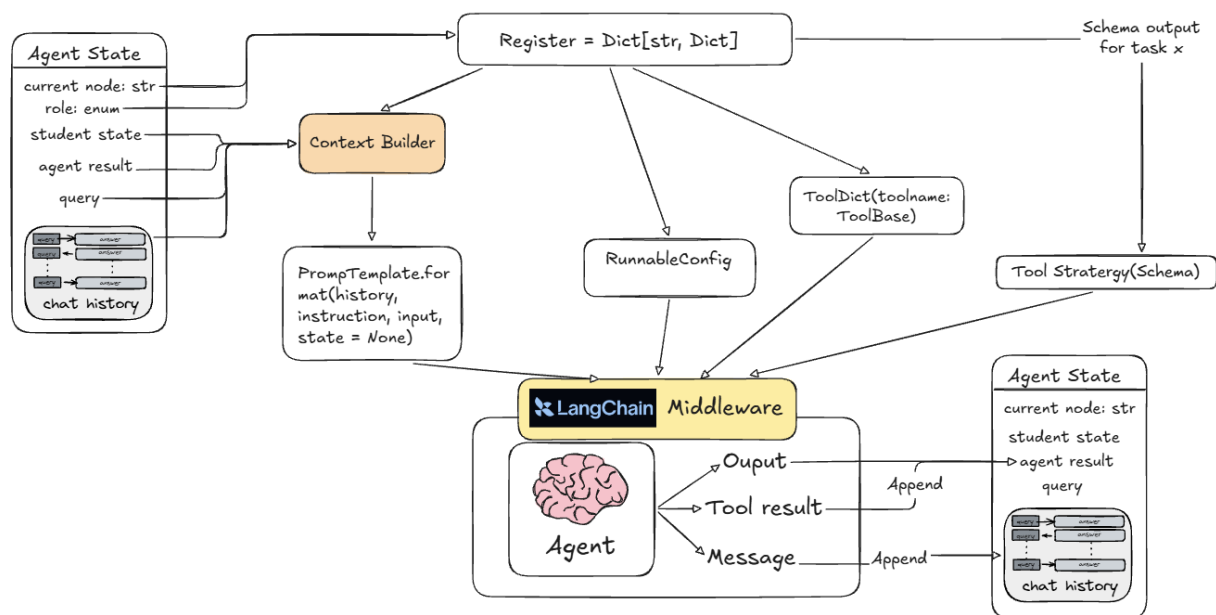
**Table 5.12:** *EQA Module core libraries.*

| Library   | Role in Module                                                                                                                                               |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LangGraph | Workflow graph builder providing typed state containers, conditional routing, and compiled sub-graph composition for deterministic agent orchestration.      |
| LangChain | Agent abstraction layer supplying tool binding, middleware protocol, structured output parsing, and the ReAct execution loop.                                |
| Pydantic  | Structured output schemas ensuring every agent response conforms to a predefined contract before reaching the student.                                       |
| asyncio   | Non-blocking concurrent execution enabling background persistence, parallel database writes, and real-time streaming without blocking the response pipeline. |

In the conventional single-agent architecture of many frameworks, the system prompt, context, and tools are fixed at initialization—forcing the model to simultaneously manage routing, retrieval, evaluation, and generation instructions, which degrades performance as task complexity grows. Langchain Middleware [14], released in December 2025, is one of the latest features providing a composable interceptor that modifies the model’s request before it reaches the language model, enabling a single agent to exhibit distinct behaviours per workflow node. Unlike role-based frameworks where changing an agent’s behaviour requires spawning a new instance entirely, the middleware approach eliminates redundant model connections while preserving per-node specialisation.

### 5.6.2 LangGraph Workflow Design

The EQA Module’s orchestration layer is built as a LangGraph state machine where each node represents either a specialised processing step or a compiled sub-workflow. Every node follows the same operation, shown in Figure 5.10: the shared Agent State enters the node, the node’s name selects its registered resources, a Context Builder assembles only the information the node needs, a single agent runs behind the middleware, and its results are written back into the Agent State for the next node. This exclusion mechanism keeps each prompt limited to the current task and prevents interference from data produced for other nodes.



**Figure 5.10:** Standard operation of a LangGraph Node

**Agent State:** The dynamic data payload and memory shared by all nodes of a workflow.

- **Current Node:** Name of the node being executed, used as the key into the Register.
- **Role:** The intent chosen by the router for the request, which determines the workflow and the instruction applied.
- **Student State:** The learner’s current position in the course, the concepts already studied and upcoming, and the mastery level of each concept.
- **Agent Result:** Non-message results accumulated from previous nodes, such as intermediate reasoning, structured outputs and the buffer of retrieved evidence.

- Query: The input from the user.
- Chat History: The ordered sequence of previous questions and answers of the session.

**Register:** A dictionary created during system start via Schemas, keyed by node name, whose value is an inner dictionary holding the resources of that node. The Register supplies three of them to the middleware to reconstruct agents (described in Paragraph 5.6.1) and one to the Context Builder:

- Tool Dictionary: The tools available to the node, grouped by task and registered once at initialisation.
- Runnable Config: Configuration that changes agent behaviour according to the task, for example a low temperature for rigid retrieval tasks.
- Tool Strategy: The output schema declared for the node's task. It overrides the structured output of the agent at run time, so the final answer is submitted as a schema-shaped tool call after any intermediate tool calls. Nodes that stream plain text to the student declare no schema.
- Instruction: The predefined meta context and task instruction passed to the Context Builder.

**Context Builder (Section 4.4.1.1):** Reads the role, student state, agent result, query and chat history from the Agent State and selects the parts relevant to the node. Context builder constructs, prunes then fills in LangChain prompt template to invoke agent asynchronously.

**Write-back:** The agent produces three kinds of results. The structured output and the tool results are appended to the agent result, where later nodes can read them, and the generated message is appended to the chat history. The updated Agent State then leaves the node, which keeps state and context consistent across the whole workflow while each node still works with its own local context and agentic behaviour.

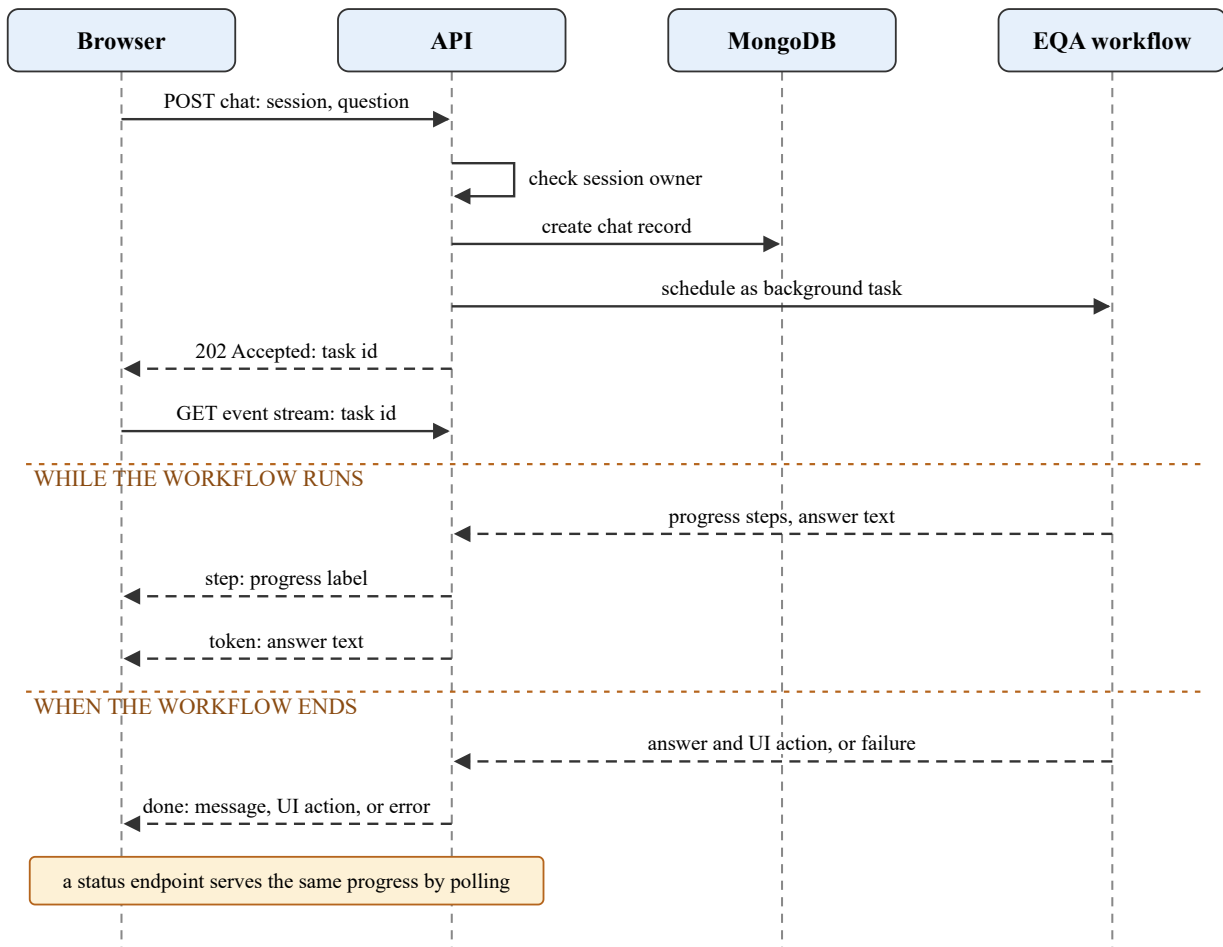
### 5.6.3 Request Lifecycle

A chat request never blocks on the workflow. The backend checks that the session belongs to the caller, stores the question, schedules the workflow as a background task and returns a task identifier at once. The browser then opens an event stream for that task: progress labels arrive while the workflow runs, followed by the answer text and a final event that carries the message and an optional interface action, or an error (Figure 5.11). A status endpoint serves the same progress by polling for clients that cannot hold a stream.

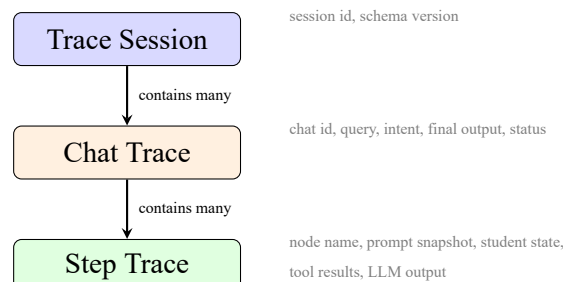
### 5.6.4 Tracing and Observability

An agentic system is an unstable, black-box pipeline. Running it in production requires observability of agent inputs, outputs and every intermediate step, including the reasoning chain, to diagnose reasoning failures, measure latency bottlenecks, and audit the decision-making process. The Tracing Module captures a structured record of every agent interaction, providing both local persistence for debugging and long-term monitoring.

The tracing system organises observability data into a three-level hierarchy, illustrated in Figure 5.12. Each level captures progressively finer-grained information about the agent's execution.



**Figure 5.11:** Sequence of one chat turn.



**Figure 5.12:** Trace data hierarchy.

At the top level, a Trace Session spans the student's entire connected session and serves as the root container. Each student message creates a Chat Trace containing the original query, the classified intent, and the final response. Within each chat, Step Traces record the execution of each workflow node: the prompt to invoke language model (truncated to fit the context window and save storage), a snapshot of the AgentState at that moment containing the student's learning state, any tool execution results, and the model's output. The writer employs dual locking mechanisms—a threading lock for synchronous contexts and an asynchronous lock for coroutine contexts—ensuring thread safety when multiple workflow nodes attempt concurrent trace writes.

For production monitoring, the system integrates with Langfuse, a cloud-based observability platform built for LLM applications. The integration operates through LangChain's callback protocol: a Langfuse callback handler is injected into the runtime configuration when a session starts, and captures every LLM invocation and tool call inside the system.

## 5.7 Frontend Module

The frontend is a single study workspace built on the authenticated session (Figure 5.13). Pages are composed from shared components, components read the authentication and session contexts, and every request passes through one client that attaches the access token and renews it when it expires. A route guard sends unauthenticated users to the sign-in page. The session manager opens the session named in the address, otherwise the one the browser last used, otherwise the latest one returned by the Student service; in every case the Student service checks that the session belongs to the caller.

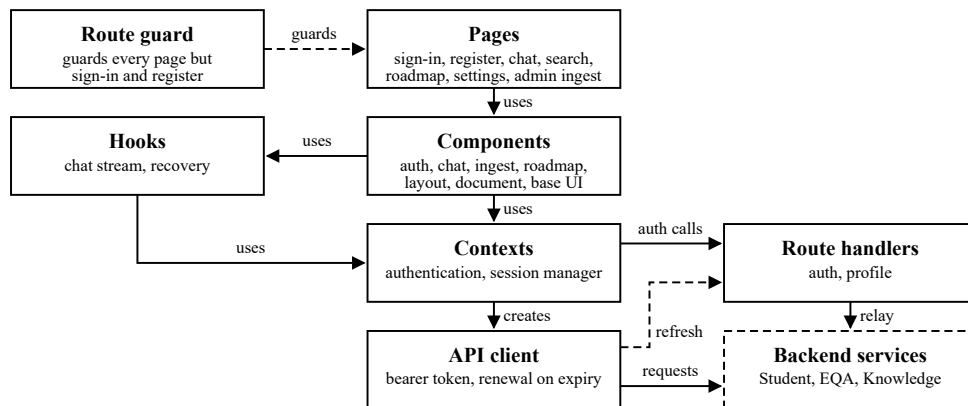


Figure 5.13: Module view of the frontend.

A chat turn follows the states of Figure 5.14. Progress steps and answer tokens arrive as a stream; if the stream closes early, the interface asks for the task status instead of sending the question again. The same status check resumes a turn that was still running when the conversation is reopened. Each result is bound to the session it came from, so a late event cannot appear in another conversation. Cited sources are listed under the answer and open the document at the cited page only when the student selects them. Chapter 6 reports how these behaviours were observed on the running system.

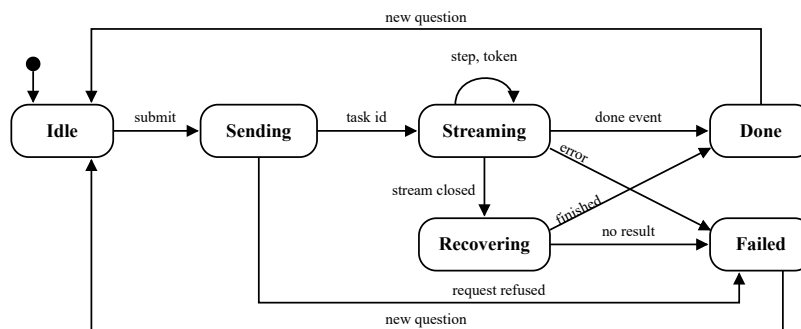


Figure 5.14: States of one chat turn in the interface.

# Chapter 6

## Experiment

This chapter shows the system of Chapter 5 running. It first covers the deployment of the storage engines, the backend and the ingestion orchestrator, then the three main functions of the system: login, course ingestion and chat. For each function it lists the endpoints involved, the frontend that exposes them, and what the backend produced or measured at run time.

### 6.1 Application Deployment

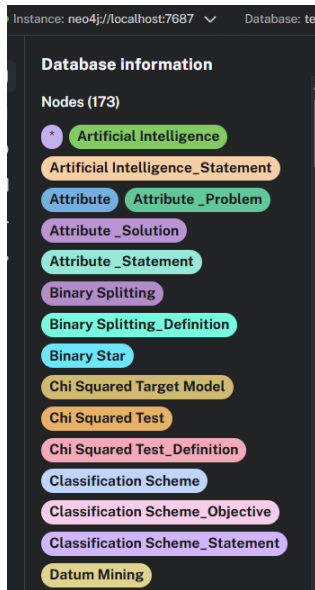
#### 6.1.1 Database Setup

The initialization of Docker containers was executed successfully, establishing the foundational database infrastructure.

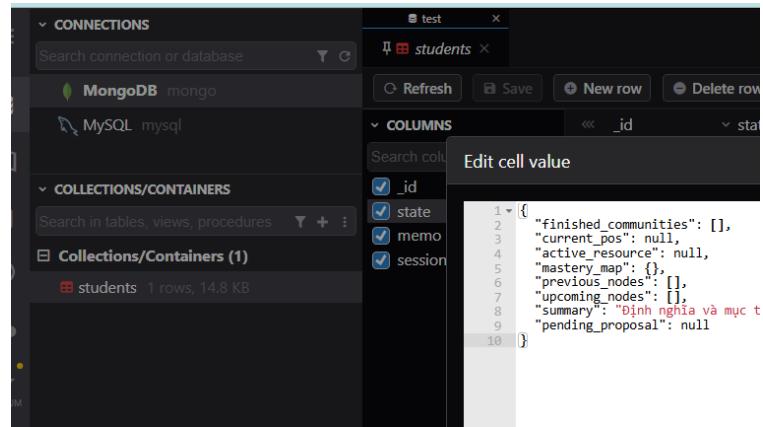
|                                                                                                                                                                                                                                                      |                                                                                     |                                                                                     |                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>minio-core</b><br>minio/minio:latest<br>9000:9000 <br>Show all ports (2)   |  |  |  | cd:2379"}<br>2026-05-19 23:33:04 milvus-etcd   {"level":"info","ts":"2026-05-19T16:33:04.942Z","caller":"flags/flag.go:113","msg":"recognized and used environment variable","variable-name":"ETCD_AUTO_COMPACTION_MODE","variable-value":"revision"}<br>2026-05-19 23:33:04 milvus-etcd   {"level":"info","ts":"2026-05-19T16:33:04.942Z","caller":"flags/flag.go:113","msg":"recognized and used environment variable","variable-name":"ETCD_AUTO_COMPACTION_RETENTION","variable-value":"1"}<br>2026-05-19 23:33:04 milvus-etcd   {"level":"info","ts":"2026-05-19T16:33:04.942Z","caller":"flags/flag.go:113","msg":"recognized and used environment variable","variable-name":"ETCD_LISTEN_CLIENT_URLS","variable-value":"http://0.0.0.0:2379"}<br>2026-05-19 23:33:04 milvus-etcd   {"level":"info","ts":"2026-05-19T16:33:04.942Z","caller":"flags/flag.go:113","msg":"recognized and used environment variable","variable-name":"ETCD_QUOTA_BACKEND_BYTES","variable-value":"4294967296"}<br>2026-05-19 23:33:04 milvus-etcd   {"level":"info","ts":"2026-05-19T16:33:04.942Z","caller":"flags/flag.go:113","msg":"recognized and used environment variable","variable-name":"ETCD_SNAPSHOT_COUNT","variable-value":"50000"}<br>2026-05-19 23:33:04 milvus-etcd   {"level":"info","ts":"2026-05-19T16:33:04.943Z","caller":"etcdmain/etcd.go:73","msg":"Running: ", "args":["/usr/local/bin/etcd"]}<br>2026-05-19 23:33:04 milvus-etcd   {"level":"warn","ts":"2026-05-19T16:33:04.943Z","caller":"etcdmain/etcd.go:105","msg":"'data-dir' was empty; using default","data-dir":"default.etcd"}<br>2026-05-19 23:33:04 milvus-etcd   {"level":"info","ts":"2026-05-19T16:33:04.944Z","caller":"etcdmain/etcd.go:116","msg":"server has been already initialized"," |
|  <b>mysql-core</b><br>mysql:8.0<br>3307:3306                                   |  |  |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|  <b>neo4j-core</b><br>neo4j:2025.12.1-ente<br>7474:7474 <br>Show all ports (2) |  |  |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|  <b>milvus-attu</b><br>zilliz/attu:latest<br>8000:3000                         |  |  |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|  <b>milvus-etcd</b><br>coreos/etcd:v3.5.5<br>                                  |  |  |  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |

Figure 6.1: Docker container deployment.

Each data container also has a monitoring interface, used for debugging and for testing the tools directly against the stores.



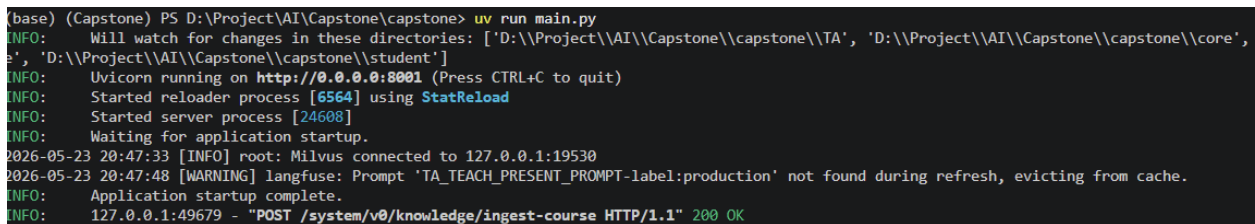
**Figure 6.2:** Neo4j Monitor Plugins



**Figure 6.3:** DBgate monitoring both MongoDB and MySQL

**Figure 6.4:** Database Admin Monitoring and Visualizing

## 6.1.2 FastAPI App Deployment



**Figure 6.5:** FastAPI successful lifecycle start

The application is hosted by uvicorn. The lifespan creates the connection pools, initializes the singleton modules, and the server listens on port 5000. Figure 6.5 shows a successful startup, with the endpoints of the main components exposed.

## 6.1.3 Ingestion Orchestrator

The ingestion pipeline runs on a Prefect server (Section 2.1.5) deployed as its own container on port 4200, which keeps its run history in a local volume. The heavy stages are registered as deployments on three process work pools, one each for document parsing, LLM extraction and speech recognition, and a worker polls each pool, so every heavy stage runs in its own process. The Prefect interface is used to follow a course run, its subflows and the logs of each task (Section 6.3).

## 6.2 Login

### 6.2.1 Service Endpoints

The Student Module exposes the endpoints for identity and session management:

- POST /register
  - Use case: Registers new student profiles, hashes credentials, and initializes a default state document.
  - Input schema:
    - \* student\_id (String): The unique identifier chosen by the student.
    - \* password (String): The plain-text password to be securely hashed.
  - Output schema:
    - \* detail (String): Confirmation message of successful registration.
- POST /login
  - Use case: Validates student credentials and issues a short-lived access token together with a longer-lived refresh token.
  - Input schema:
    - \* username (Form String): The student's unique identifier.
    - \* password (Form String): The student's password.
  - Output schema:
    - \* access\_token (String): The JWT used for subsequent authenticated requests.
    - \* refresh\_token (String): The token exchanged for a new access token; the frontend proxy keeps it in an HTTP-only cookie.
    - \* is\_admin (Boolean): Whether the account may ingest course material.
    - \* token\_type (String): The token category, typically set to "bearer".
- POST /refresh
  - Use case: Exchanges a valid refresh token for a new access token, re-reading the administrator flag.
  - Input schema:
    - \* refresh\_token (String): The refresh token forwarded by the frontend proxy from its cookie.
  - Output schema:
    - \* access\_token, is\_admin, token\_type: As returned by /login.
- POST /session/start
  - Use case: Generates a session-specific UUID token to isolate chat interactions.
  - Input schema:
    - \* Authorization (Header String): Bearer token (JWT) of the authenticated student.
  - Output schema:
    - \* session\_id (String): A uniquely generated UUID functioning as an opaque token for a specific chat instance.
- GET /sessions

- Use case: Lists the stored sessions of the authenticated student for the conversation history.
- Input schema:
  - \* `limit, offset` (Query Integer): Paging of the list.
  - \* `q` (Query String): Optional text filter.
  - \* `Authorization` (Header String): Bearer token (JWT) of the authenticated student.
- Output schema:
  - \* `items` (List of Objects): Each session's `id`, `name`, `preview` (first question) and `updated_at`, most recently updated first.
  - \* `total` (Integer): The number of matching sessions.
- `POST /sessions/{session_id}/resume`
  - Use case: Reopens a stored session of the same student, for example after a page reload or a backend restart.
  - Input schema:
    - \* `session_id` (Path String): The UUID of the stored session.
    - \* `Authorization` (Header String): Bearer token (JWT) of the authenticated student.
  - Output schema:
    - \* `session_id` (String): The resumed session, registered again for chat.
- `DELETE /session/end`
  - Use case: Discards the active session token to cleanly terminate the session.
  - Input schema:
    - \* `session_id` (Query String): The UUID of the session to terminate.
    - \* `Authorization` (Header String): Bearer token (JWT) of the authenticated student.
  - Output schema: Returns HTTP 204 (No Content) indicating successful deletion.

## 6.2.2 Frontend

The sign-in and registration pages call these endpoints through the frontend proxy, which keeps the refresh token in an HTTP-only cookie and renews the access token when it expires. A route guard sends unauthenticated users to the sign-in page, and the session manager reopens the session named in the address, otherwise the one last used in the browser, otherwise the latest session returned by `/sessions` (Section 5.7).

### 6.2.3 System Runtime

The module handles user sessions, including the detection of an existing account during registration.

```
INFO: 127.0.0.1:56103 - "POST /system/v0/student/register HTTP/1.1" 409 Conflict
INFO: 127.0.0.1:56103 - "POST /system/v0/student/login HTTP/1.1" 200 OK
INFO: 127.0.0.1:56103 - "POST /system/v0/student/session/start HTTP/1.1" 200 OK
2026-05-23 22:39:50 [INFO] TA.api.route: TA task 3a7f100c-0c53-4bba-9db2-814297925483 scheduled for session d6ccf424-6b72-4851-acba-a1806685285b.
```

**Figure 6.6:** Existing ID returns 409; login, session start, and task scheduling succeed.

In the frontend live test (Section 6.4.2), two student accounts signed in; the second account did not display the first account's conversations, and the first account resumed its stored session after a backend restart.

## 6.3 Course Ingestion

### 6.3.1 Service Endpoints

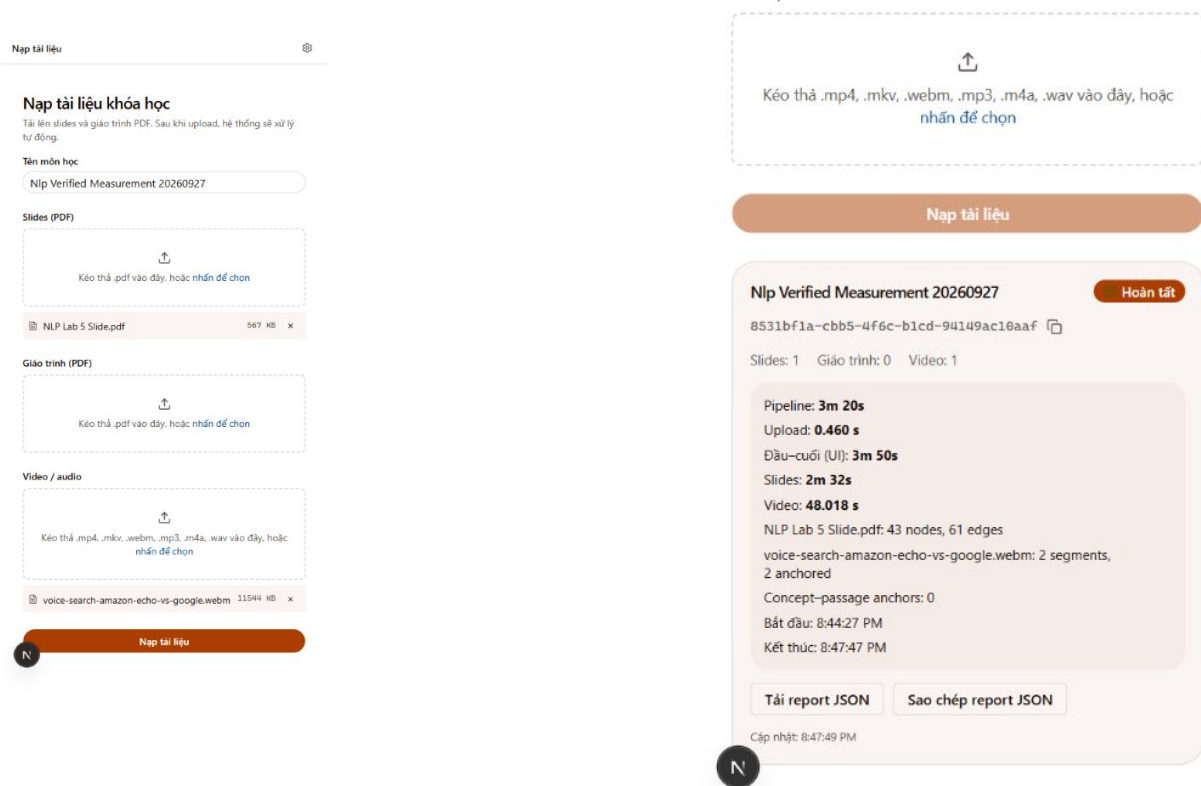
Ingestion is restricted to administrator accounts and takes three requests, with the file bytes sent directly to object storage between the first and the second:

- POST /upload-url
  - Use case: Validates the file names and types (PDF, or video and audio in mp4, mkv, webm, mp3, m4a and wav) and returns one presigned upload address per file. The browser then uploads each file with a direct PUT to MinIO, so the backend never carries the file bytes.
  - Input schema:
    - \* course\_name (String): The title of the course being ingested.
    - \* file\_names (List of Strings): The files to be uploaded.
  - Output schema:
    - \* targets (List of Objects): One file\_name and presigned url per file.
- POST /ingest-course
  - Use case: Checks that the uploaded files exist and that the workers required by the submitted material types are available, then submits a course flow run to the orchestrator. A missing worker capability is answered with HTTP 503 listing what is missing.
  - Input schema:
    - \* course\_name (String): The title of the course being ingested.
    - \* slide\_files (List of Strings): The slide files to be processed.
    - \* textbook\_files (List of Strings): The textbook files to be processed.
    - \* video\_files (List of Strings): The video or audio files to be processed.
    - \* reset (Boolean): Whether to reset the existing course data before proceeding.
  - Output schema:
    - \* status (String): The acceptance status of the run.
    - \* message (String): A descriptive confirmation message.
    - \* flow\_run\_id (String): The identifier of the submitted run, used to follow it.
    - \* details (Object): The number of slides, textbooks and videos queued.
- GET /ingest-report
  - Use case: Returns the report that the run writes to object storage when it starts and updates when it ends.
  - Input schema:
    - \* course (Query String): The course of the run.

- \* `run_id` (Query String, optional): The run to read; the latest run when omitted.
- Output schema: The run status, the start and finish times, the duration of each stage, the outputs per file (sections and passages, nodes and edges, segments and anchored segments), the number of concept–passage anchors, and any errors.

## 6.3.2 Frontend

The administrator page (Figure 6.7) takes a course name and three groups of files: slides, textbooks, and video or audio. It requests the upload addresses, uploads the files concurrently to object storage with a progress bar per file, and submits the run without resetting existing course data. A run card then polls the report every four seconds and shows the status, the duration of each stage, the outputs of each file and a link to the run in the Prefect interface; once the run ends, the report can be downloaded or copied as JSON.



**Figure 6.7:** Ingestion page: files selected (left); completed run card (right).

## 6.3.3 System Runtime

Figure 6.8 shows the completed course flow of the measured run in the Prefect interface: the slide subflow, the persistence of its graph, then the video subflow. Each subflow opens into its stage flows and tasks, with their own states and logs.

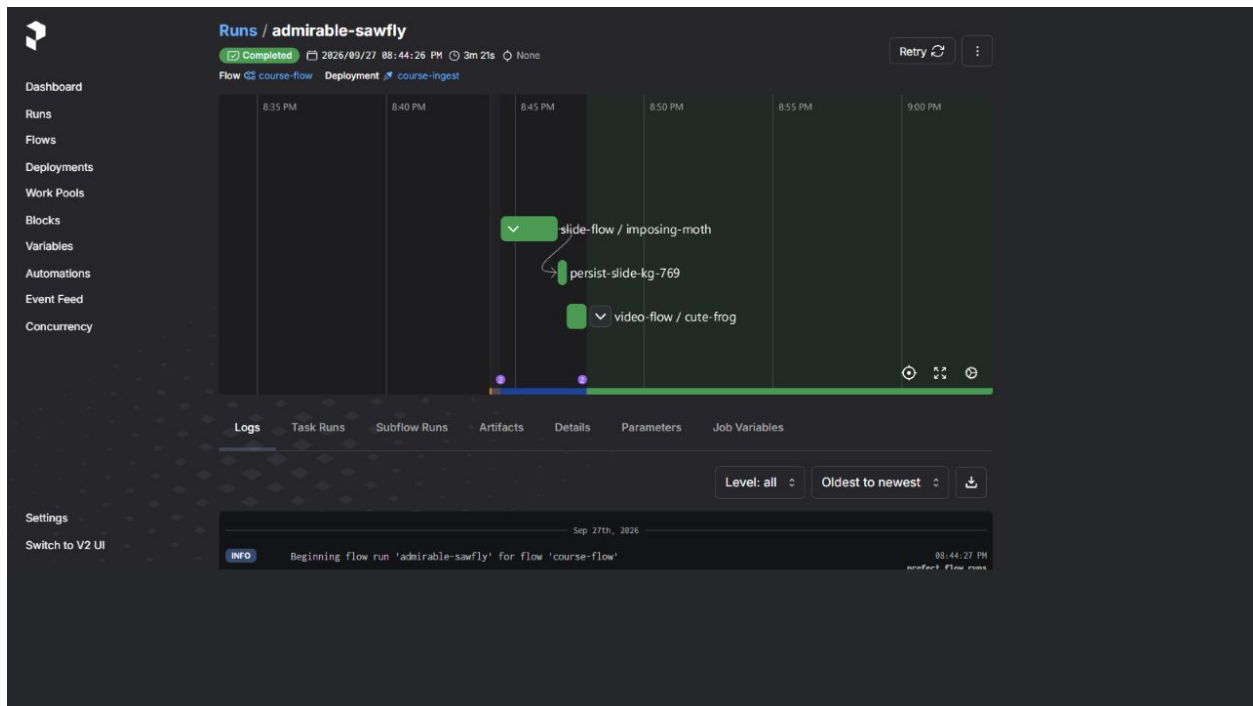


Figure 6.8: Completed course flow in the Prefect interface.

The extraction produces normalized, filtered nodes and edges.

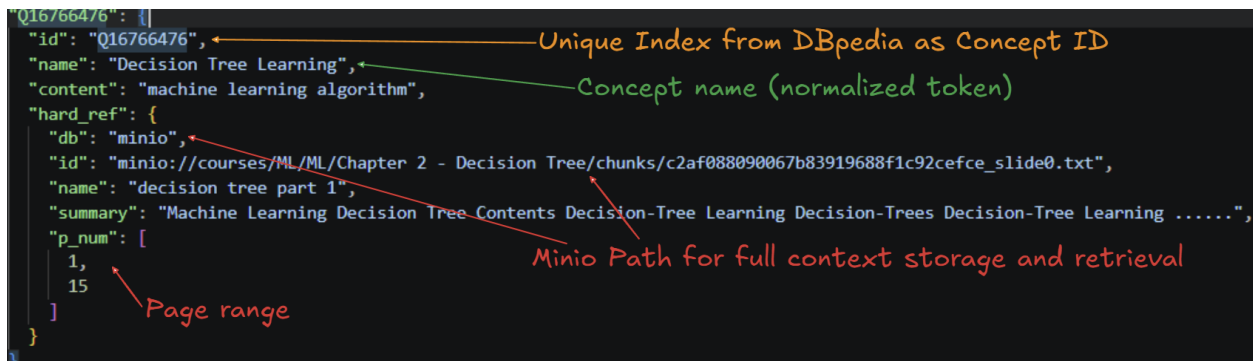
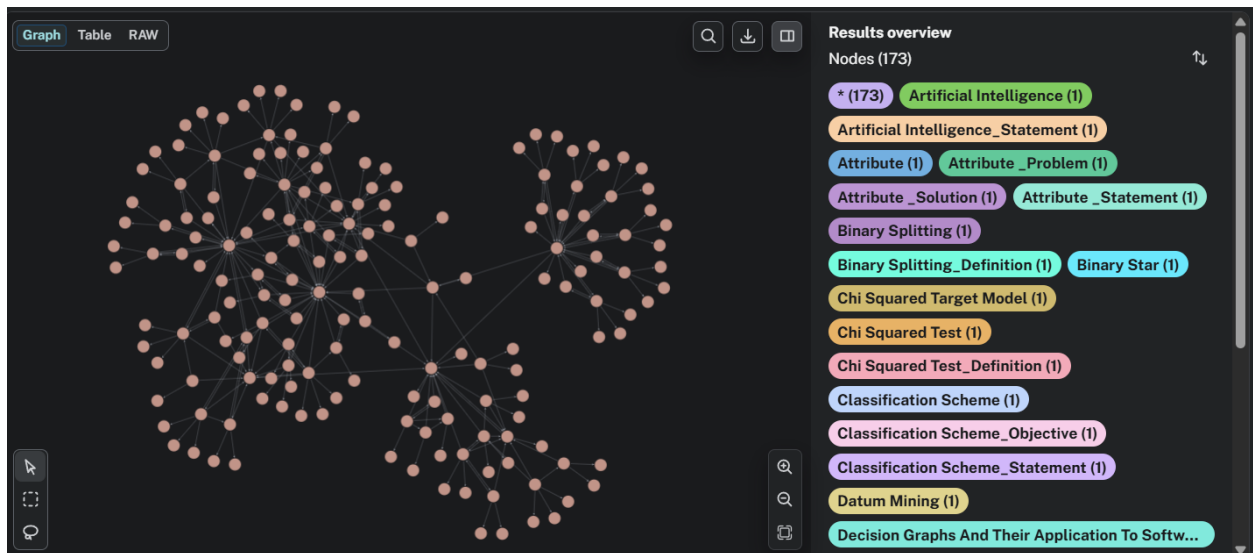


Figure 6.9: Extracted Concept Nodes

The resulting graph is written to Neo4j.



**Figure 6.10:** Completed knowledge graph in Neo4j.

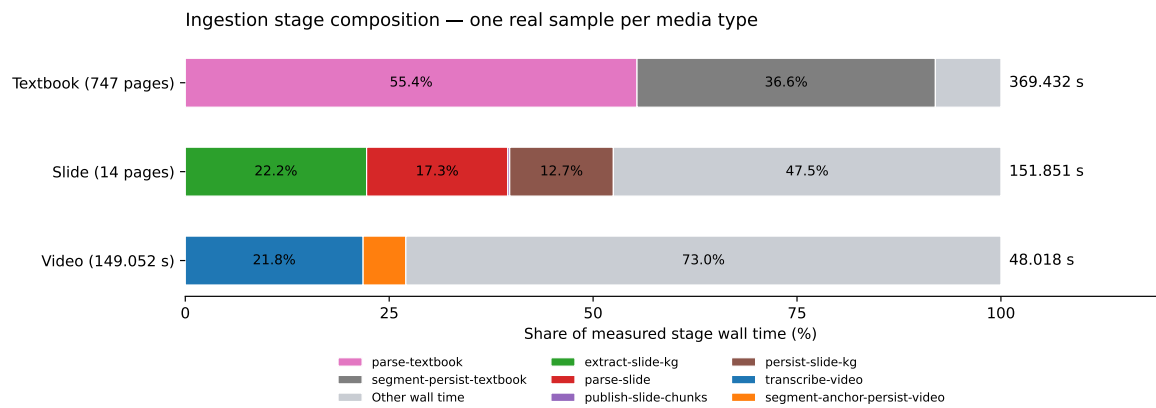
### 6.3.4 Measurement

Live ingestion was measured on 27 September 2026 through the administrator page, on a laptop with an AMD Ryzen 7 6800HS processor, 16 GB of memory and an NVIDIA RTX 3060 Laptop GPU with 6 GB of memory. Graph and topic extraction used GPT-OSS 120B through Ollama cloud, speech recognition used the *tiny.en* Whisper model on the CPU with 8-bit integer computation, and embeddings used SciBERT. The stores already held earlier courses and the runs did not reset them. The inputs were a 747-page NLP textbook, a 14-page slide deck on word embeddings, and a 149-second public video demonstrating voice search.

**Table 6.1:** Measured ingestion time and output per source, one sample each.

| Source   | Input     | Stage time (s) | Output                                |
|----------|-----------|----------------|---------------------------------------|
| Textbook | 747 pages | 369.4          | 446 sections, 1,227 passages          |
| Slide    | 14 pages  | 151.9          | 43 nodes, 61 edges, 40 vectors        |
| Video    | 149 s     | 48.0           | 2 segments, both anchored to concepts |

The textbook row comes from a completed textbook subflow whose parent run later failed while persisting the slide graph. After that failure was corrected, the slide and the video were submitted again in a separate run, which completed without errors: its pipeline took 199.97 s, and the browser recorded 229.78 s from submission to the receipt of the final report, including 0.46 s for the concurrent uploads of both files. These totals cover the slide and the video only. Because that run had no textbook, it skipped the concept–passage linking stage, and its report shows zero anchors. No task in either run was served from the cache or retried by the orchestrator, and the stored passages, vectors and anchored segments were checked afterwards against the reported counts.



**Figure 6.11:** Stage-time composition per source; “other” covers dispatch, startup, waiting and result transfer.

Figure 6.11 breaks each stage into the task durations recorded by Prefect. The textbook is dominated by document parsing (204.6 s, 55.4%) and by the fused segmentation, embedding and persistence task (135.1 s, 36.6%). The slide spends 33.8 s in LLM extraction, 26.3 s in parsing and 19.3 s in graph and vector persistence, while 47.5% of its stage time lies outside the tasks. The video is transcribed in 10.5 s, faster than real time, but 73.0% of its stage time is spent outside the tasks. For short inputs, dispatching a stage to its worker process and starting it therefore costs more than the work itself.

These are single measurements of one run per source type on a populated database. They show that each route executes and persists its output; they are not a throughput benchmark, and they do not measure the accuracy of the extraction or of the transcription.

## 6.4 Chat

### 6.4.1 Service Endpoints

The Educational QA Agent exposes the chat endpoints, and the Knowledge Module the read-only endpoints used by the study workspace:

- POST /chat
  - Use case: Serves as the primary tutoring interface. It receives user queries alongside their active session IDs, subsequently executing the multi-agent LangGraph tutoring pipelines to formulate contextual responses.
  - Input schema:
    - \* session\_id (String): The UUID for the active chat session authenticated by the user.
    - \* user\_input (String): The actual query or message submitted by the student.
    - \* language (String): The language preference toggle (e.g., “vn” or “eng”).
  - Output schema:
    - \* task\_id (String): A unique identifier assigned to track the asynchronous background task.

- \* `status` (String): The initial state, typically indicating receipt of the request.
  - \* `agent` (String): The default agent assigned to process the workflow.
- `GET /chat/stream/{task_id}`
  - Use case: The main channel of a chat turn. Opens a server-sent event stream for the task, available only to the student who submitted it.
  - Input schema:
    - \* `task_id` (Path String): The identifier returned by `POST /chat`.
    - \* `Authorization` (Header String): Bearer token (JWT) of the authenticated student.
  - Output schema: A sequence of events: `step` (progress label), `token` (answer text as it is generated), and a final `done` (full message and optional `ui_action`) or `error`.
- `GET /chat/status/{task_id}`
  - Use case: Returns the same progress by polling, for clients that cannot hold a stream or that reconnect after the stream closed. Like the stream, it answers only the student who submitted the task.
  - Input schema:
    - \* `task_id` (Path String): The unique identifier of the background task being polled.
    - \* `Authorization` (Header String): Bearer token (JWT) of the authenticated student.
  - Output schema:
    - \* `status` (String): The current operational state (e.g., "working", "finished", or "Fail").
    - \* `agent_name`, `intent`, `thought` (String, optional): Metadata illustrating the agent's current step, intended action, and reasoning.
    - \* `result` (Object, optional): Available upon completion, containing the final message string and optional `ui_action` dictionaries.
    - \* `error` (String, optional): Descriptive failure logs in the event of pipeline exceptions.
- `GET /roadmap/{course}`
  - Use case: Returns the concept graph of a course for the roadmap view, decorated with the student's mastery and status when a personal path exists.
  - Input schema:
    - \* `course` (Path String): The course to display.
    - \* `Authorization` (Header String): Bearer token (JWT) of the authenticated student.
  - Output schema: The course concepts and prerequisite links, each concept with its mastery and status, and a `has_personal_path` flag.
- `GET /search`
  - Use case: Searches the course material by meaning, over concept vectors and textbook passages.

- Input schema:
  - \* `q` (Query String): The search text, 2 to 200 characters.
  - \* `course` (Query String, optional): Restricts the search to one course.
- Output schema: Two groups, concepts and passages, each hit with its title, snippet, score, source document and page.
- `GET /courses`, `GET /courses/{course_name}/topics`
  - Use case: List the courses held in object storage and the topics of one course, for the sidebar of the workspace.
  - Input schema: `course_name` (Path String) for the topic list.
  - Output schema: The list of course names, or the course with its list of topics.
- `GET /pdf/{course_name}/{topic}`, `GET /pdf/raw/{course_name}/{file_name}`
  - Use case: Serve the PDF of one topic, or the original uploaded PDF, to the document reader; a missing file returns HTTP 404.
  - Input schema: The course with the topic or the original file name (Path Strings).
  - Output schema: The PDF bytes.

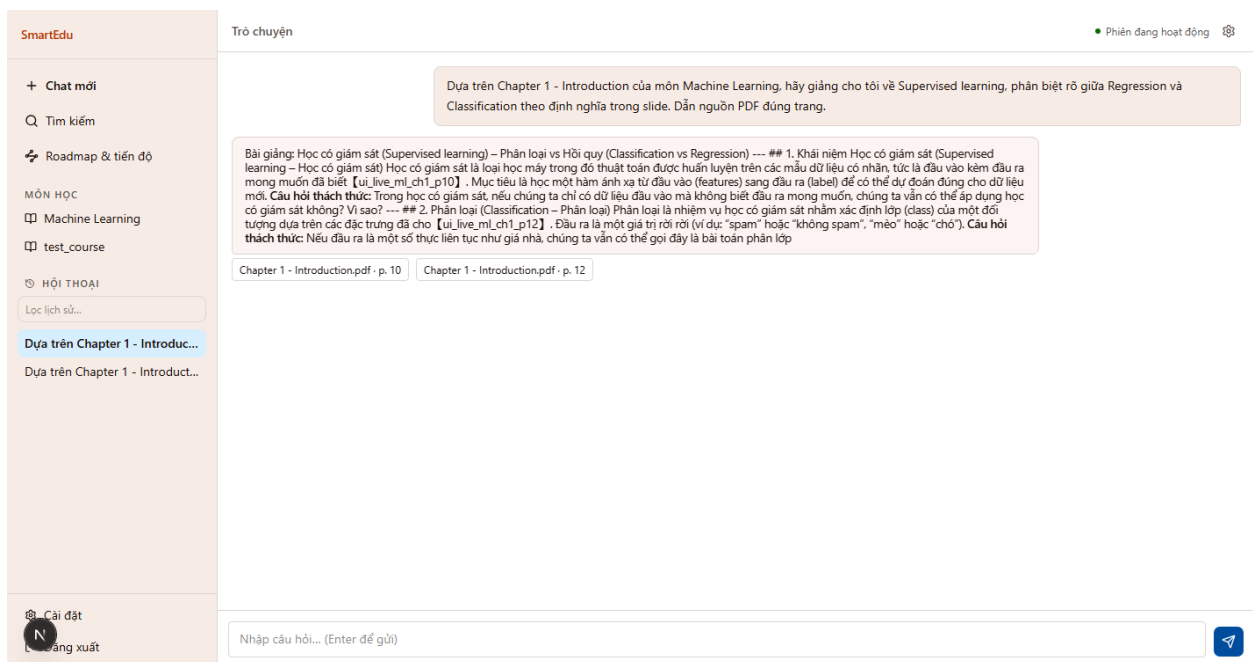
### 6.4.2 Frontend and Live Validation

The latest live test was exercised on 24 September 2026 with a locally running production build, the authenticated backend and the existing storage services. Two separate student accounts were used to check that conversation history does not cross account boundaries (Section 6.2). For the document-reader check, the first three pages of a Machine Learning slide already present in the project were staged as one topic PDF, and its 32-page original was staged as the source PDF. This sample tests document navigation; it is not presented as a completed course-ingestion run. The teaching request and the roadmap were then repeated on 26 September 2026 with a third student account against the Machine Learning Chapter 1 slide, and those two rows of Table 6.2 report the repeated run.

**Table 6.2:** Observed frontend behavior in the local live test.

| Action              | Observation                                                                                                                                                                                                                                                                                          |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Document navigation | Selecting the sample topic opened page 2 of 3. A direct source-document link opened page 12 of the 32-page original.                                                                                                                                                                                 |
| Phone layout        | Chat and PDF appeared as separate tabs. Returning to Chat kept an unsent draft; returning to PDF kept the selected page.                                                                                                                                                                             |
| Search              | The unfiltered request returned concept and passage groups from the available benchmark corpus. Restricting the request to the staged Machine Learning course returned no indexed hits, as no ingestion was performed for that sample.                                                               |
| Roadmap             | The Machine Learning course displayed a prerequisite graph of Supervised learning, Classification and Regression with the no-personal-path notice; no mastery percentage was shown.                                                                                                                  |
| Teaching request    | A question on Supervised learning, Regression and Classification in Chapter 1 completed with all workflow nodes executed. Two slides (pages 10 and 12) were retrieved, the lecture cited both inline, and two citation controls appeared under the answer; selecting the first opened page 10 of 32. |

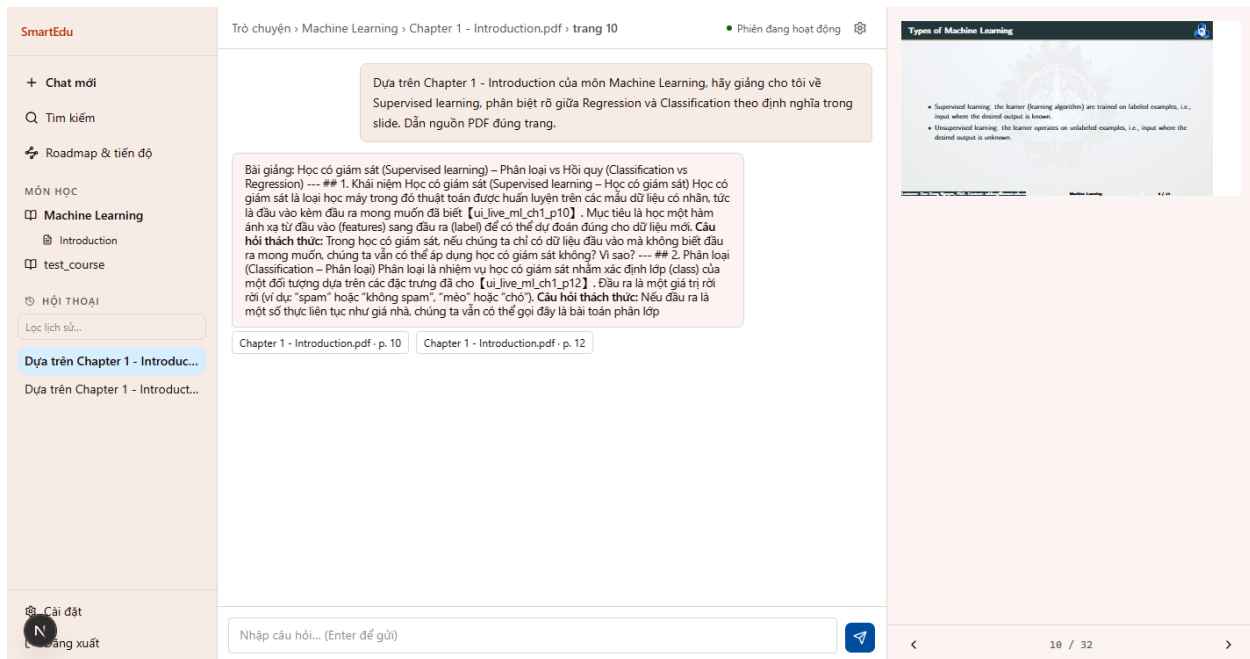
Figure 6.12 shows the answer to the Chapter 1 question. The lecture keeps the inline page markers, and one citation control per cited slide is placed below it, labelled with the file name and page.



The screenshot shows the SmartEdu interface. On the left is a sidebar with navigation options: Chat mới, Tìm kiếm, Roadmap & tiến độ, MÔN HỌC (Machine Learning, test\_course), and HỘI THOẠI. The main content area displays a lecture answer titled "Dựa trên Chapter 1 - Introduction của môn Machine Learning, hãy giảng cho tôi về Supervised learning, phân biệt rõ giữa Regression và Classification theo định nghĩa trong slide. Dẫn nguồn PDF đúng trang." Below the title is a text block containing the lecture content with inline citations: "Bài giảng: Học có giám sát (Supervised learning) – Phân loại vs Hồi quy (Classification vs Regression) --- ## 1. Khái niệm Học có giám sát (Supervised learning – Học có giám sát) Học có giám sát là loại học máy trong đó thuật toán được huấn luyện trên các mẫu dữ liệu có nhãn, tức là đầu vào kèm đầu ra mong muốn đã biết [ui\_live\_ml\_ch1\_p10] . Mục tiêu là học một hàm ánh xạ từ đầu vào (features) sang đầu ra (label) để có thể dự đoán đúng cho dữ liệu mới. Câu hỏi thách thức: Trong học có giám sát, nếu chúng ta chỉ có dữ liệu đầu vào mà không biết đầu ra mong muốn, chúng ta vẫn có thể áp dụng học có giám sát không? Vì sao? --- ## 2. Phân loại (Classification – Phân loại) Phân loại là nhiệm vụ học có giám sát nhằm xác định lớp (class) của một đối tượng dựa trên các đặc trưng đã cho [ui\_live\_ml\_ch1\_p12] . Đầu ra là một giá trị rời rạc (ví dụ: "spam" hoặc "không spam", "mèo" hoặc "chó"). Câu hỏi thách thức: Nếu đầu ra là một số thực liên tục như giá nhà, chúng ta vẫn có thể gọi đây là bài toán phân lớp". Below the text are two citation controls: "Chapter 1 - Introduction.pdf · p. 10" and "Chapter 1 - Introduction.pdf · p. 12". At the bottom is a search bar with the placeholder text "Nhập câu hỏi... (Enter để gửi)".

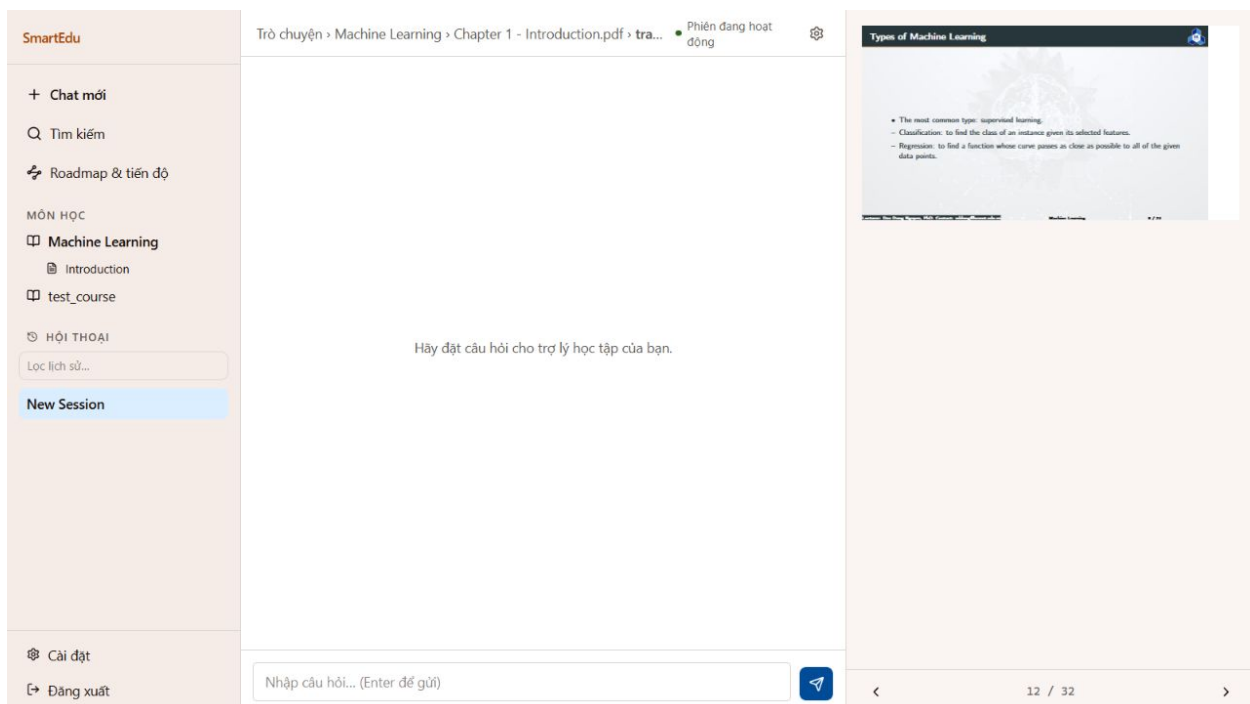
**Figure 6.12:** Lecture answer with inline citations.

Figure 6.13 shows the desktop workspace after selecting the first citation. The document panel opens the original slide file at page 10 of 32, the top bar records the course, file and page, and the sidebar retains the selected course and the active session.



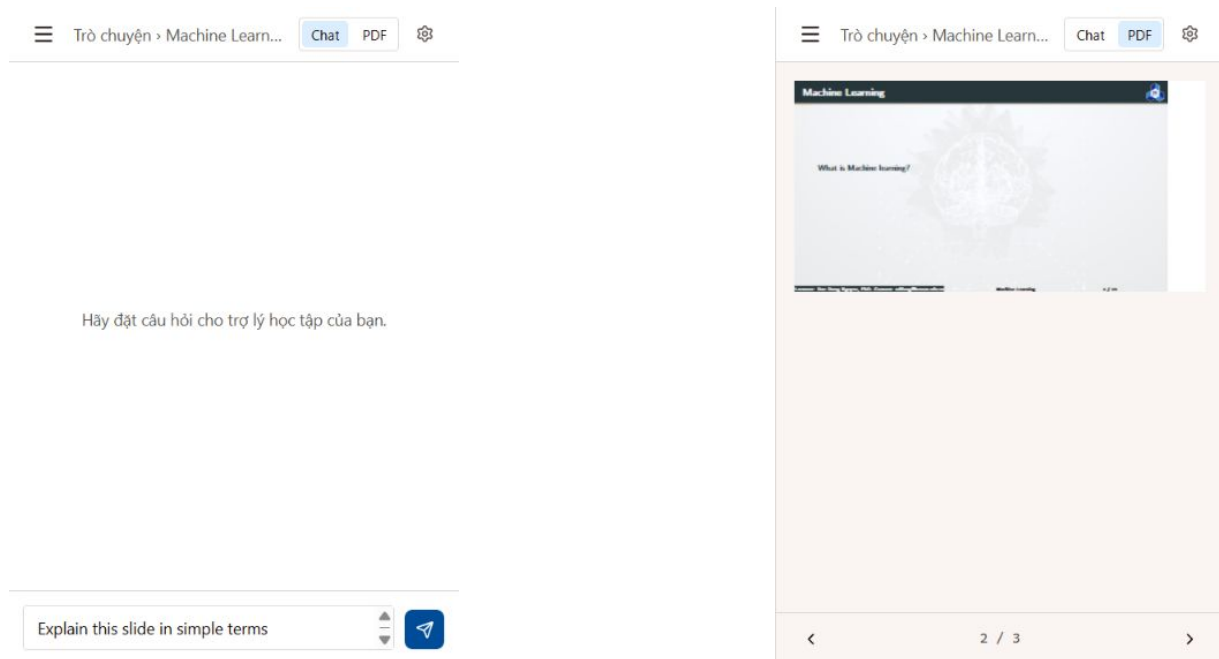
**Figure 6.13:** Desktop view with the cited slide open.

The original source document follows a different page coordinate system. Figure 6.14 records page 12 of 32, opened from a document target rather than by advancing through the three-page topic file. This distinction is necessary for citations that refer to a page in the original textbook or slide file.



**Figure 6.14:** Original source PDF opened directly.

On a phone-sized viewport, the same workspace becomes two tabs (Figure 6.15). The PDF tab kept page 2 while the Chat tab retained the text typed before switching. The menu provides the navigation that remains visible in the desktop sidebar.



**Figure 6.15:** *Phone Chat and PDF tabs.*

The search and roadmap screens also expose missing-data states rather than filling them with assumed results. Figure 6.16 shows that the course filter removes benchmark hits unrelated to the staged sample. Figure 6.17 shows the course graph of the Machine Learning course with its prerequisite links, and the notice that no mastery percentage appears because this student has no personal learning path yet.

### Tìm trong kho học liệu

Khái niệm và đoạn sách liên quan đến câu hỏi của bạn.

Machine Learning

Tìm kiếm

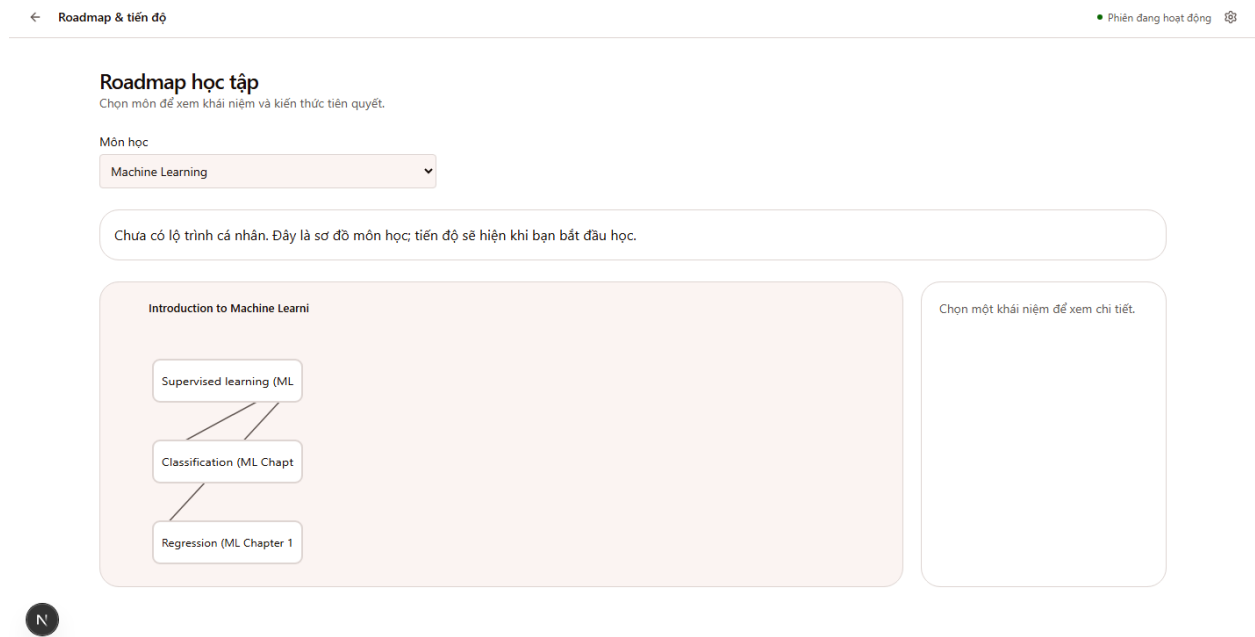
#### Khái niệm (0)

Không có kết quả.

#### Đoạn sách (0)

Không có kết quả.

**Figure 6.16:** *Course-filtered search.*



**Figure 6.17:** Course roadmap with prerequisite links.

A separate legacy test object was named as a PDF but contained plain text. Its topic link produced a missing-document response, so it was excluded from the document-reader observations above. This finding concerns the stored fixture and remains visible in the course list.

## 6.4.3 System Runtime

After a query is submitted, the client follows the event stream and displays the current step and the active agent. The answer text appears as it is generated, and the final event carries the complete message.

```
login Successful
Submitting query to http://127.0.0.1:8001/system/v0/ta/chat...
Task accepted - id=3a7f100c-0c53-4bba-9db2-814297925483, status=received, agent=TA
Waiting 2 seconds before first status check...
[ 2s] status=working | agent=TA_Router | intent=retrieve
[ 17s] status=working | agent=TA_Router | intent=retrieve
[ 32s] status=working | agent=TA_Router | intent=retrieve
[ 47s] status=working | agent=TA_Router | intent=retrieve
[ 62s] status=finished | agent=TA_Retrieve_Finish | intent=retrieve

--- TA Response ---
## Định nghĩa và mục tiêu của **Học máy** (Machine Learning) - **Học máy**

- **Học máy** (Machine Learning) là một **lĩnh vực nghiên cứu** (field of study) tập trung vào việc xây dựng các **chương trình** (programs) có khả năng **tự động cải thiện** (automatically improving) thông qua **kinh nghiệm** (experience).
- Mục tiêu **chính** (primary objective) của **Học máy** là **tăng cường hiệu suất** (enhance performance) trên các **nhiệm vụ** (tasks) bằng cách **học** (learning) từ **kinh nghiệm** và **đánh giá** (using a defined performance measure) dựa trên **độ đo hiệu suất** (performance measure).

### Gợi ý học tập tiếp theo
Nếu bạn muốn đi sâu hơn, hãy khám phá các **khái niệm nền tảng** như:
- **Học có giám sát** (Supervised Learning) - **Học có giám sát**
- **Học không giám sát** (Unsupervised Learning) - **Học không giám sát**
- **Học tăng cường** (Reinforcement Learning) - **Học tăng cường**

Bạn có thể bắt đầu bằng cách đọc các tài liệu giới thiệu về **Học có giám sát** để nắm vững cách **Mô hình** (models) học từ **dữ liệu gán nhãn** (labeled data).

Result saved to: test/TA/TA_v0_223818_2305.json
Time taken for question 1 [What is machine Learning]:
65.08477306365967
```

**Figure 6.18:** Client-side agentic interaction flow.

The system also logs the tool results and messages of each agent, so they can be reused within the session and reviewed afterwards.

```
EXAMPLES:
- Query: "Tôi mới bắt đầu, nên học gì?" → recommend_new() (global)
- Query: "Học Deep Learning" → recommend_new(course_filter="Deep Learning")
"""
),
AIMessage(
    content="",
    tool_calls=[
        {
            "name": "recommend_new",
            "args": {
                "course_filter": "Machine Learning"
            }
        }
    ]
),
ToolMessage(
    name="recommend_new",
    content=""
)

TOP 10 HUB CONCEPTS (by connectivity):
1. [Topic] Decision Tree Learning
- course: Machine Learning
- connections: 20
- mastery: 0
- desc: machine learning algorithm

2. [Concept] Decision Support System
- course: Machine Learning
- connections: 18
- mastery: 0
- desc: system that supports business or organizational decisions

3. [Concept] Reduced Error Pruning
- course: Machine Learning
- connections: 12
- mastery: 0
- desc: pruning using hold-out validation

4. [Concept] Heraldic Attribute
- course: Machine Learning
```

```
AIMessage(
    content=""
)
Dựa trên knowledge graph, các hub node phù hợp để bắt đầu học Machine Learning gồm:
1. Decision Tree Learning
→ Thuật toán ML nền tảng dùng cho classification/regression.
2. Supervised Learning
→ Paradigm quan trọng nhất cho người mới bắt đầu.
3. Inductive Bias
→ Hiểu assumptions của model khi học dữ liệu.
4. Hold Out Set
→ Khái niệm validation/test trong training pipeline.
5. Information Gain Ratio
→ Metric quan trọng trong Decision Tree.
Đây là các node có độ kết nối cao, phù hợp làm entry point cho roadmap học Machine Learning.
"""
),
```

Figure 6.19: Two snippets of logged agent workflow results

The live test above checks behaviour, not speed. The latency of the retrieval workflow was measured separately in the benchmark of Section 4.5 (Table 4.4).

# Chapter 7

## Conclusion

### 7.1 Achieved Results

The SmartEdu system has been integrated into a working, agent-driven educational platform. Focusing on practical functionality and stable outputs, the project achieved the following results:

- **Unified Knowledge Graph:** The system automatically builds a connected Knowledge Graph from course material. In our experiments the constructed graph showed almost no concept fragmentation, and concepts stayed consistent throughout the course, giving the agents a reliable shared base.
- **Grounded AI Operations:** Guided by the agent harness, the multi-agent system runs stably. It retrieves evidence, maps out learning roadmaps, and identifies backbone concepts; grounding every answer in retrieved evidence reduced hallucination and off-topic drift in our experiments, although it does not remove them entirely.
- **Microservice Design:** The backend is split into three services with narrow entry points and single-writer data ownership, and state management is kept separate from AI processing.
- **Multimodal Input Processing:** The platform preserves the structural layout and visual references of the original materials, allowing agents to read and use this context for better answers.
- **Error Handling:** Agentic failures are detected and handled with fallbacks, and a busy or failing model produces an explicit message instead of a silent wrong result, so the system remains responsive.
- **Acceptable UI/UX:** The frontend is functional and comprehensible, allowing students to view documents and interact with the system.

### 7.2 System Limitations

Despite its functional success, the current version of the system has several practical limitations that affect its speed and long-term usability:

- **Unclear Course-Level Paths:** Defining prerequisites for an entire course is still difficult. The exact threshold for deciding when to merge overlapping topics remains undecided, which can cause confusion in the macro-roadmap.

- **Incomplete Frontend:** The frontend is not fully developed; only its completed parts were tested (Section 6.4.2), which limits usability and user experience.
- **High Latency:** QA time with local SLM is still somewhat unstable. A simple query takes 20-60 seconds, while complex tasks like generating a roadmap or a teaching response can take up to 200 seconds.
- **Limited Long-Term Tracking:** The context from which the Educational QA Agent assesses a situation is still about 80% tied to the present chat session. Agentic memory stores and retrieves past conversations, but it is not yet fully tested or optimized for long-term tracking, so the system lacks a reliable way to track and evaluate a student's progress over a long period.
- **Password Storage:** Passwords are hashed with unsalted SHA-256. A slow, salted algorithm such as bcrypt or Argon2 is required before a production deployment.
- **Limited Evaluation Coverage:** The evaluation of the system is currently limited to retrieval on MuSiQue. The quality of the knowledge graph, the roadmap, and the teaching workflow on real course material have not yet been evaluated, so the system's performance in real-world scenarios is still uncertain.
- **Security:** The system is not yet hardened against attacks. It is still vulnerable to SQL injection, XSS, and other common web application attacks.
- **Ingestion Recovery:** Every ingestion run is tracked by the orchestrator and its report, and the results of completed stages are preserved, but there is no effective recovery mechanism yet: a failed run is not resumed automatically and must be rerun manually on Prefect.

## 7.3 Future Plan

The limitations above define the next steps of the project:

- **Course-Level Paths:** Define a rule for merging overlapping topics, and evaluate the macro-roadmap on complete courses.
- **Frontend:** Complete the remaining screens and test them with real students.
- **Ingestion Recovery:** Ingestion runs are now orchestrated and tracked with Prefect, and completed stage results are kept in its cache. The next step is a recovery mechanism that resumes a failed run from its last completed stage automatically, replacing the current manual rerun.
- **Long-Term Tracking:** Test and optimize the agentic memory, so that a student's progress can be tracked and evaluated across many sessions.
- **Evaluation Coverage:** Extend the evaluation beyond retrieval on MuSiQue to the quality of the knowledge graph, the roadmap and the teaching workflow on real course material.
- **Security:** Replace the unsalted SHA-256 password hashing with bcrypt or Argon2.

# Bibliography

- [1] Qurat Ul Ain, Mohamed Amine Chatti, Jean Qussa, Amr Shakhshir, Rawaa Alatrash, et al. The edukgpipeline: An optimized pipeline for automatic educational knowledge graph construction. *arXiv preprint arXiv:2509.05392*, 2025.
- [2] Allen Institute for AI. MuSiQue-Answerable leaderboard. [https://leaderboard.allenai.org/musique\\_ans](https://leaderboard.allenai.org/musique_ans). Test-set results also listed at <https://www.sota2.com/research/sota/question-answering-on-musique-answerable-test>. Accessed 2026-09-25.
- [3] Sydney Anuyah, Mehedi Mahmud Kaushik, Krishna Dwarampudi, Rakesh Shiradkar, Arjan Durresi, and Sunandan Chakraborty. Automated knowledge graph construction using large language models and sentence complexity modelling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025.
- [4] Joao T. Aparicio, Elisabete Arsenio, Francisco Santos, and Rui Henriques. Using dynamic knowledge graphs to detect emerging communities of knowledge. *Knowledge-Based Systems*, 2024.
- [5] Zhendong Chu, Shen Wang, Jian Xie, Tinghui Zhu, Yibo Yan, Jinheng Ye, et al. Llm agents for education: Advances and applications. *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
- [6] Docker. Docker overview, 2023.
- [7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization, 2024. *arXiv:2404.16130*.
- [8] Yuyuan Feng, Zhishang Xiang, Chaobin Yang, Qichao Ma, Zerui Chen, Yujing Zhang, Ke Huang, Chuanjie Wu, Zhaoxu Liu, Yili Wang, Xin He, Jiapu Wang, Zijin Hong, Hao Chen, Yuanchen Bei, Kun Wang, Shengyuan Chen, Ningyu Zhang, Enyan Dai, et al. Graph engineering in the era of llm agents: From individual intelligence to system intelligence. *arXiv:2608.21156*, 2026.
- [9] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems 38 (NeurIPS)*, 2024.
- [10] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025. *arXiv:2502.14802*.
- [11] Haoyu Huang, Chong Chen, Zeang Sheng, Yang Li, and Wentao Zhang. Can llms be good graph judge for knowledge graph construction?, 2025.

- [12] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. Stanford University, 3rd edition, 2026. Online manuscript released January 6, 2026.
- [13] LangChain Inc. Langchain documentation. <https://docs.langchain.com/>, 2022. Accessed: 2022-05-24.
- [14] LangChain Inc. Agent middleware documentation. <https://www.langchain.com/blog/agent-middleware/>, 2025. Accessed: 2025-12-05.
- [15] Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. NV-Embed: Improved techniques for training LLMs as generalist embedding models. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.
- [16] Yen-Ting Lee, Keerthi Koneru, Zahra Moslemi, Sheethal Kumar, and Ramesh Radhakrishnan. Aema: Verifiable evaluation framework for trustworthy and controlled agentic llm systems. *arXiv preprint arXiv:2601.11903*, 2026.
- [17] Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning, 2023. *arXiv:2308.03281*.
- [18] Chen Liang, Jianbo Ye, Zhaohui Wu, Bart Pursel, and C Lee Giles. Recovering concept prerequisite relations from university course dependencies. In *Thirty-First AAAI Conference on Artificial Intelligence*, 2017.
- [19] Chenxi Liu, Yongqiang Chen, Tongliang Liu, James Cheng, Bo Han, and Kun Zhang. On the thinking-language modeling gap in large language models. *arXiv preprint arXiv:2505.12896*, 2025.
- [20] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 2024.
- [21] Rongxin Liu, Carter Zenke, Charlie Liu, Andrew Holmes, Patrick Thornton, and David J. Malan. Teaching cs50 with ai: Leveraging generative artificial intelligence in computer science education. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education*, 2024.
- [22] Congmin Min, Sahil Bansal, Joyce Pan, Abbas Keshavarzi, Rhea Mathew, and Amar Viswanathan Kannan. Towards practical graphrag: Efficient knowledge graph construction and hybrid retrieval at scale. *arXiv:2507.03226*, 2025.
- [23] Haoliang Ming, Feifei Li, Xiaoqing Wu, and Wenhui Que. Retrieval as reasoning: Self-evolving agent-native retrieval via llm-wiki. *arXiv:2605.25480*, 2026.
- [24] Belinda Mo, Kyssen Yu, Joshua Kazdan, Joan Cabezas, Proud Mpala, Lisa Yu, Chris Cundy, Charilaos Kanatsoulis, and Sanmi Koyejo. Kggen: Extracting knowledge graphs from plain text with language models, 2025.
- [25] Niklas Muennighoff, Hongjin Su, Liang Wang, Nan Yang, Furu Wei, Tao Yu, Amanpreet Singh, and Douwe Kiela. Generative representational instruction tuning. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

- [26] Davy Tsz Kit Ng, Eagle Kai Chi Chan, and Chung Kwan Lo. Opportunities, challenges and school strategies for integrating generative ai in education. *Computers and Education: Artificial Intelligence*, 8:100373, 2025.
- [27] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [28] Liangming Pan, Chengjiang Li, Juanzi Li, and Jie Tang. Prerequisite relation learning for concepts in moocs. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017.
- [29] Gowtham Ramesh, Makesh Sreedhar, and Junjie Hu. Single sequence prediction over reasoning graphs for multi-hop qa. *arXiv preprint arXiv:2307.00335*, 2023.
- [30] Simplilearn. What is aws? an ultimate guide to amazon web services, 2023.
- [31] Sharon Solomon. Frontegg - what is oauth? definition, how it works, and tips for success, 2023.
- [32] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific reports*, 9, 2019.
- [33] Harsh Trivedi, Niranjana Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- [34] Jing Wang, Zheng Li, Lei Li, Fan He, Liyu Lin, Yao Lai, Yan Li, Xiaoyang Zeng, and Yufeng Guo. Principle-guided verilog optimization: Ip-safe knowledge transfer via local-cloud collaboration. *10.48550/arXiv.2508.05675*, 2025.
- [35] Shu Wang, Yingli Zhou, and Yixiang Fang. Bookrag: A hierarchical structure-aware index-based approach for retrieval-augmented generation on complex documents. *arXiv:2512.03413*, 2025.
- [36] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.
- [37] Orion Weller, Michael Boratko, Iftexhar Naim, and Jinhyuk Lee. On the theoretical limitations of embedding-based retrieval. *arXiv preprint arXiv:2508.21038*, 2025.
- [38] C. M. Wyss. How to tame your llm: Semantic collapse in continuous systems. *arXiv preprint arXiv:2512.05162*, 2025.
- [39] Bowen Zhang and Harold Soh. Extract, define, canonicalize: An LLM-based framework for knowledge graph construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9820–9836, Miami, Florida, USA, 2024. Association for Computational Linguistics.
- [40] B. Zhao, J. Sun, B. Xu, X. Lu, Y. Li, J. Yu, M. Liu, T. Zhang, Q. Chen, H. Li, et al. Edukg: a heterogeneous sustainable k-12 educational knowledge graph. *arXiv preprint arXiv:2210.12228*, 2022.